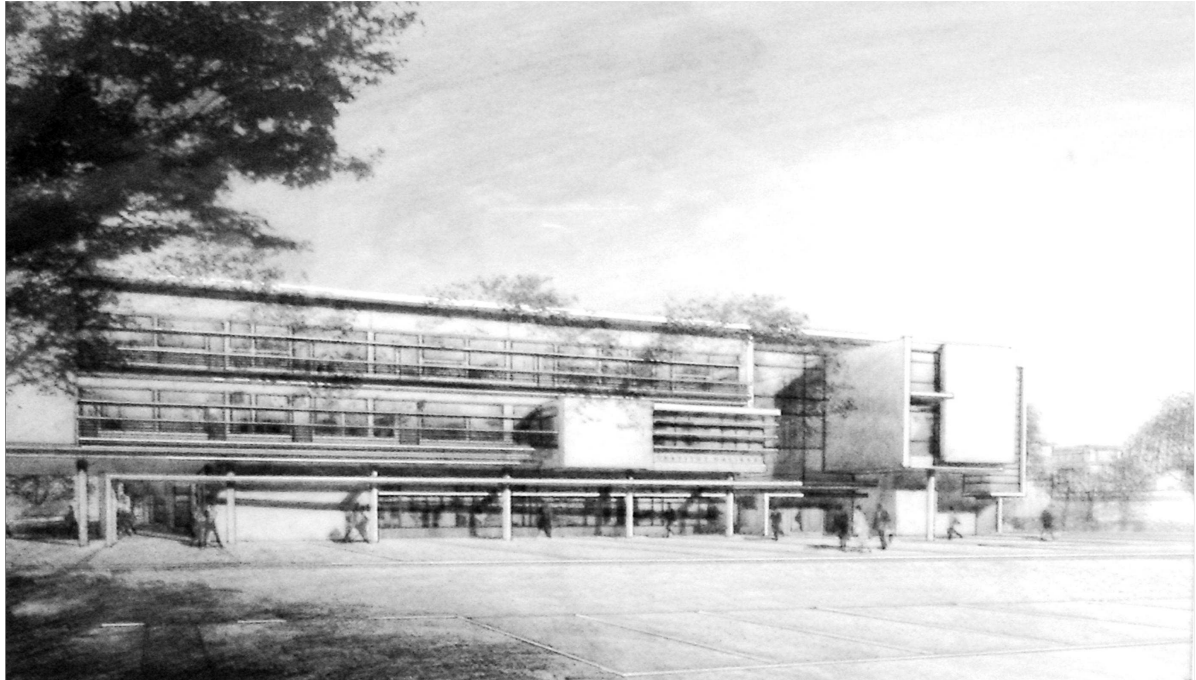




ASPECTS MATERIELS & LOGICIELS DU MICROCONTRÔLEUR PIC 16F877

RESUME INFORMATIONS PROJETS

THIEBAUT Bernard

ELEMENTS SUR LE LCD IMPLANTÉ SUR LA CARTE PICDEM 2 PLUS

Jusqu'ici les seuls **périphériques de sortie** vus étaient :

↳ **LED** : visuel,

↳ **BUZZER** : sonore.

Simples à mettre en œuvre (commande en TOR, phase d'apprentissage) mais **limités** quant à l'information fournie...

Ici, on va utiliser un **LCD** (Liquid Cristal Display) **alphanumérique** :

↳ avec 2 **lignes** de 16 **caractères**.



Plus **compliqués** à mettre en œuvre mais plus intéressants pour **visualiser des informations** comme les messages ou les résultats numériques.

ELEMENTS SUR LE LCD IMPLANTE SUR LA CARTE PICDEM 2 PLUS (suite)

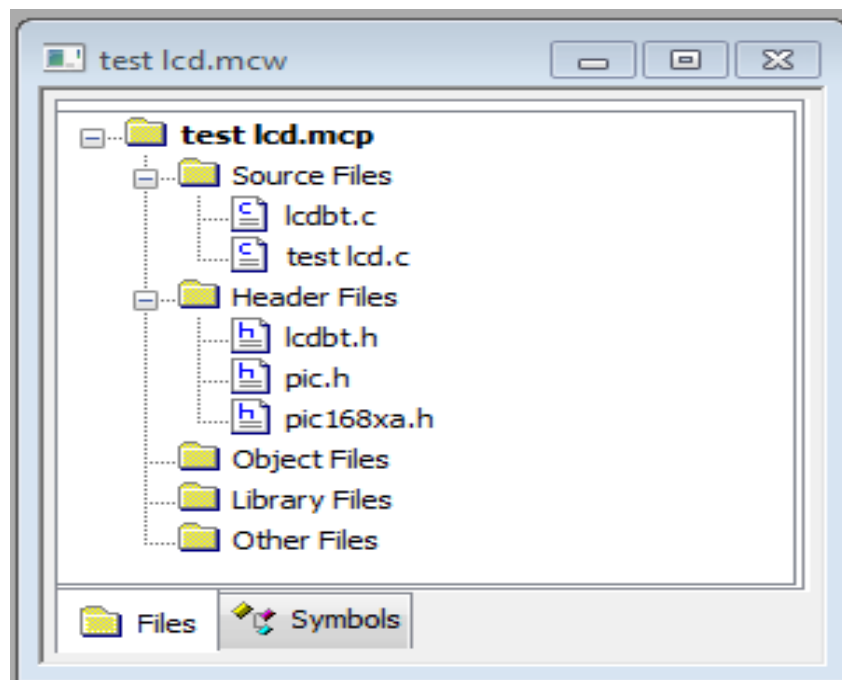
Le projet test lcd.mcp fourni

Utilisation du projet **test lcd.mcp** :

- pour donner une **trame** incomplète (et donc à terminer pour la rendre opérationnelle),
- et pour éviter de tout écrire en « partant de zéro ».

Par contre, il faut essayer de bien comprendre la structuration de ce projet au niveau :

- ↳ des **fichiers.c** associés : projet **multisources**,
- ↳ des **fichiers.h** associés : lisibilité, évolutivité,



- ↳ des **fonctions proposées** : principe du « **légo** ».

ELEMENTS SUR LE LCD IMPLANTE SUR LA CARTE PICDEM 2 PLUS (suite)

Le projet test lcd.mcp

Conseils généraux

Bien suivre la méthodologie **guidée** du texte du TP :

- **comprendre** les différentes étapes,
- **ne pas sauter** des étapes,
- **valider** son travail **au fur et à mesure ...**

Si elle est volontairement très guidée, c'est parce qu'elle pourra vous resservir de base pour implanter d'autres types d'interfaces, comme par exemple (liste non exhaustive) :

- un clavier (voir plus loin),
- un écran graphique (voir plus loin),
- un module ultrasons pour mesurer des distances (voir plus loin),
- un moteur pas à pas (voir plus loin),
- un module de reconnaissance vocale (voir plus loin),
- une caméra (projet robotique),
- un module de positionnement GPS,
- ou de façon générale, tout autre type de capteur ou d'actionneur (projet robotique)...

Les étapes d'implantation

Etape 1 : **Compréhension du hardware**

Cas idéal : ici, bien **comprendre** la documentation du **LCD**.

En voici un résumé :

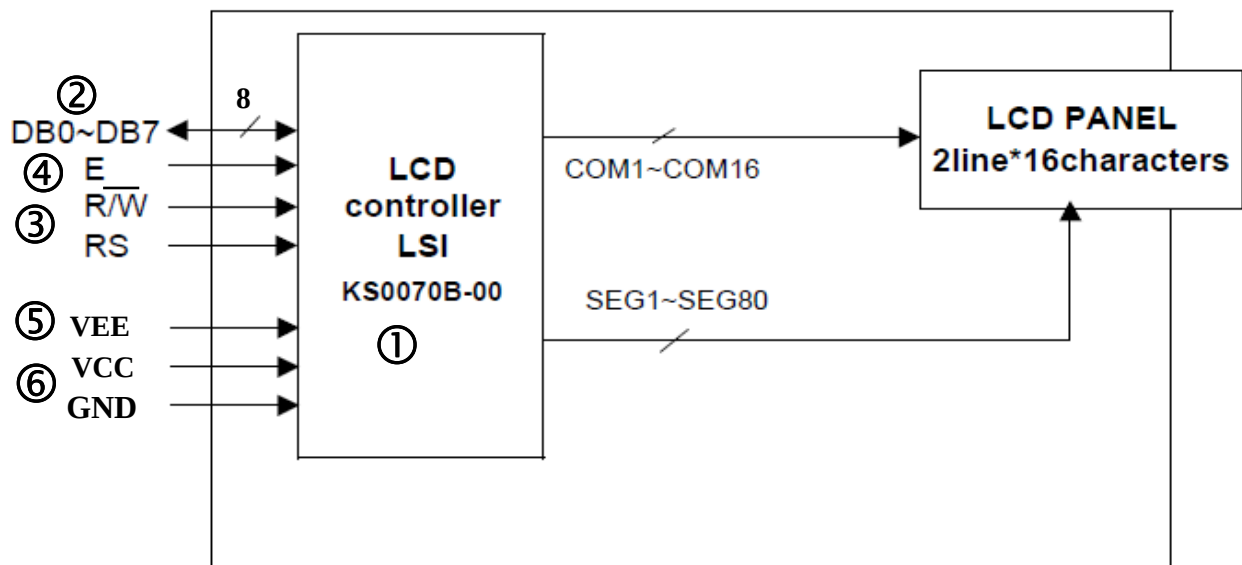


Figure III.3. (page 70 polycopié TP)

① le « cœur » du **LCD** comprenant entre autre :

deux registres accessibles en **lecture** ou **écriture** :

- ↳ un registre d'**instructions IR** ou d'**état** de taille **8** bits :
 - pour recevoir (en **écriture**) les différentes commandes : initialisation du LCD, gestion du curseur, effacement, modes de fonctionnement ...
 - pour renvoyer (en **lecture**) une info sur l'état du **LCD**.
- ↳ un registre de **données DR** de taille **8** bits aussi :
 - pour recevoir (en **écriture**) le code **ASCII** des caractères à afficher sur le LCD.

Les étapes d'implantation (suite)

Etape 1 : (suite)

② les 8 broches **d'entrée/sortie DB0 à DB7** pour **échanger** les informations (contenu de **IR** et de **DR**) avec le **PIC**,

③ les pattes d'entrée **RS** (comme **Register Select**) et **R/ $\overline{W}$** (comme **Read/Write**) du **LCD** pour accéder à ces deux registres internes :

$\begin{array}{c} \text{RS} \\ \text{R}/\overline{\text{W}} \end{array}$	0	1
0	Ecriture dans IR	Ecriture dans DR
1	Lecture de IR	Lecture de DR

Figure III.4. (page 71 photocopié TP)

④ **après avoir positionné** les entrées **RS** et **R/ $\overline{W}$** suivant ce que l'on veut faire, il faut envoyer un « pulse » sur l'entrée **E**.

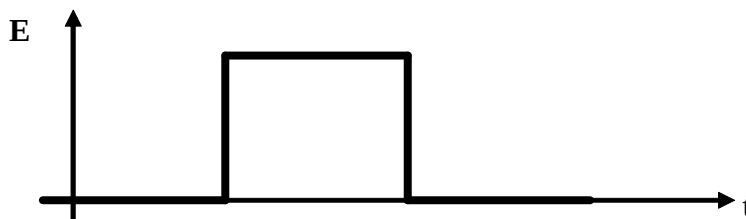


Figure III.5. (page 71 photocopié TP)

Les étapes d'implantation (suite)

Etape 1 : (suite)

Exemple de chronogrammes issus du data sheet du LCD

**Cycle d'écriture ($R/\bar{W} = 0$)
dans IR si RS=0 ou dans DR si RS=1**

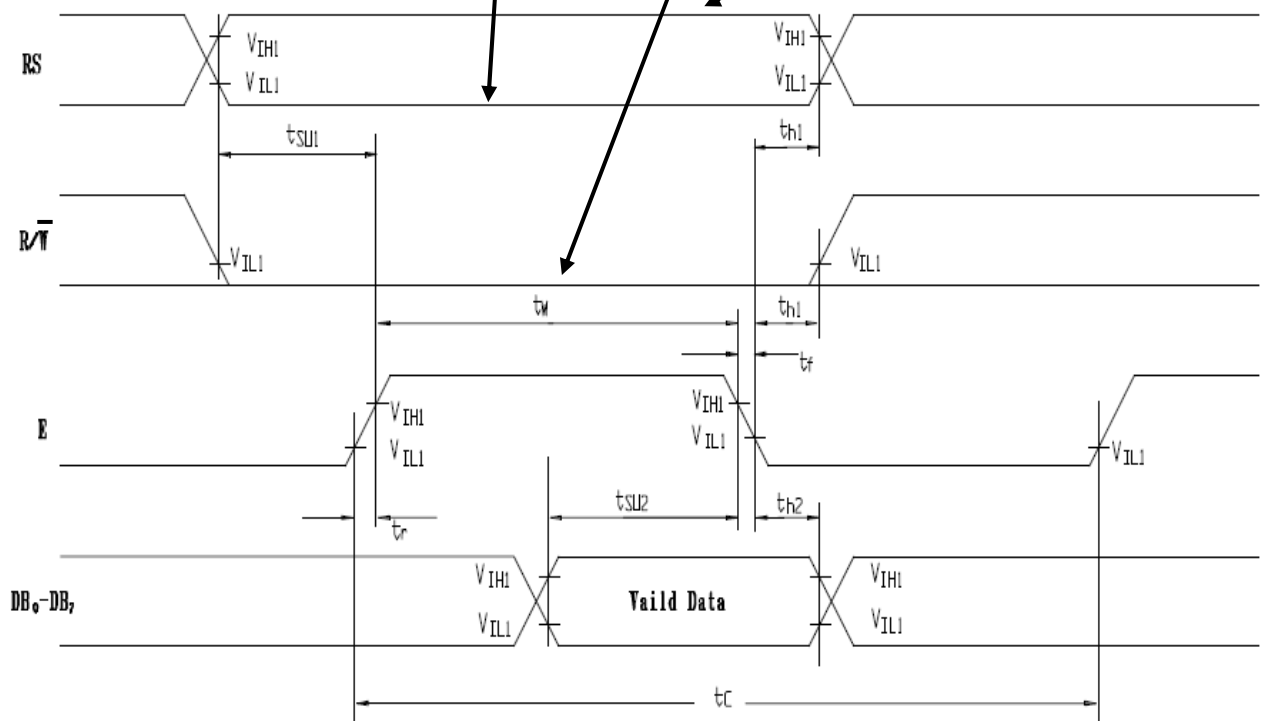


Figure III.6. (page 71 polycopié TP)

⑤ la patte d'entrée **VEE** du **LCD** pour régler le contraste,

⑥ les pattes d'entrée **VCC** et **GND** pour l'alimentation du **LCD**.

Les étapes d'implantation (suite)

Etape 2 :

Lecture du schéma d'implantation
Affectation des différentes broches
Initialisation des ports concernés du PIC

Schéma d'implantation du LCD

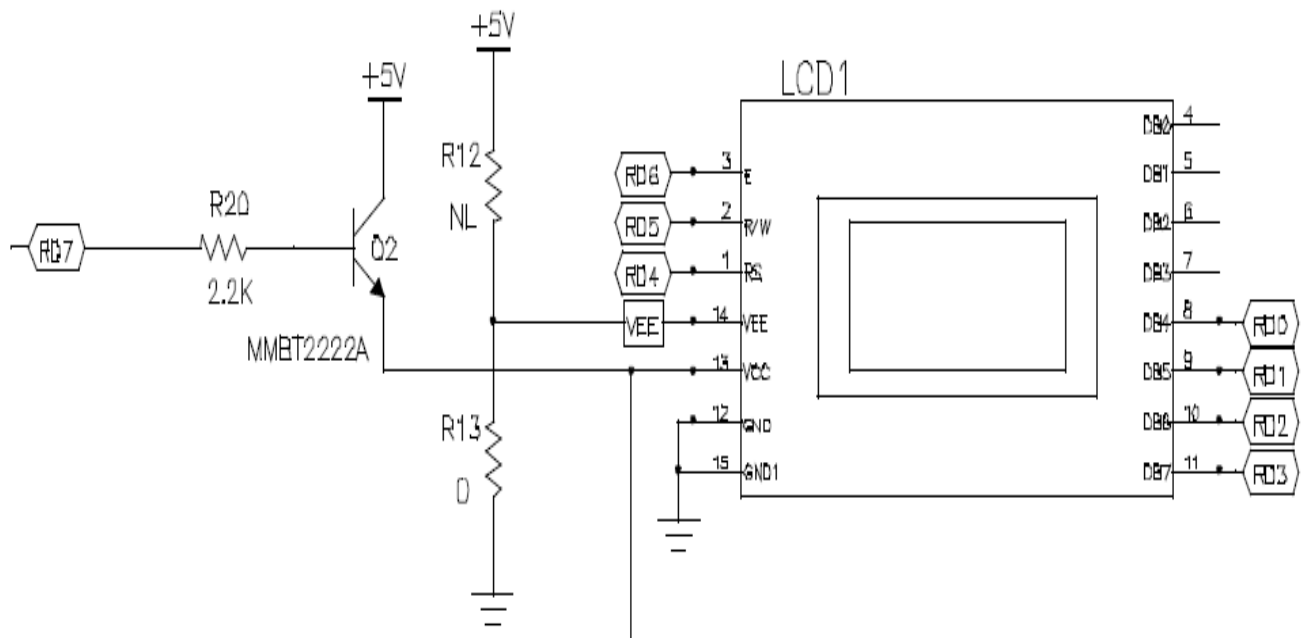


Figure III.7. (page 73 polycopié TP)

Pour faire un lien «**disible**» entre le **hardware** et le **software** :

↳ attribuer un nom **symbolique « parlant »** aux différentes broches mise en œuvre par l'interface (cf.schéma ci-dessus): dans le fichier **lcdbt.h**.

↳ écrire les **fonctions d'initialisation** des ports du PIC concernés par l'interface (direction, valeurs initiales en sortie): dans le fichier **lcdbt.c**.

Résumé du cours PIC : ANNEXE 1
(informations supplémentaires pendant manipulations TP 3)

Les étapes d'implantation (suite)

Etape 3 : Initialisation software du LCD

Respect de la **procédure préconisée** par le constructeur.

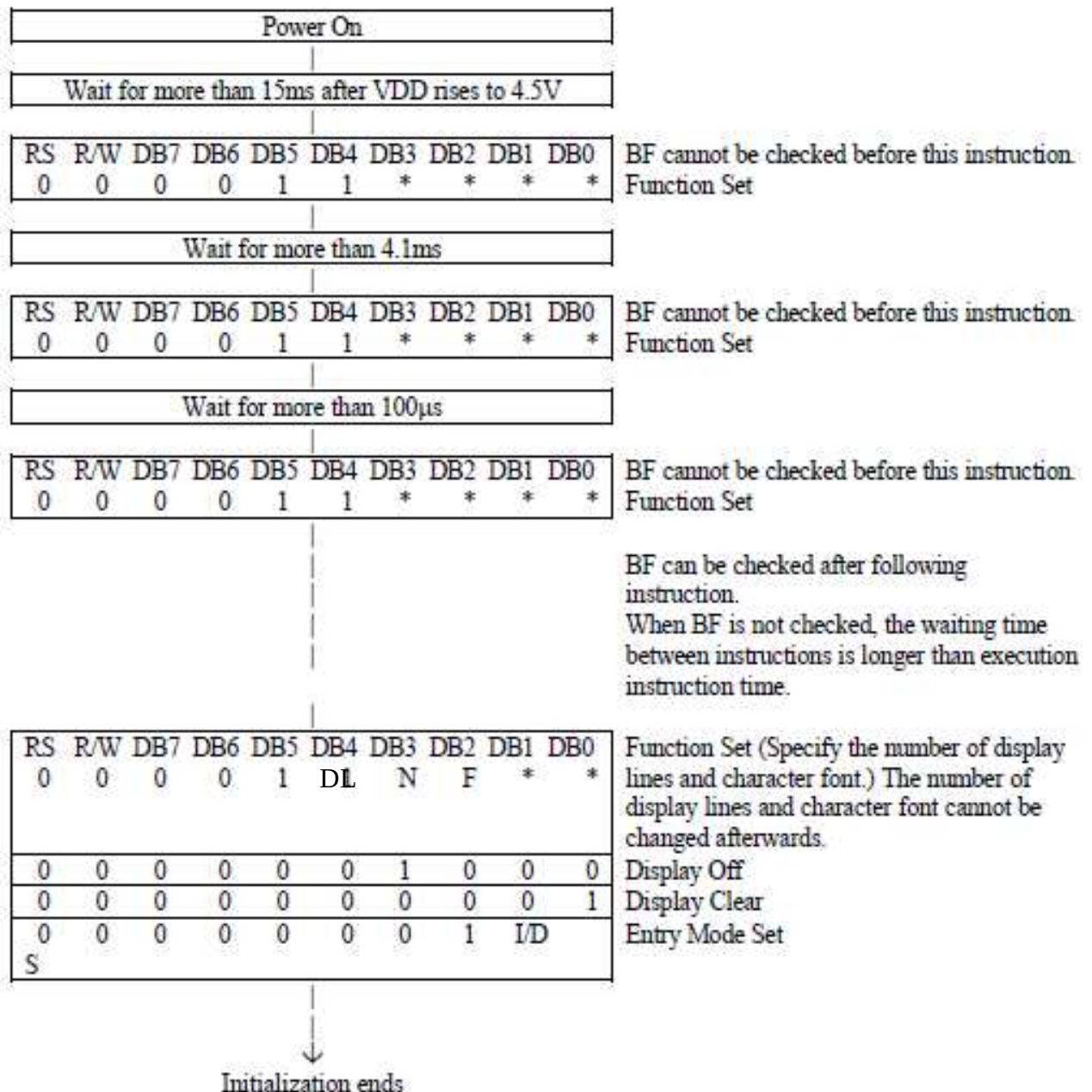


Figure III.8. (page 75 polycopié TP)

Les étapes d'implantation (suite)

Etape 3 : (suite)

C'est la fonction :

void **init_LCD** (void)

qui effectue l'initialisation du LCD.

Principales contraintes à respecter ici :

⇒ Temporisations à respecter entre chaque commande
(cf. fonction **tempo_N_ms** (int **ms**) : **TIMER0** sous pooling)

⇒ Echanges en mode **4 bits** :
(via les broches **bidirectionnelles** **RD0** à **RD3** du **PORTD**)

- d'abord le **quartet** de poids **fort**,
- puis le **quartet** de poids **faible**.

⇒ Faire le lien avec le **tableau des commandes** de la figure III.9. de la page 76 du polycopié de TP.

Objectifs de cette étape dans le TP :

Vérifier la conformité entre la fonction **init_LCD** déjà écrite et l'**organigramme** de la figure III.8. précédente.

Les étapes d'implantation (suite)

Etape 4 :

Ecriture d'un caractère sur le LCD

Dans le fichier **lcdbt.c**, il faut écrire le corps de la fonction :

void **print_char** (char)

permettant d'envoyer (et donc d'afficher) un caractère dans le registre **DR** du **LCD**.

Conseils à respecter :

⇒ **Bien comprendre** et **s'inspirer** de la fonction (aussi dans le fichier **lcdbt.c**)

void **ecriture_commande** (char)

déjà écrite pour vous et qui fait un peu la même chose mais dans le registre **IR** du **LCD**.

⇒ **Tester** la fonction **print_char** via le main du projet

test lcd.mcp

avant de passer à l'étape suivante.

Les étapes d'implantation (suite)

Etape 5:

Ecriture d'une chaîne de caractères sur le LCD

Toujours sans le fichier **lcdbt.c**, il faut écrire le corps de la fonction :

void **print_string** (char *)

permettant d'envoyer (et donc d'afficher) une chaîne de caractères sur le **LCD**.

Conseils à respecter :

⇒ **Utiliser bien sûr** la fonction :

void **print_char** (char)

écrite (et testée) à l'étape précédente.

⇒ **Tester** la fonction **print_string** via le main du projet

test lcd.mcp

avant de passer à l'étape suivante.

Les étapes d'implantation (suite)

Etape 6:

Bibliothèque de fonctions de gestion du curseur

Pour faciliter son utilisation ultérieure, il faut écrire un ensemble de fonctions (dans le fichier **lcdbt.c**) permettant de piloter plus convivialement le LCD.
(voir le cahier des charges de chacune de ces fonctions à la page 78 du polycopié de TP...)

- utilisation de la commande **SET DDRAM ADDRESS**

 ↪ pour choisir la position d'affichage sur l'écran

- utilisation de la commande **SHIFT**

 ↪ pour décaler le curseur ou l'écran

- utilisation de la commande **DISPLAY CLEAR**

 ↪ pour effacer l'écran

Conseils à respecter :

⇒ Tester **toutes ces fonctions** via le main du projet

test lcd.mcp

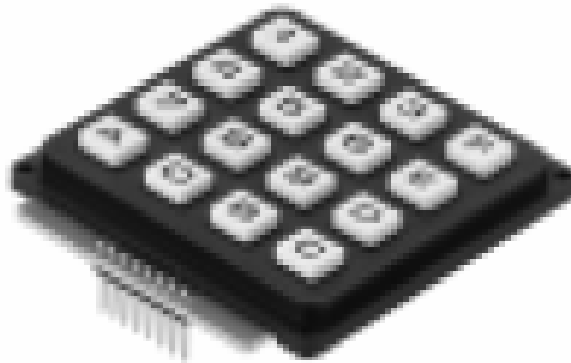
avant de passer à la suite du TP.

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER

Jusqu'ici le principal **périphérique d'entrée** vu était :

↳ le **BP (Bouton Poussoir)** à 2 états : **on** ou **off**.

Simple à mettre en œuvre (penser quand même à gérer les rebonds, inhérent à tout contact mécanique) mais **limité** quant à l'information fournie.



Plus **compliqué** à mettre en œuvre mais plus **intéressant** pour interagir avec un éventuel utilisateur :

- ↳ **saisie** de données numériques,
- ↳ **saisie** de messages,
- ↳ **choix** dans un menu,
- ↳ **sélections** diverses...

**(voir documentations complètes dans le répertoire
documentations de votre répertoire de travail)**

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

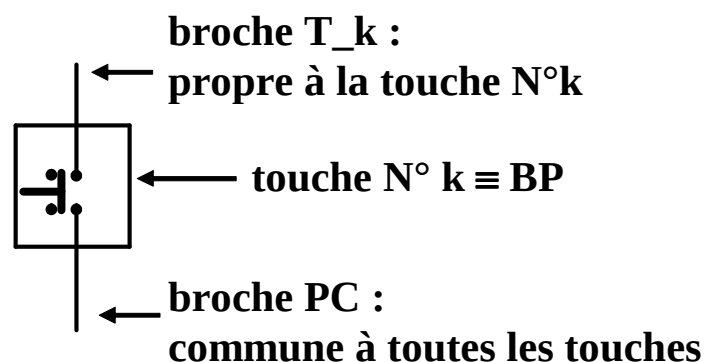
De façon générale, on aura affaire à **2 types** de claviers :

- ↳ les claviers à **Point Commun PC**,
- ↳ les claviers **matriciels**.

CLAVIERS A POINT COMMUN PC

Les claviers à **PC** se gèrent un peu comme un ensemble de **t BP** (où **t** est le **nombre de touches** du clavier) car chaque **touche N° k** est en fait un **BP** entre :

- la broche **PC** : la même pour toutes les touches et
- la broche **T_k** : propre à chaque **touche N° k**.



ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

CLAVIERS A POINT COMMUN PC (suite)

Ce clavier possède donc **t+1 broches**.

Il va donc mobiliser **t pattes du PIC**.
(t lignes de **PORT** Tout Ou Rien initialisées en **entrée**)

A charge du **programme de gestion** du clavier à PC de venir détecter le numéro de la touche enfoncée :

↳ soit par **pooling** :

En venant scruter une à une chacune des touches pour savoir si l'utilisateur a appuyé sur l'une d'entre elle.
Il faudra bien sûr toujours faire attention aux rebonds.

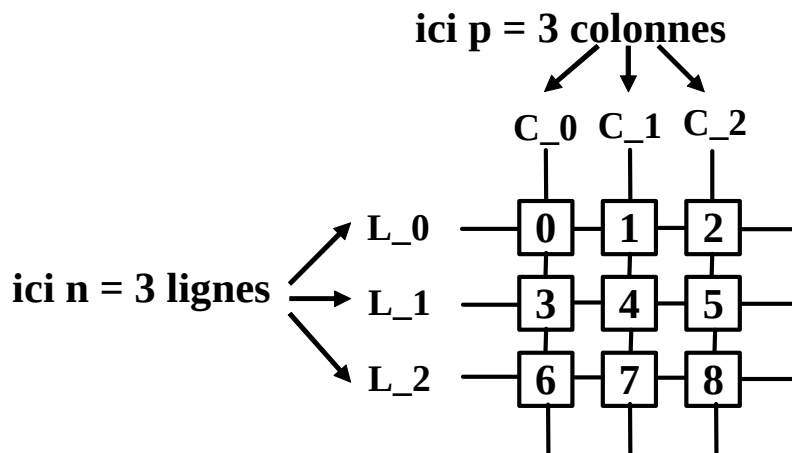
↳ soit sous **interruption** :

En utilisant par exemple une petite logique additionnelle à base de portes AND ou OR, on peut générer un signal d'interruption **INT_CLAV** qui sera actif à chaque appui sur l'une quelconque des touches.
Ce signal d'interruption pourra être envoyé sur la broche **RB0/INT** du PIC qui joue le rôle d'interruption extérieure. C'est ensuite la routine d'interruption associée qui détectera le numéro de la touche enfoncée.

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

CLAVIER MATRICIEL $n \times p$

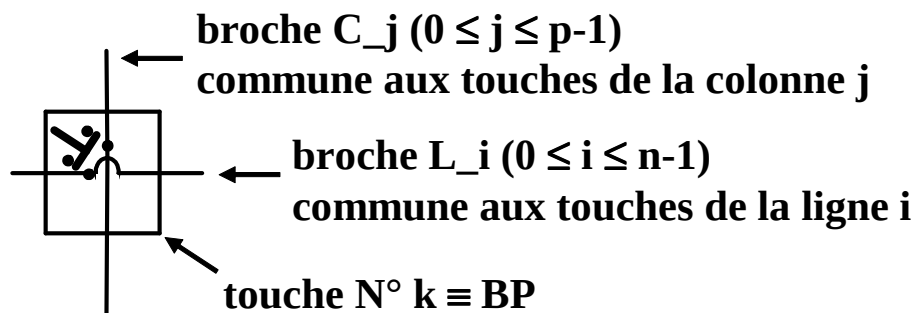
Comme son nom l'indique, on introduit la notion de **ligne** (n lignes) et de **colonne** (p colonnes).



Ce clavier comporte donc $t = n.p$ touches.

Chaque **touche** $N^{\circ}k$ est aussi un **BP** qui, enfoncé, assurera la liaison entre :

- la ligne L_i où se trouve la touche $N^{\circ}k$ et
- la colonne C_j où se trouve la touche $N^{\circ}k$.



On a donc : $k = j + i * p$.

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

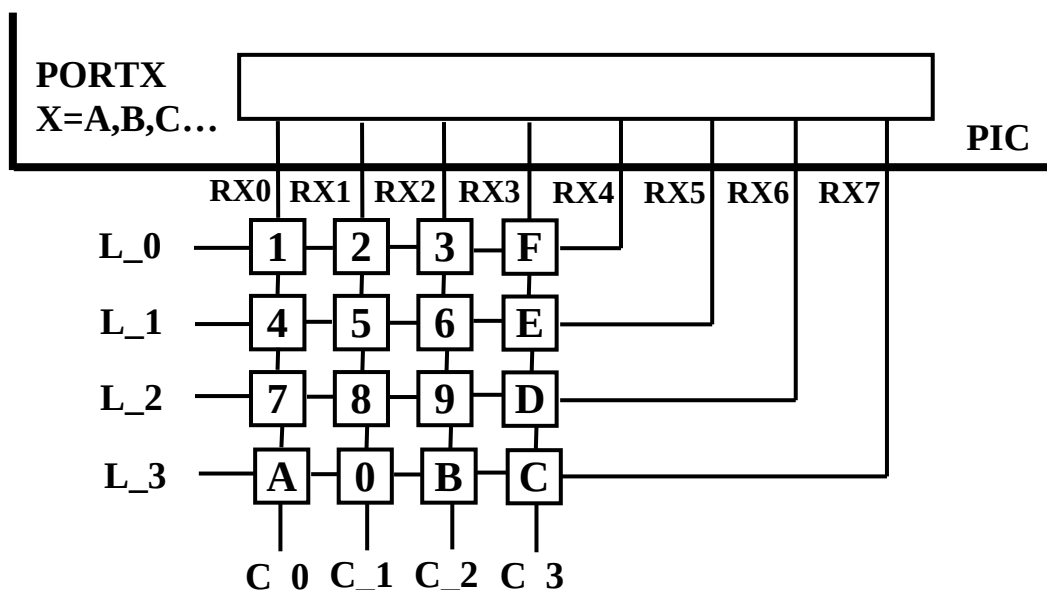
CLAVIER MATRICIEL (suite)

Ce clavier (qui possède donc **n+p broches**) va donc mobiliser **n+p pattes du PIC**, ce qui est plus **faible** et donc un **avantage** par rapport à un clavier de type **PC**.

Par contre, la gestion logicielle d'un clavier matriciel est plus délicate. La méthode d'implantation d'un clavier matriciel est comparable à celle utilisée dans le TP N°3 pour le LCD alphanumérique.

En voici les grandes lignes et étapes.

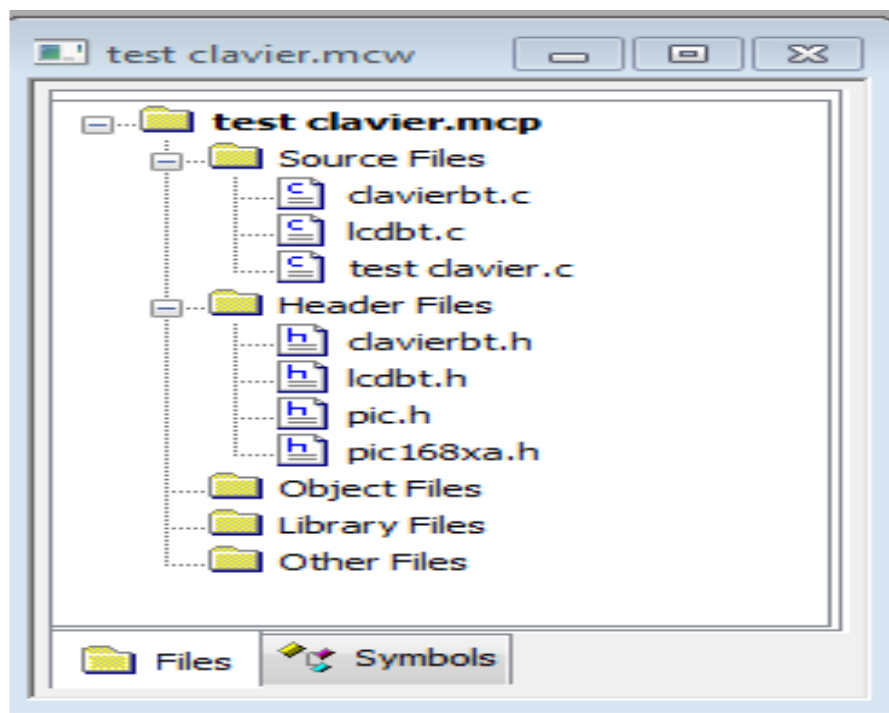
ETAPE 1 : IMPLANTATION HARDWARE (cas fréquent d'un clavier 4 × 4)



ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

ETAPE 2 : STRUCTURATION DU PROJET test clavier.mcp

Voici une structuration « **professionnelle** » possible pour un projet **test clavier.mcp** incluant un **clavier** et utilisant aussi le **lcd alphanumérique** présent sur la carte **PICDEM 2 PLUS** :



L'objectif est **d'écrire** et **de tester** les fichiers :

- ↳ **clavierbt.h** : lien avec le **hardware**,
- ↳ **clavierbt.c** : incluant les fonctions de **bas niveau** permettant de gérer le clavier.

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

ETAPE 3 : ECRITURE DU FICHIER `clavierbt.h`

Contenant entre autre :

- une liste de **#define** comme par exemple :

```
// affectation des broches RXi du PIC
#define      C_0      RX0
...

// pour la directionnalité des lignes du PORTX
#define      TRIS_C_0  TRISX0
...
```

- la définition d'un tableau pour les codes des touches déclaré par exemple en EEPROM (directive **const**) pour ne pas prendre de place en RAM :

```
const unsigned char code_touche[ ] =
{'1','2','3',.....,'C'}
```

- et d'autres déclarations spécifiques au clavier suivant les besoins ...

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

ETAPE 4 : ECRITURE DU FICHIER `clavierbt.c`

C'est le cœur du projet !!!

Les pattes **RX0** à **RX3** (**colonnes**) sont mises en **sortie**.

Les pattes **RX4** à **RX7** (**lignes**) sont mises en **entrée**.

↳ écrire une fonction de prototype :

void **init_clavier** (void) qui effectue ces affectations.

  **Attention : oubli fréquent...**

Penser aussi à rajouter une résistance de « pull-up » vers le 5V pour fixer la tension au repos sur les broches **RX4** à **RX7**... sinon ça ne fonctionnera pas !!!

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

ETAPE 4 : ECRITURE DU FICHIER `clavierbt.c`

Voici le principe de la scrutation du clavier :

- on active la **colonne 0** en sortant respectivement **1 1 1 0** sur les broches **RX3** à **RX0**,
- on lit ensuite le **PORTX** :
 - si une des entrées de ligne **RXj** (pour j= 4 à 7) est à **0**, c'est qu'on a appuyé sur la touche se trouvant à l'intersection de la colonne 0 et de cette ligne. Ce qui permet de trouver le N° de la touche enfoncée et d'arrêter la scrutation,
 - si toutes les entrées **RXj** (pour j=4 à 7) sont à **1**, alors on passe à la colonne suivante (en sortant respectivement **1 1 0 1** sur les broches **RX3** à **RX0**) et on recommence, jusqu'à la **colonne 3**.

Si à la fin de la scrutation des 4 colonnes, aucune entrée **RXj** n'est passée à **0**, c'est qu'aucune touche n'a été enfoncée.

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

ETAPE 4 : ECRITURE DU FICHIER `clavierbt.c` (suite)

Il sera donc au final et par exemple intéressant d'écrire les fonctions prototypes suivantes :

```
// fonction kbhit qui renvoie :  
// - 0 si aucune touche n'est enfoncée  
// - 1 si une touche enfoncée.  
char kbhit (void)
```

```
// fonction getkey qui renvoie :  
// - le code de la touche si une touche est enfoncée,  
// - 0 sinon.  
char getkey (void)
```

ETAPE 5 : ECRITURE DU FICHIER `test clavier.c`

C'est la partie la plus facile.

Elle contient un main qui par exemple affichera sur le LCD alphanumérique tout caractère frappé au clavier.

ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite)

ETAPE 6 : TESTS ET VALIDATION FINALE

A vous de jouer...



Attention : problème possible...

Si vous utilisez le **PORTB** pour brancher votre clavier, **n'oubliez pas** qu'en phase de débogage avec ICD3, les broches **RB6** et **RB7** servent au **dialogue avec ICD3**.

Et lorsque tout fonctionnera correctement, vous pourrez avantageusement réutiliser vos fichiers `clavierbt.h` et `clavierbt.c` que vous ajouterez à tous vos projets utilisant ce clavier.

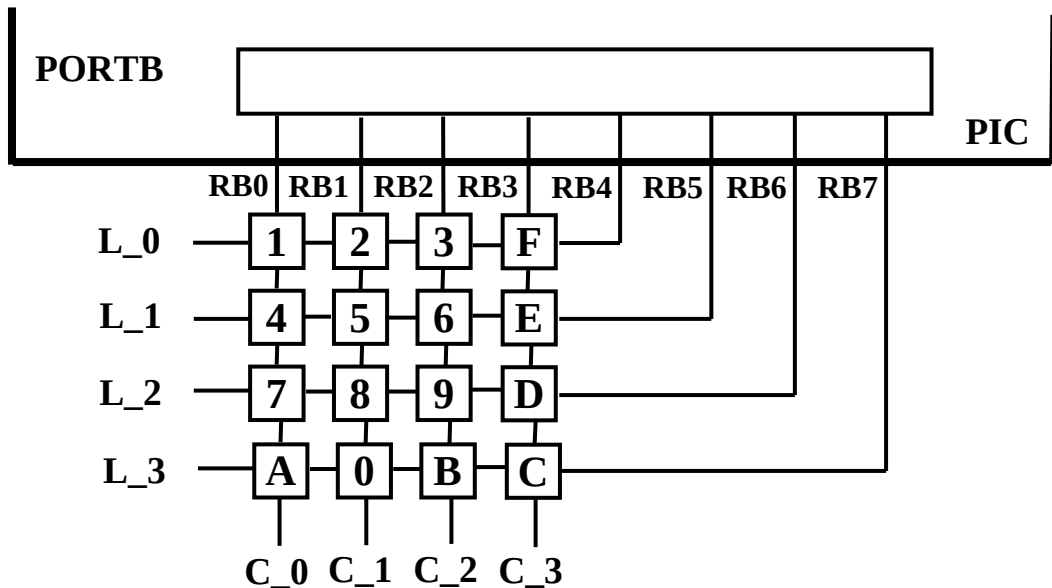
Et le tour est joué !!!

Les deux pages suivantes peuvent aussi vous être utiles !!!

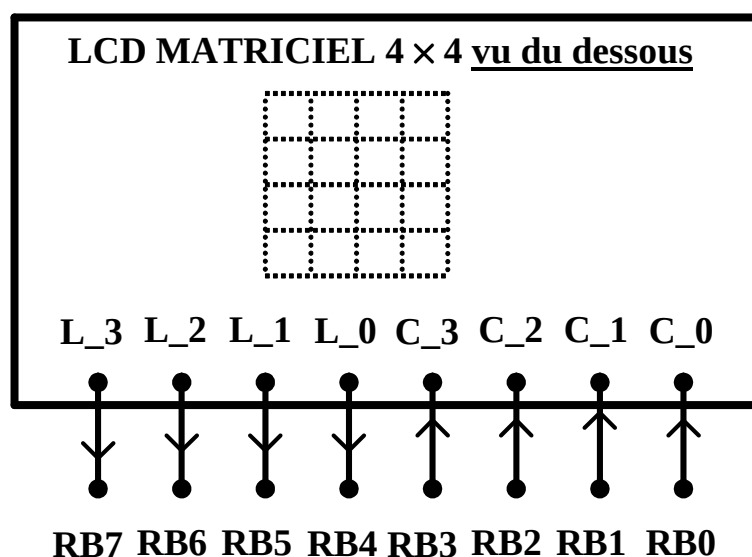
ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite et fin)

DERNIERE AIDE

Si vous utilisez le **PORTB** pour relier votre clavier matriciel, conformément au schéma ci-dessous :



Ce qui correspond au câblage suivant :



ELEMENTS SUR L'IMPLANTATION D'UN CLAVIER (suite et fin)

DERNIERE AIDE

Vous pouvez alors utiliser la procédure suivante sous **MPLAB** pour tester votre hardware :

- **File Import...** et aller chercher **dans votre répertoire** de travail dans le sous répertoire : **fichiers HEX OK/source en C** le fichier : **test clavier ICD3.hex** (c'est un exécutable)
Puis choisir l'outil **MPLAB ICD3** par : **Debugger Select Tool**, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.
Et enfin lancer ce programme : comme indiqué précédemment seules les 2 premières lignes du clavier fonctionneront.

- ou alors faire la même chose avec le fichier : **lcd clavier flash.hex**
qu'il faut par contre **programmer** dans l'EEPROM du PIC par : **Programmer Select Tool MPLAB ICD3** et envoyer sur l'ICD3 par **Programmer Program**.
Retirer alors le connecteur **RJ11** et vérifier que votre clavier complet fonctionne.

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE

En TP, nous avons vu un **LCD alphanumérique** : lorsqu'on lui envoie le code ASCII d'un caractère, il est capable d'allumer les bons pixels pour visualiser ce caractère sur l'écran (car il possède en interne une ROM de caractères faisant le lien entre ce code ASCII reçu et les pixels à allumer ou éteindre en conséquence).

Pour un **LCD graphique**, la problématique est tout autre car on peut avoir accès à tous les pixels de ce LCD pour les allumer ou les éteindre à volonté. Mais les fonctions de gestion de bas niveau seront plus difficiles à écrire.

On va présenter ici le LCD **DMX 973** fabriqué par la société **LASCAR ELECTRONICS**.



**(voir documentations complètes dans le répertoire
documentations de votre répertoire de travail)**

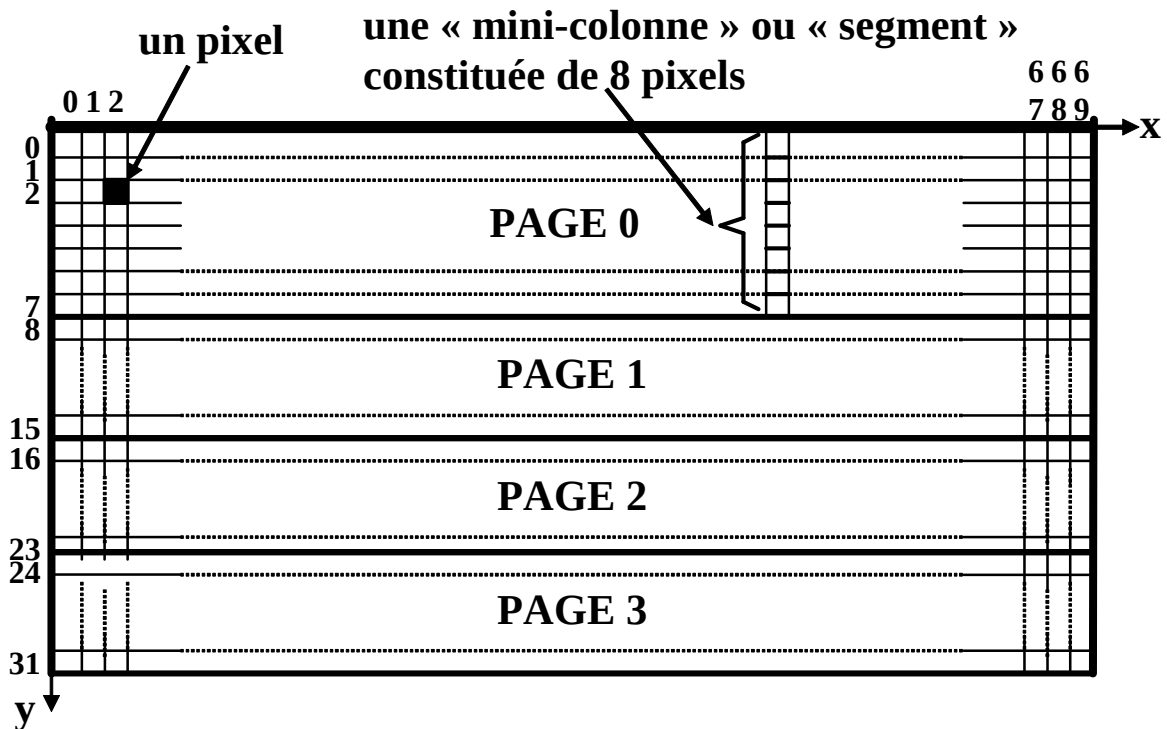
ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

RESOLUTION $70 \times 32 = 2240$ pixels

L'écran comprend **32 lignes** (ligne_0 à ligne_31) de **70 pixels** chacune (colonne_0 à colonne_69)

ORGANISATION DE L'ECRAN

L'écran est divisé en **4 pages** (page_0 à page_3)



Chaque page a une « hauteur » de 8 pixels et correspond à **70 « mini-colonnes »** ou « **segment** » de taille 8 pixels (soit 8 bits) chacune.

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

BROCHAGE

Le brochage du LCD graphique est « quasiment » compatible avec le standard LCD alphanumérique.

Ce qui présente l'avantage de pouvoir remplacer l'un par l'autre sans modification hardware.

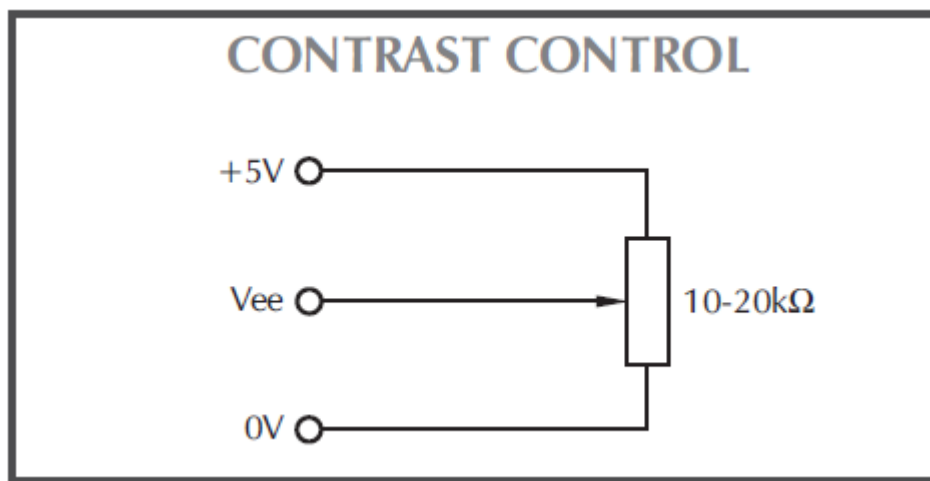
1. 0V.	}Power supply.
2. +5V.	
3. V _{ee} .	LCD contrast control input (see diagram).
4. RS.	Register Select input.
5. R/ $\overline{W}$	Read/Write input.
6. E	Enable signal.
7-14. D0-D7.	8Bit bi-directional μ P data bus.
15. LMP-.	}Backlight supply, maximum 90mA.
16. LMP+.	

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

BROCHAGE (suite)

Broche 3. : Vee

Pour régler le contraste, avec un simple potentiomètre.



Broche 4. : RS

De façon analogue à un LCD alphanumérique, permet de sélectionner :

- ↳ soit le registre **de commande** : **RS = 0**,
- ↳ soit le registre **de données** : **RS = 1**.

Broche 5. : R / $\overline{W}$

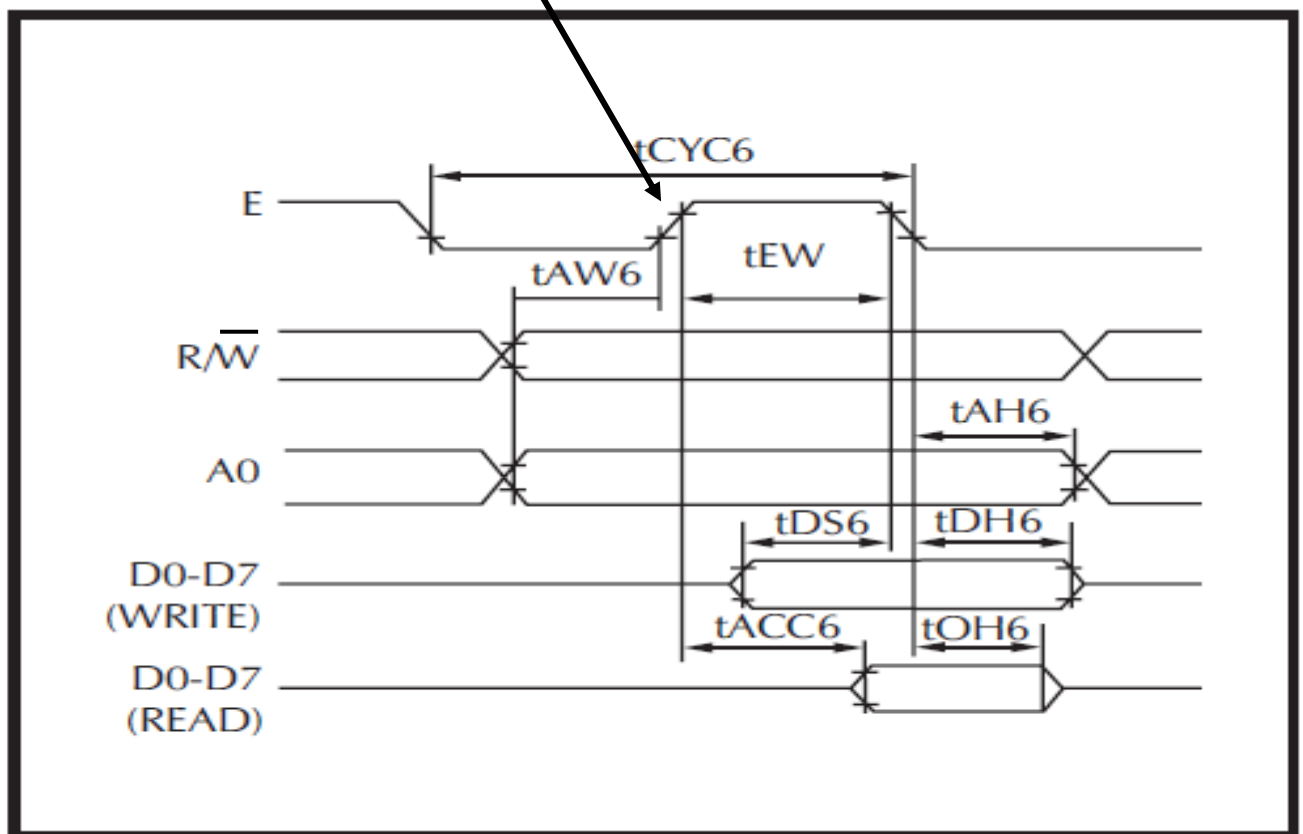
Pour différencier une **lecture** et une **écriture**.

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

BROCHAGE (suite)

Broche 6. : E

De façon analogue à un LCD alphanumérique, cette broche sert, en la « **pulsant** », à valider un cycle de lecture ou d'écriture



Broches 7. à 14. : D0 à D7

Pour les échanges en **lecture** ou en **écriture**.

Résumé du cours PIC : ANNEXE 3

(informations supplémentaires pour des projets)

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

LISTE DES INSTRUCTIONS

COMMANDS		CODE											FUNCTION	
		A0	RD	WR	D7	D6	D5	D4	D3	D2	D1	D0		
1	Display ON/OFF	0	1	0	1	0	1	0	1	1	1	0/1	Turns all display on or off, independently of display RAM data or internal status. 1: ON 0: OFF (Power saving mode with static drive on).	
2	Display Start Line	0	1	0	1	1	0	Display Start Address (0-31)				Specifies RAM line corresponding to upper most line (COM0) of display.		
3	Set Page Address	0	1	0	1	0	1	1	1	0	Page (0-3)		Sets display RAM page in page address register.	
4	Set Column (Segment)	0	1	0	0	Column Address (0-79)						Sets display RAM column address in column address register.		
5	Read Status	0	0	1	B U S Y	A D S C Y	O D / O F F	R E S E T	0	0	0	0	Reads the following status; BUSY 1: Internal operation, 0: Ready ADC 1: CW output (forward), 0: CCW output (reverse) ON/OFF 1: Display off, 0: Display on RESET 1: Being reset 0: Normal	
6	Write Display Data	1	1	0	Write Data						Writes data from data bus into display RAM.			Display RAM location whose address has been preset is accessed. After access, the column address is incremented by 1.
7	Read Display Data	1	0	1	Read Data						Reads data from display RAM onto data bus.			
8	Select ADC	0	1	0	1	0	1	0	0	0	0	0/1	Used to invert relationship of assignment between display RAM column addresses and segment driver outputs. 0: CW output (forward) 1: CCW output (reverse).	
9	Static Drive ON/OFF	0	1	0	1	0	1	0	0	1	0	0/1	Selects normal display or static driving operation. 1: Static drive (power saving mode). 0: Normal driving.	
10	Read Modify Write	0	1	0	1	1	1	0	0	0	0	0	Increments column address counter by 1 when display data is written. (This is not done when data is read.)	
11	END	0	1	0	1	1	1	0	1	1	1	0	Clears Read Modify Write mode.	
12	Reset	0	1	0	1	1	1	0	0	0	1	0	Sets display start line register on the first line. Also sets column address counter and page address counter to 0.	

R/W	RS = A0	
	0	1
0	Ecriture dans le registre de commande : instructions 1 à 4 et 8 à 12	Ecriture dans le registre de données : instruction 6
1	Lecture du registre de commande : instruction 5	Lecture du registre de données : instruction 7

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

Exemples d'utilisation :

↳ **instruction 2 :**

choix de la ligne qui sera affichée en haut de l'écran
(permet par exemple de faire du « scrolling » d'écran)

↳ **instruction 3 :**

sélection page xx : **00** pour la page **0**,
 01 pour la page **1**,
 10 pour la page **2**,
 11 pour la page **3**.

écrire **0b 1011 10xx** dans le registre de commande

Grâce à cette instruction, le LCD tient à jour un **pointeur de page**.

↳ **instruction 4 :**

sélection « mini-colonne » xxx xxxx (de 0 à 69)

écrire **0b 0xxx xxxx** dans le registre de commande

Grâce à cette instruction, le LCD tient à jour un **pointeur de « mini-colonne »**.

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

Exemples d'utilisation (suite) :

↪ instruction 6 :

pour **donner une valeur** sur 8 bits **D7-D0** à la « mini-colonne » pointée par les 2 instructions précédentes :

- si bit à **1** : pixel **allumé**,
- si bit à **0** : pixel **éteint**.

D0 correspond au pixel de la ligne de plus petit numéro dans la page concernée et **D7** au pixel de la ligne de plus grand numéro dans cette même page.

↪ instruction 7 :

pour **lire la valeur** sur 8 bits de la « mini-colonne » pointée par les instructions 3 et 4 précédentes :

- si pixel **allumé** : bit à **1**,
- si pixel **éteint** : bit à **0**.

Notons que pour les accès au registre de données par les instructions **6** et **7**, le pointeur de « mini-colonne » est automatiquement incrémenté, ce qui permet d'enchaîner plusieurs accès successifs sans mettre à jour ce pointeur.

Les autres instructions servent à configurer le LCD.

↪ exemple : instruction 8 :

permutation des colonnes (la dernière devient la première, l'avant-dernière la deuxième...) pour faire un effet « miroir ».

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

Voici enfin une liste (non exhaustive) de fonctions de « bas niveau » qu'il est intéressant d'écrire pour piloter le LCD graphique.

Comme d'habitude, il faut penser d'abord à la **structuration** du projet, pour le rendre réutilisable dans un autre contexte.

(s'inspirer de la méthode déjà utilisée pour le LCD alphanumérique ou pour le clavier matriciel...)

Ces fonctions seront donc mises dans un fichier **lcdgraphiquebt.c** qu'il faudra attacher à tous les projets utilisant ce LCD.

Sans oublier d'écrire un fichier **lcdgraphiquebt.h** pour les aspects hardware...

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

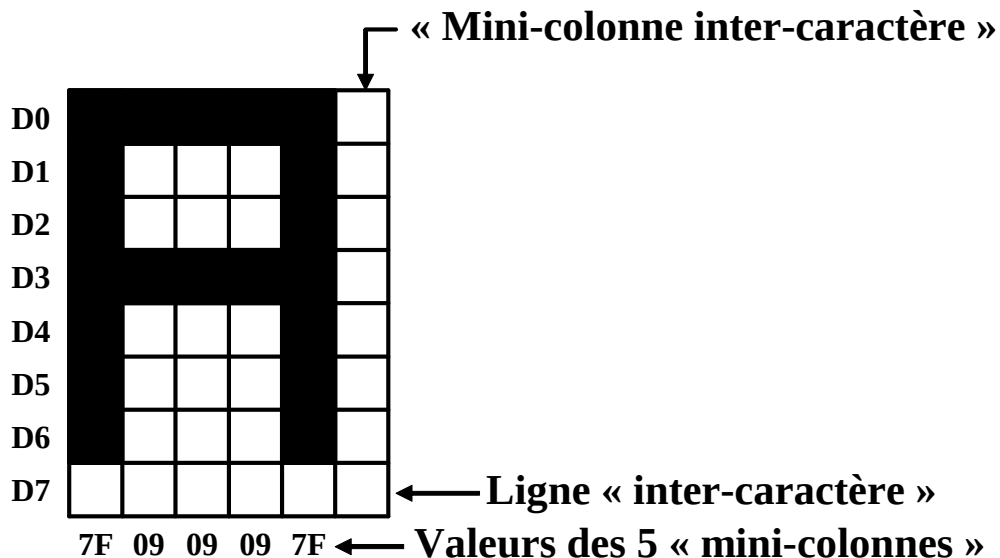
```
// *****  
// envoi d'une commande de valeur commande dans le  
// registre de commande  
// contrainte : respect des chronogrammes précédents  
void write_commande (unsigned char commande)  
// *****  
  
// *****  
// envoi d'une donnée de valeur data dans le  
// registre de données  
// data représente la valeur d'une « mini-colonne »  
// (pixels à allumer ou à éteindre)  
// contrainte : respect des chronogrammes précédents  
void write_data (unsigned char data)  
// *****  
  
// *****  
// lecture d'une donnée dans le registre de données  
// la valeur retournée est la valeur de la « mini-colonne »  
// (pixels allumés ou éteints)  
// contrainte : respect des chronogrammes précédents  
unsigned char read_data (void)  
// *****
```

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

```
// *****
// écriture d'un caractère de code ASCII car sur le LCD
// principe : voir ci-dessous
// penser à utiliser la fonction write_data précédente ...
void write_char (unsigned char car)
// *****
```

Le caractère 'A' en font 5 × 7

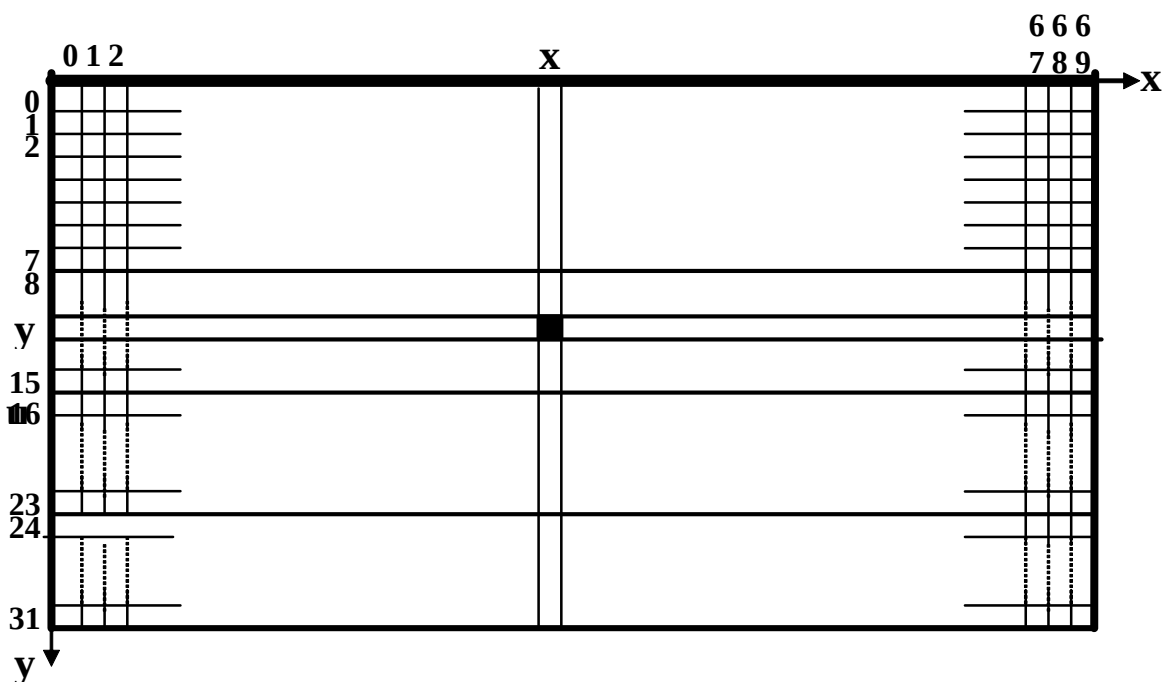


Remarque : on pourrait aussi passer en paramètre la position où on veut afficher ce caractère : sa page et sa « mini-colonne » de début dans cette page...

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

```
// *****
// allumage (si etat = 1) ou extinction (si etat = 0) d'un
// pixel de coordonnées x et y sur le LCD
// principe : voir ci-dessous
void pixel (unsigned char x, unsigned char y, char etat)
// *****
```



Il faut donc, à partir des paramètres x et y :

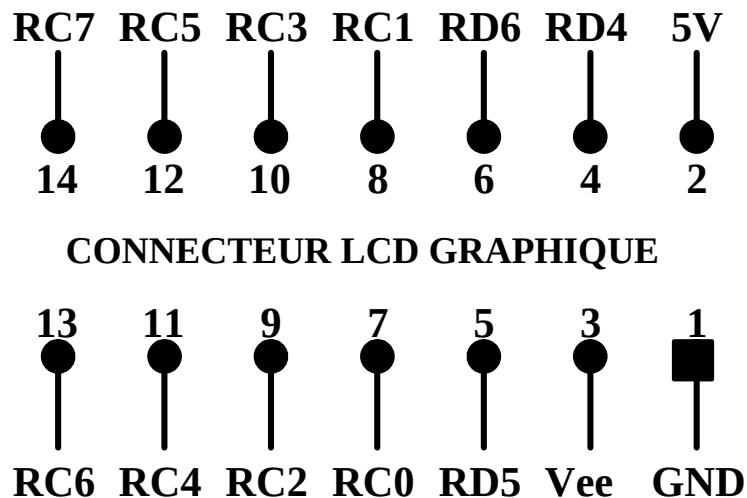
- trouver la page où se trouve le pixel concerné,
- trouver dans cette page la ligne où se trouve ce pixel,
- allumer ce pixel si $etat = 1$ ou l'éteindre si $etat = 0$.

Les deux pages suivantes peuvent aussi vous être utiles !!!

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite et fin)

DERNIERE AIDE

Si vous utilisez les **PORTC** et **PORTD** pour relier votre LCD graphique, conformément au schéma ci-dessous :



alors vous pouvez le tester sous **MPLAB** en faisant :

File Import... et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

fichiers HEX OK/source en C le fichier :

lcd graphique ICD3.hex (c'est un exécutable)

Puis choisir l'outil **MPLAB ICD3** par :

Debugger Select Tool, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.

Et enfin lancer ce programme.

ELEMENTS SUR L'IMPLANTATION D'UN LCD GRAPHIQUE (suite et fin)

DERNIERE AIDE

Si, de plus, vous disposez d'un clavier matriciel câblé comme indiqué dans **l'annexe 2**, vous pouvez alors utiliser la procédure suivante sous **MPLAB** pour tester votre clavier et votre LCD graphique :

File Import... et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

fichiers HEX OK/source en C le fichier :

memospeed flash.hex (c'est un exécutable)

qu'il faut par contre **programmer** dans l'EEPROM du PIC par : **Programmer Select Tool MPLAB ICD3** et envoyer sur l'ICD3 par **Programmer Program**.

Retirer alors le connecteur **RJ11** et vérifier que cette petite application fonctionne correctement.

ELEMENTS SUR L'IMPLANTATION D'UN EMETTEUR/RECEPTEUR A ULTRA SONS

Le but est ici **d'envoyer une onde sonore** à un instant **t_i** connu en utilisant un **émetteur à ultra sons**.

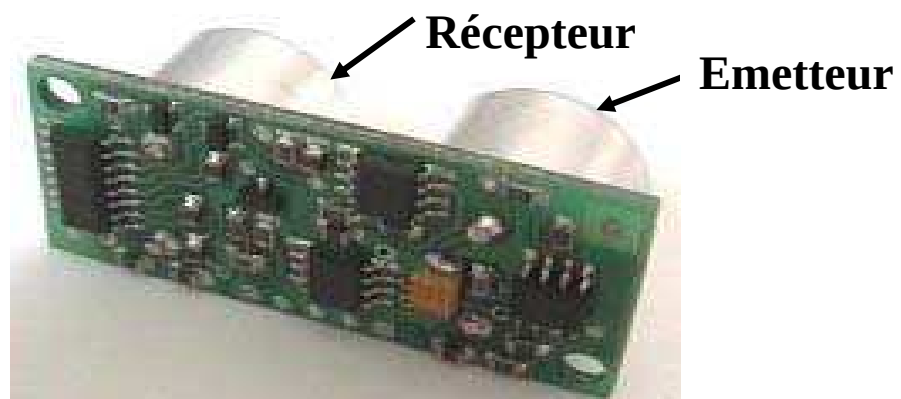
Cette onde se **propage** dans l'air jusqu'à un obstacle (situé à une distance **d**) qui la **réfléchit**.

L'onde réfléchie (écho) revient vers le **récepteur** qui peut détecter l'instant **t_f** de son arrivée.

En calculant la durée $t_f - t_i$ on peut, connaissant la vitesse de déplacement du son dans l'air, on peut **en déduire** la distance **d** de l'obstacle.

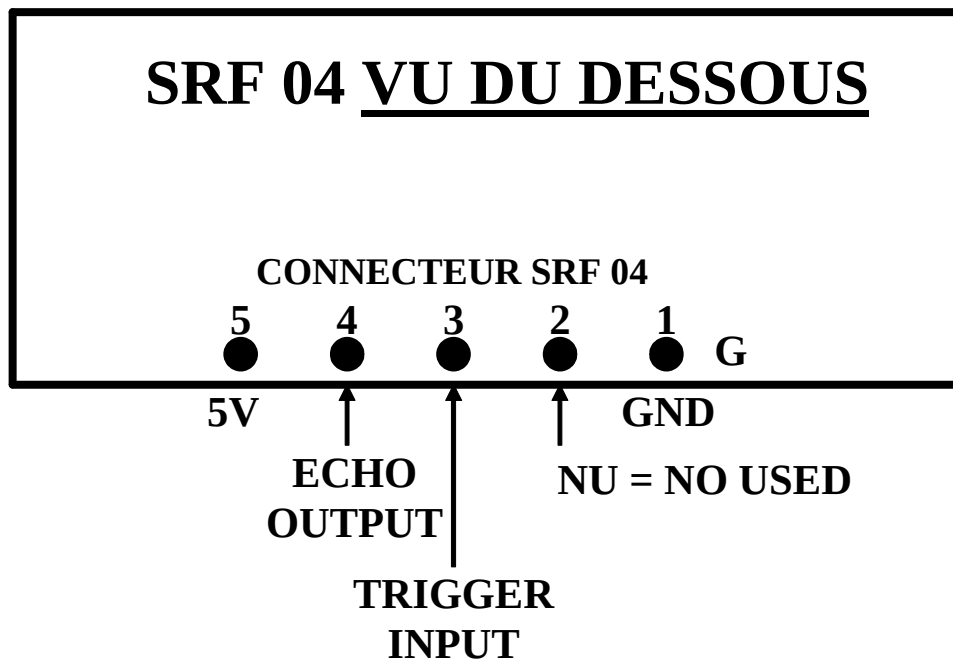
C'est le principe de base d'un **télémètre à ultra sons**.

L'émetteur à ultra sons présenté ici est le **SRF 04**.



**(voir documentations complètes dans le répertoire
documentations de votre répertoire de travail)**

ELEMENTS SUR L'IMPLANTATION D'UN EMETTEUR/RECEPTEUR A ULTRA SONS (suite) BROCHAGE DU SRF 04



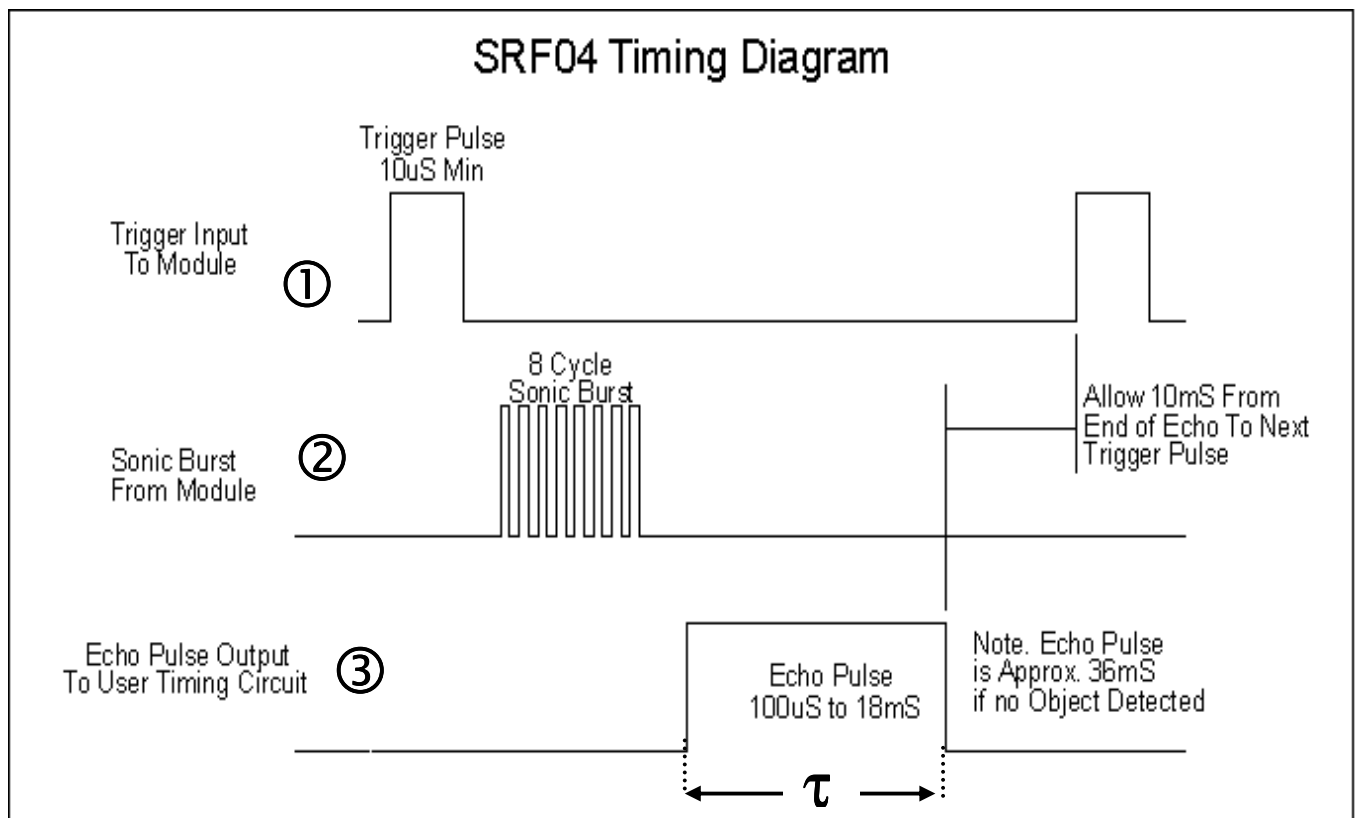
A quoi servent ces broches ?

Par rapport au principe de la mesure de distance vu précédemment, une partie du travail est effectué par le module **SRF 04**.

Pour s'en convaincre et comprendre l'utilité des broches **TRIGGER INPUT** et **ECHO OUTPUT**, il faut examiner les chronogrammes constructeur présentés à la page suivante.

ELEMENTS SUR L'IMPLANTATION D'UN EMETTEUR/RECEPTEUR A ULTRA SONS (suite)

CHRONOGRAMMES CONSTRUCTEUR



① : envoi d'un **pulse** d'une durée supérieure à $10\ \mu\text{s}$ sur l'**entrée TRIGGER** du **SRF 04**,

② : en réponse, l'émetteur du **SRF 04** émet une salve de 8 « burst » ultra sons à une fréquence de **40 kHz**.

③ : signal **en créneau** généré sur la **sortie ECHO** du **SRF 04** de durée τ est proportionnelle à la distance **d**.

ELEMENTS SUR L'IMPLANTATION D'UN EMETTEUR/RECEPTEUR A ULTRA SONS (suite)

UTILISATION DU SRF 04

Le principe est assez simple et comporte 5 étapes :

① le **PIC** envoie un pulse d'une durée supérieure à $10\ \mu\text{s}$ sur l'entrée **TRIGGER** du **SRF 04**.

② il faut ensuite, via la fonction **INPUT CAPTURE** du **PIC** programmée sur front montant, surveiller l'apparition du premier front montant sur la sortie **ECHO** du **SRF 04**, ce qui permet de déterminer l'instant t_i de ce front.

③ puis, toujours avec la fonction **INPUT CAPTURE** du **PIC** mais cette fois-ci programmée sur front descendant, surveiller l'apparition du premier front descendant sur la sortie **ECHO** du **SRF 04**, ce qui permet de déterminer l'instant t_f de ce front.

④ la différence $\Delta T = t_f - t_i$ donne la **durée** du pulse.

⑤ on applique ensuite la formule
$$d(\text{cm}) = \frac{\Delta T(\mu\text{s})}{58}$$

donnée par le constructeur du **SRF 04** pour déterminer la distance **d** de l'obstacle.

Les deux pages suivantes peuvent aussi vous être utiles !!!

ELEMENTS SUR L'IMPLANTATION D'UN EMETTEUR/RECEPTEUR A ULTRA SONS (suite et fin)

DERNIERE AIDE

Comme d'habitude (méthode déjà vue et qui doit maintenant être un réflexe) structurez bien votre projet en écrivant tout d'abord deux fichiers :

↳ **telemetre.h** : pour le hardware,
(définition des broches...)

↳ **telemetre.c** : pour les fonctions de » bas niveau »,
comme par exemple :

```
// *****  
// initialisation des broches du PIC concernées  
void init_telemetre(void)  
// *****
```

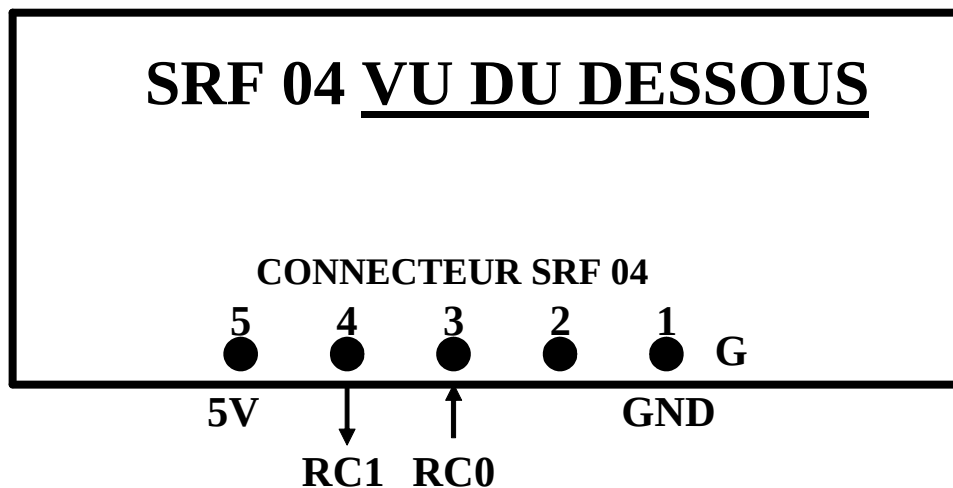
```
// *****  
// génération d'un pulse sur l'entrée TRIGGER  
void pulse(void)  
// *****
```

...

ELEMENTS SUR L'IMPLANTATION D'UN EMETTEUR/RECEPTEUR A ULTRA SONS (suite et fin)

DERNIERE AIDE

Si de plus, vous utilisez le câblage suivant pour relier le module **SRF 04** :



alors vous pouvez le tester sous **MPLAB** en faisant :

File Import... et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

fichiers HEX OK/source en C le fichier :

telemetre lcd ascii.hex (c'est un exécutable)

Puis choisir l'outil **MPLAB ICD3** par :

Debugger Select Tool, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.

Et enfin lancer ce programme.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS

INTRODUCTION GENERALE

Très utilisés en robotique ou dans les imprimantes par exemple, les moteurs pas à pas sont assez faciles à piloter.

**Leur principe est de transformer
une impulsion électrique en une rotation angulaire
appelée le pas du moteur.**

Leur principal avantage est de permettre un contrôle de la vitesse ou de la position **en boucle ouverte**, contrairement par exemple aux moteurs à courant continu où une mesure de la vitesse ou de la position est indispensable pour assurer ce contrôle (on parle alors d'asservissement de vitesse ou de position).

Il existe différentes familles de moteur pas à pas :

- le moteur à **aimant permanent**,
- le moteur à **réluctance variable**,
- le moteurs **hybride** (mélange des 2 précédents)

Nous parlerons dans un premier temps du moteur pas à pas à aimant permanent, car au niveau du principe, c'est le plus simple à comprendre.

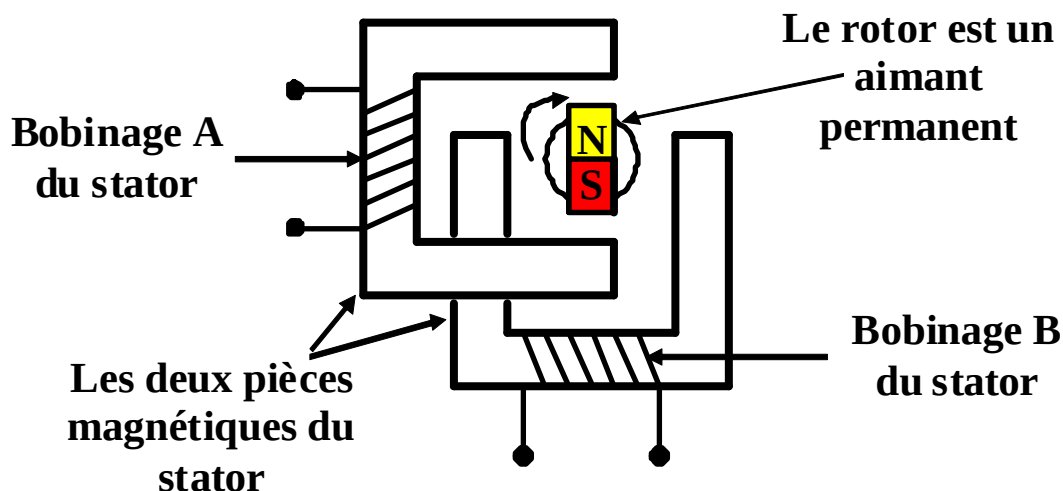
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

Le **rotor** est constitué d'un **aimant permanent**.

Le **stator** est réalisé par **deux pièces magnétiques**, chacune entourée d'un **bobinage** :

quand on fait circuler un courant dans ce(s) bobinage(s), on transforme le stator en **électroaimant**, polarisé dans un sens ou dans l'autre suivant le sens de ce courant.



La rotation est assurée par l'interaction entre le champ magnétique créé par le stator et l'aimant du rotor.

Dans la pratique, c'est la configuration des bobinages qui permet de définir plusieurs sous-familles.

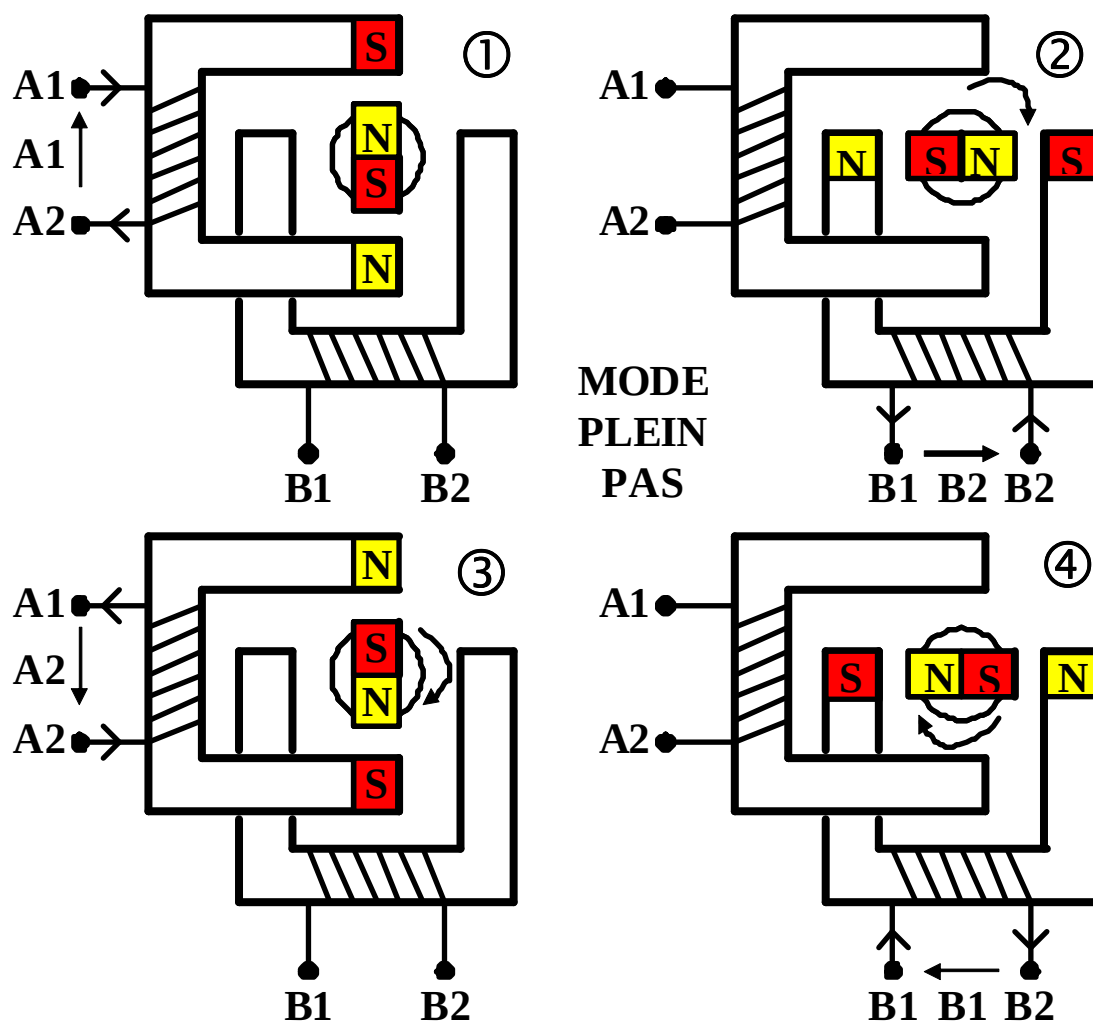
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

- les moteurs à 4 fils ou moteurs bipolaires :

bipolaires, car le courant circule dans chaque bobinage tantôt dans un sens tantôt dans l'autre. Le principe de la rotation dite en **mode plein pas (ou pas entier)** est décrit ci-dessous, d'après une animation sur la page web :

<http://sitelec.org/cours/abati/flash/pas.swf>



ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

On peut résumer ce principe en notant que c'est la séquence :

A1	B2	A2	B1	A1	B2	A2	B1	...
un tour sens horaire				un tour sens horaire				...

qui permet d'assurer la rotation du moteur dans **le sens horaire**.

Pour changer le sens de rotation, il suffit d'appliquer le même principe, mais dans l'ordre ① , ④ , ③ et ②...
(et donc permuter les phases ② et ④)

Ce qui revient à dire que c'est la séquence :

A1	B1	A2	B2	A1	B1	A2	B2	...
un tour sens anti-horaire				un tour sens anti-horaire				...

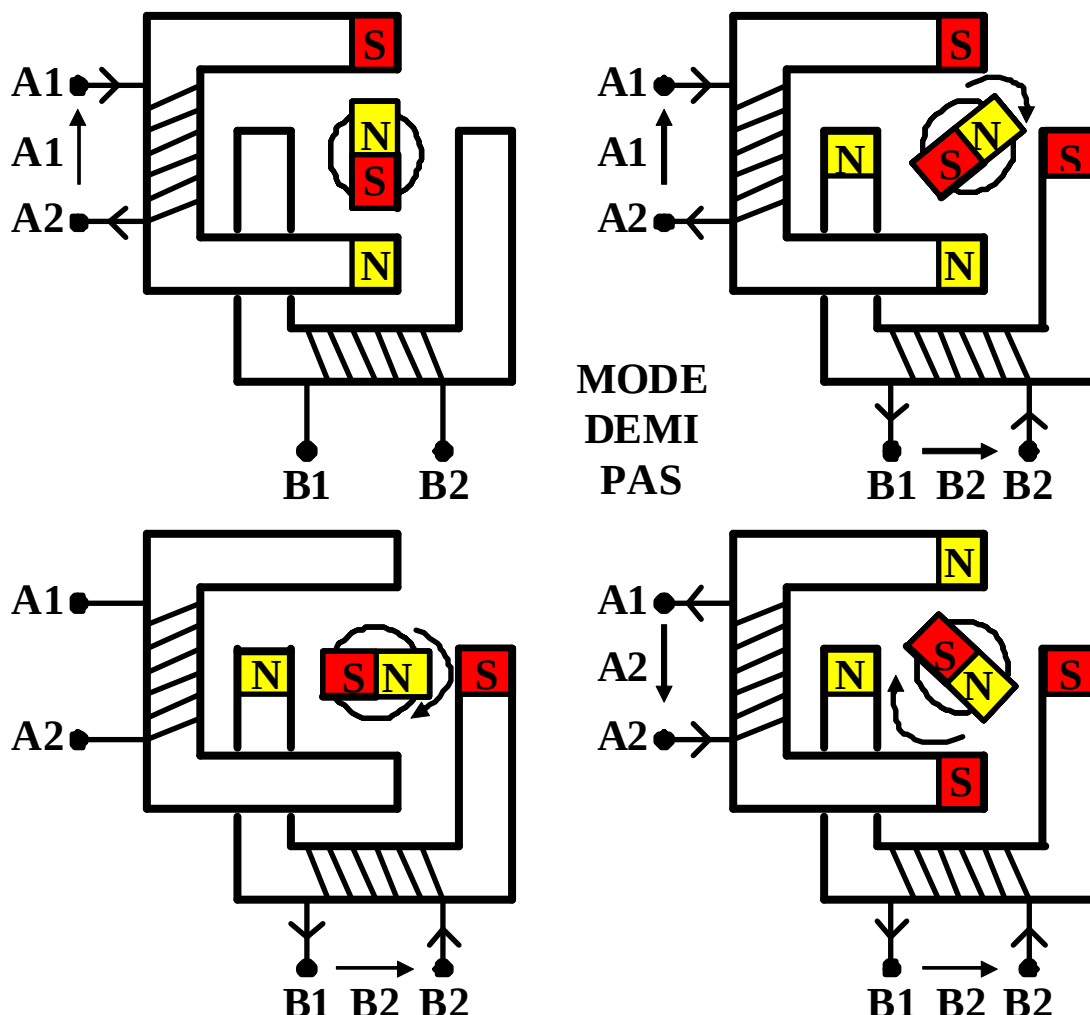
qui fait tourner le moteur dans **le sens anti-horaire**.

On voit également sur cet exemple (mode pas entier), qu'on obtient **4 pas par tour**, chaque pas correspondant à une rotation de **90°**.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

Il existe aussi le **mode demi-pas** obtenu en intercalant des états intermédiaires où deux bobinages sont commandés en même temps, comme indiqué ci-dessous sur un demi-tour, le principe étant le même pour l'autre demi-tour.



ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

Ce qui revient à dire que c'est la séquence :

A1	A1B2	B2	B2A2	A2	A2B1	B1	B1A1	...
un tour sens horaire								...

qui permet d'assurer la rotation du moteur en **mode demi-pas** dans **le sens horaire**.

Dans ce cas là, il y a bien **8 pas par tour**, chaque pas correspondant à une rotation de **45°**.

Et c'est bien sûr la séquence :

A1	A1B1	B1	B1A2	A2	A2B2	B2	B2A1	...
un tour sens anti-horaire								...

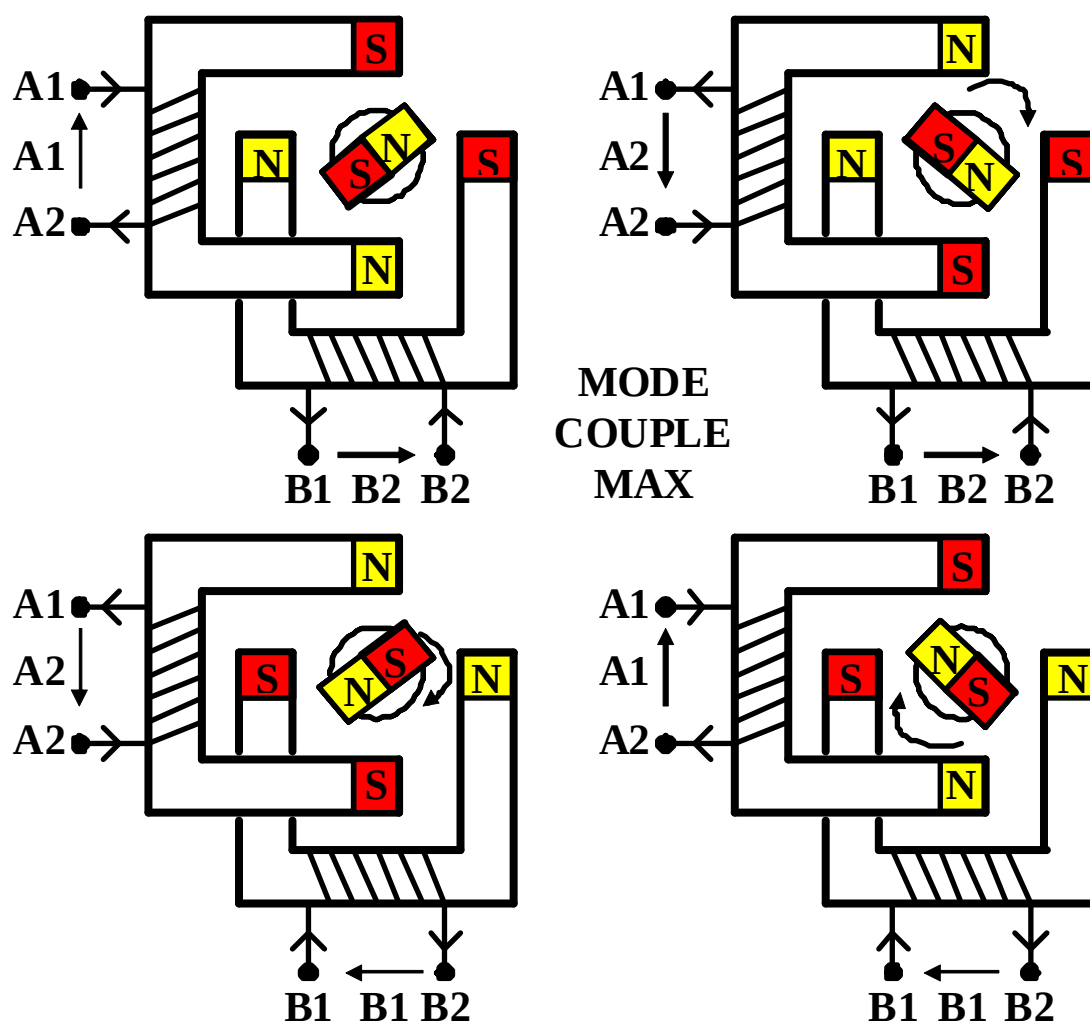
qui fera tourner le moteur en **mode demi-pas** dans **le sens anti-horaire**.

Il existe enfin le **mode couple max** qui est obtenu en supprimant, dans les séquences demi-pas précédentes, les états où un seul bobinage est commandé.

C'est donc un fonctionnement plein pas, mais comme on commande tout le temps **deux bobinages en même temps**, les champs magnétiques seront plus importants et donc le couple du moteur aussi.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT



En résumé, séquences pour le mode couple max :

A1B2	B2A2	A2B1	B1A1	A1B2	B2A2	A2B1	B1A1	...
un tour sens horaire				un tour sens horaire				...
A1B1	B1A2	A2B2	B2A1	A1B1	B1A2	A2B2	B2A1	...
un tour sens anti-horaire				un tour sens anti-horaire				...

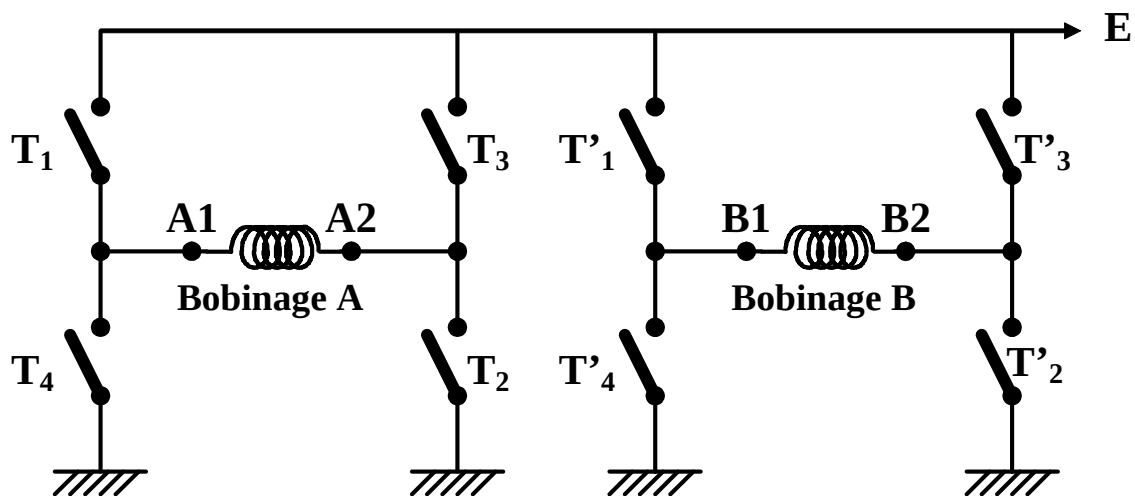
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT ELECTRONIQUE DE COMMANDE MOTEUR 4 FILS

Pour un moteur bipolaire et quelque soit son mode de fonctionnement, le courant n'a pas toujours le même sens dans les bobinages.

Pour assurer l'inversion de ce courant, une solution consiste à utiliser un « **pont en H** » par bobinage.

Les 8 interrupteurs T_i et T'_i sont en fait des transistors.



Dans un tel « pont en H », deux transistors situés sur la même verticale (comme (T_1 et T_4) ou (T_3 et T_2)) ne peuvent être passants ensemble. Pourquoi ?

Par contre, deux transistors « diagonalement opposés » (comme (T_1 et T_2) ou (T_3 et T_4)) doivent être passants ensemble. Pourquoi ?

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT ELECTRONIQUE DE COMMANDE MOTEUR 4 FILS

Faisons maintenant le lien avec les séquences de commande précédentes.

- l'état **A1** correspond à **T₁** et **T₂** **ON**, les autres **OFF**.
- l'état **A2** correspond à **T₃** et **T₄** **ON**, les autres **OFF**.
- l'état **B1** correspond à **T'₁** et **T'₂** **ON**, les autres **OFF**.
- l'état **B2** correspond à **T'₃** et **T'₄** **ON**, les autres **OFF**.

Ce qui permet de dresser les tableaux suivants, où :

1 correspond à **ON** et **0** à **OFF**.

- mode plein pas, sens horaire :

Etat	T ₁	T ₂	T ₃	T ₄	T' ₁	T' ₂	T' ₃	T' ₄
A1	1	1	0	0	0	0	0	0
B2	0	0	0	0	0	0	1	1
A2	0	0	1	1	0	0	0	0
B1	0	0	0	0	1	1	0	0

Pour le sens **anti-horaire**, permuter **B2** et **B1**.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT ELECTRONIQUE DE COMMANDE MOTEUR 4 FILS

- mode demi-pas, sens horaire :

Etat	T ₁	T ₂	T ₃	T ₄	T' ₁	T' ₂	T' ₃	T' ₄
A1	1	1	0	0	0	0	0	0
A1B2	1	1	0	0	0	0	1	1
B2	0	0	0	0	0	0	1	1
B2A2	0	0	1	1	0	0	1	1
A2	0	0	1	1	0	0	0	0
A2B1	0	0	1	1	1	1	0	0
B1	0	0	0	0	1	1	0	0
B1A1	1	1	0	0	1	1	0	0

Pour le sens **anti-horaire**, permuter B2 et B1.

- mode couple max, plein-pas, sens horaire :

Etat	T ₁	T ₂	T ₃	T ₄	T' ₁	T' ₂	T' ₃	T' ₄
A1B2	1	1	0	0	0	0	1	1
B2A2	0	0	1	1	0	0	1	1
A2B1	0	0	1	1	1	1	0	0
B1A1	1	1	0	0	1	1	0	0

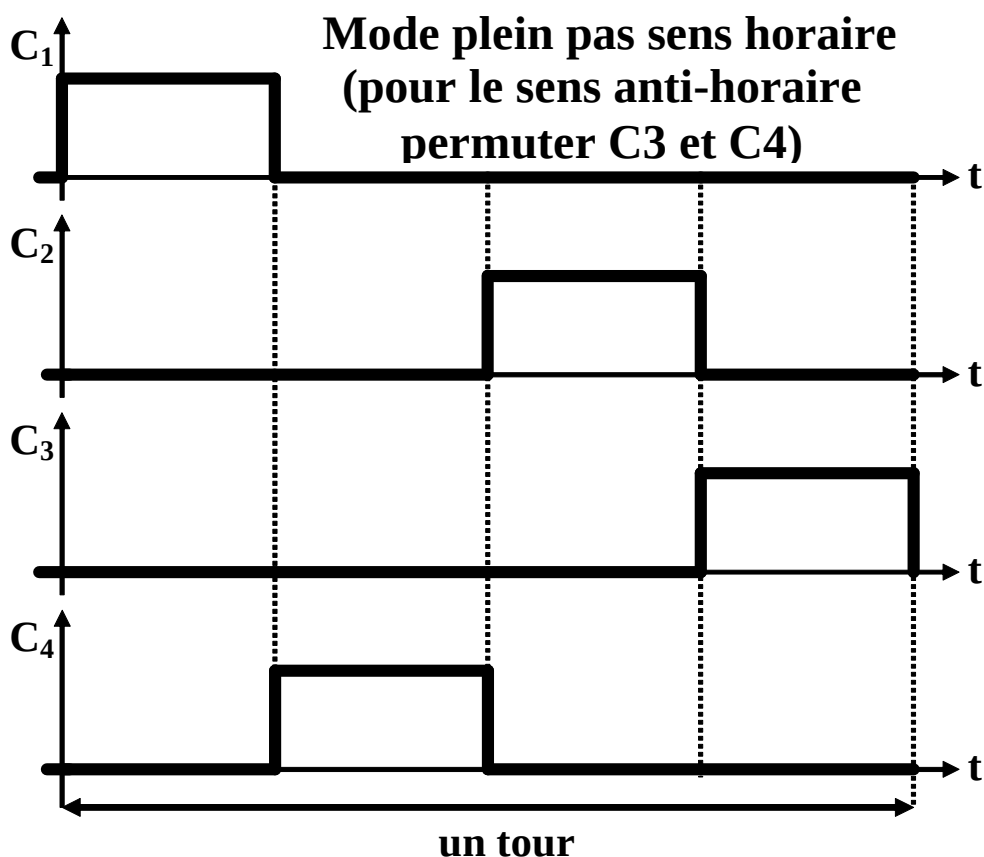
Pour le sens **anti-horaire**, permuter B2 et B1.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT ELECTRONIQUE DE COMMANDE MOTEUR 4 FILS

On peut aussi raisonner avec des chronogrammes, qui sont parfois « plus parlants ». On appellera :

- C_1 le signal de commande de T_1 et T_2 qui au niveau haut les rend passants et au niveau bas les bloque,
- C_2 celui pour T_3 et T_4 avec la même logique,
- C_3 celui pour T'_1 et T'_2 avec la même logique,
- et C_4 celui pour T'_3 et T'_4 avec la même logique,



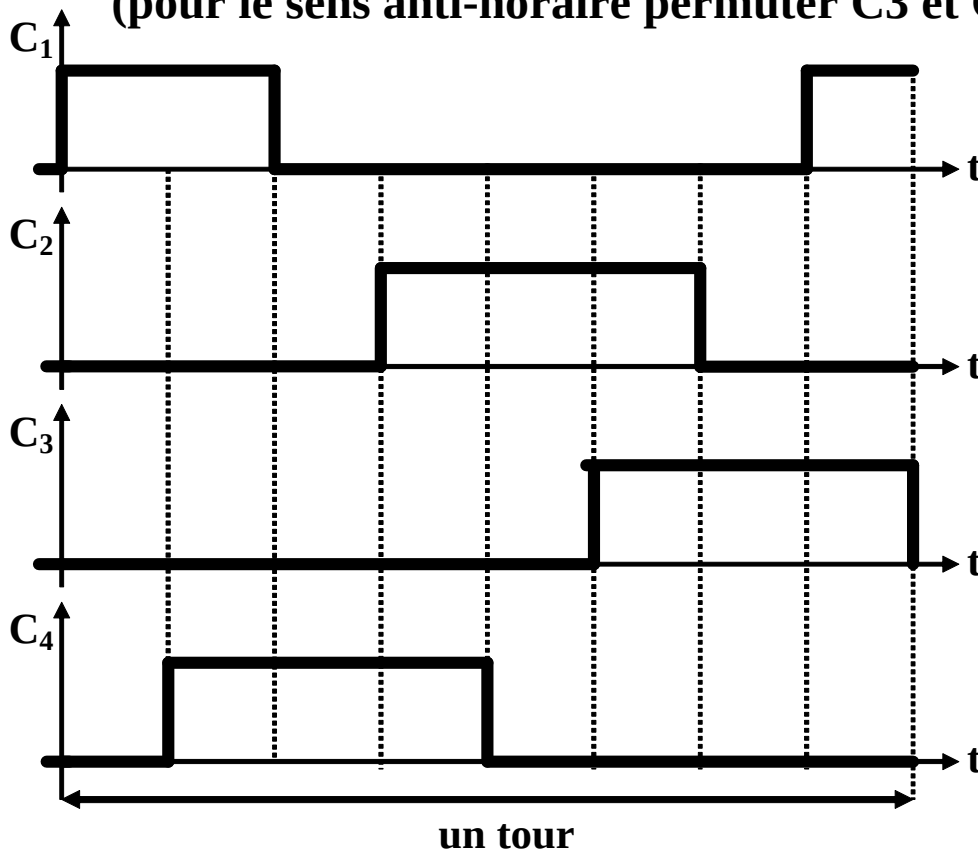
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LES PAS A PAS A AIMANT PERMANENT

LE MOTEUR PAS A PAS A AIMANT PERMANENT

Mode demi-pas sens horaire

(pour le sens anti-horaire permuter C3 et C4)

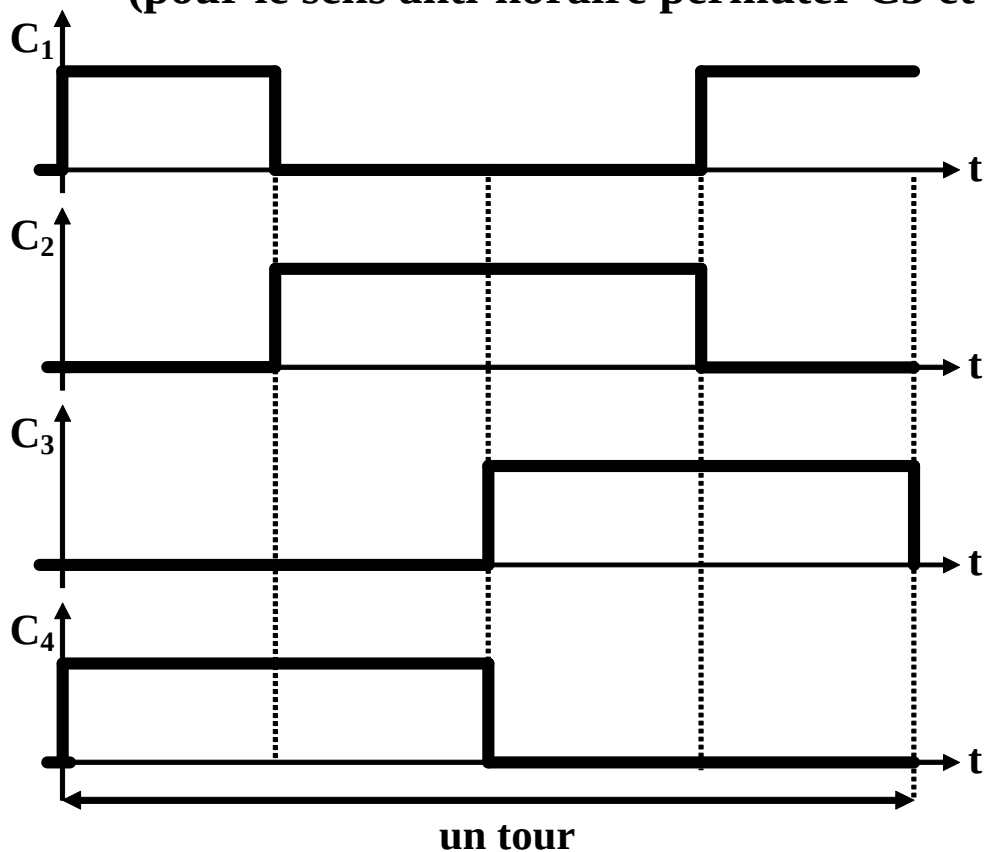


ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT ELECTRONIQUE DE COMMANDE MOTEUR 4 FILS

Mode couple max sens horaire

(pour le sens anti-horaire permuter C3 et C4)

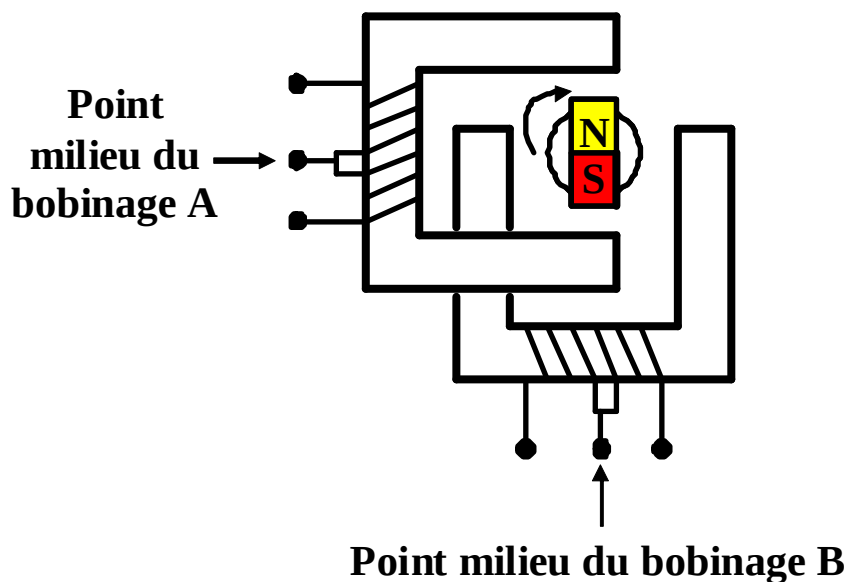


On voit que $C_2 = \overline{C_1}$ et que $C_4 = \overline{C_3}$ et donc deux signaux de commande suffisent au final pour le mode plein pas couple max.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

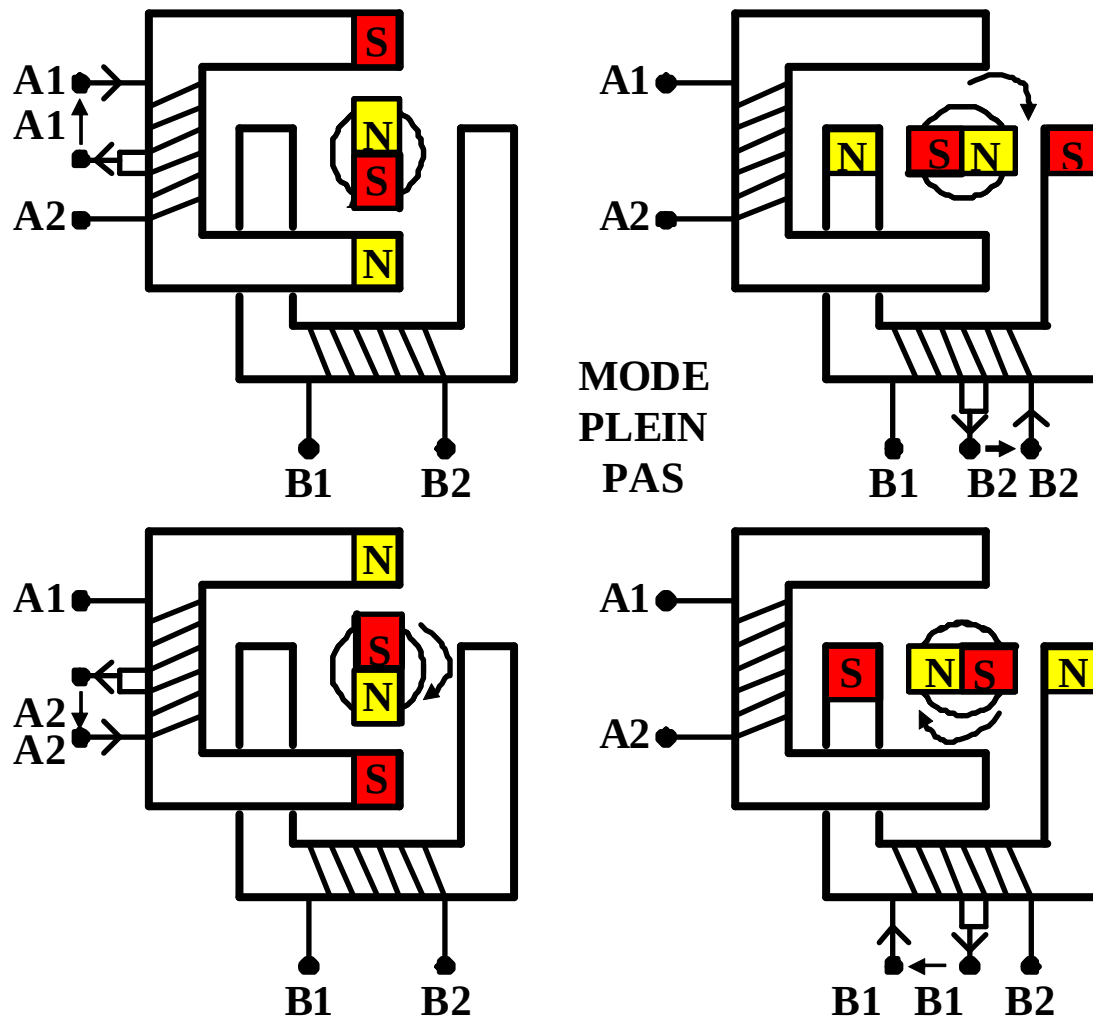
- **les moteurs à 6 fils ou moteurs unipolaires :**
unipolaires, car le courant circule dans chaque bobinage toujours dans le même sens. Pour cela, il a été rajouté 2 fils supplémentaires, correspondant aux 2 points milieu des bobinages d'un moteur à 4 fils.



Grâce à ces points milieux, comme on peut le voir sur le principe de la page suivante donné pour une rotation en **mode plein pas** dans le **sens horaire**, le courant circule toujours dans le même sens dans chaque « demi-bobinage ».

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT



On note que c'est la même séquence qu'un moteur 4 fils

A1	B2	A2	B1	A1	B2	A2	B1	...
un tour sens horaire				un tour sens horaire				...

qui permet d'assurer la rotation du moteur 6 fils en **mode plein pas** dans le **sens horaire**.

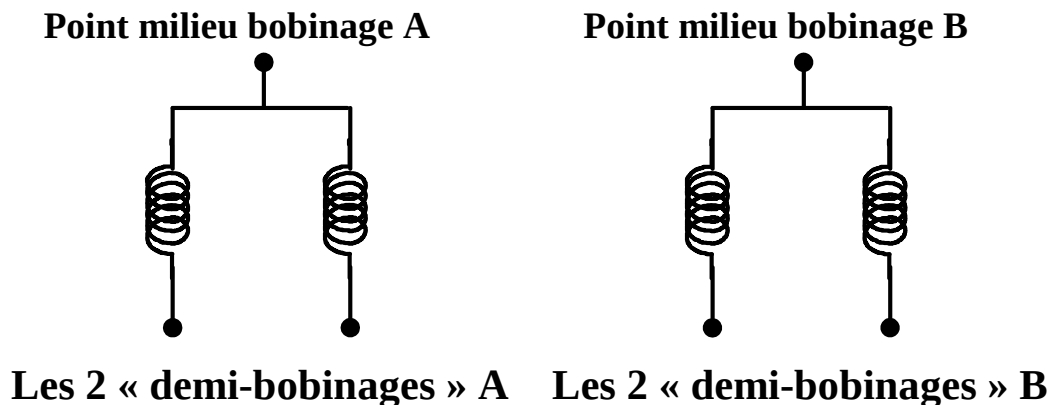
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

On pourrait aussi montrer que ce moteur à 6 fils fonctionne aussi en **mode demi-pas** et en mode **couple max**.

Et finalement quelque soit le mode de fonctionnement, les séquences de commande vues pour un moteur 4 fils restent les mêmes pour un moteur 6 fils.

En fait tout se passe comme si on avait 4 bobinages (en fait 4 « demi-bobinages ») au lieu des 2 bobinages d'un moteur 4 fils bipolaire.



Notons d'ailleurs qu'un moteur unipolaire peut fonctionner en bipolaire. Il suffit de laisser en l'air les deux points milieux. Par contre la réciproque est fausse.

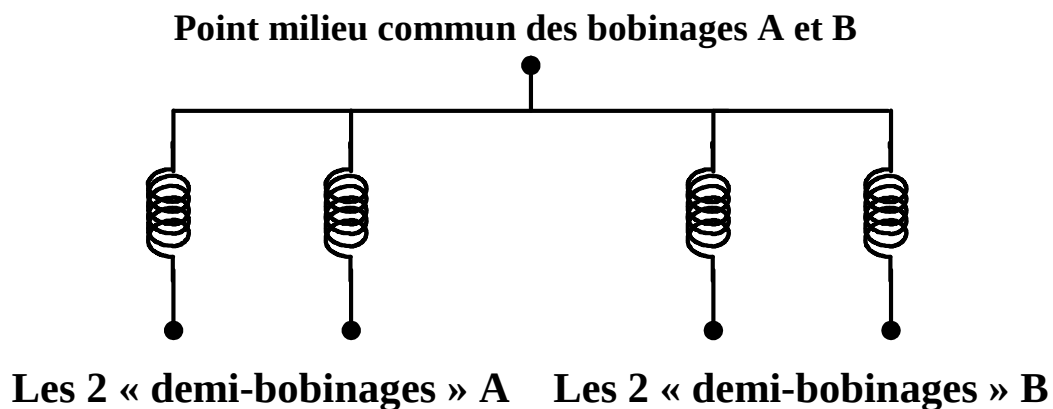
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT

Notons enfin qu'il existe aussi :

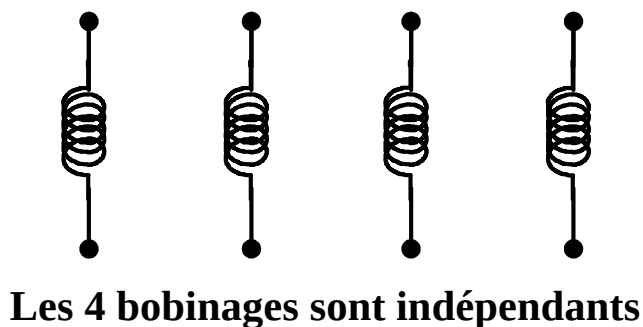
- les moteurs **5 fils** :

Les deux points milieu des 2 bobinages sont reliés entre eux.



- les moteurs **8 fils** :

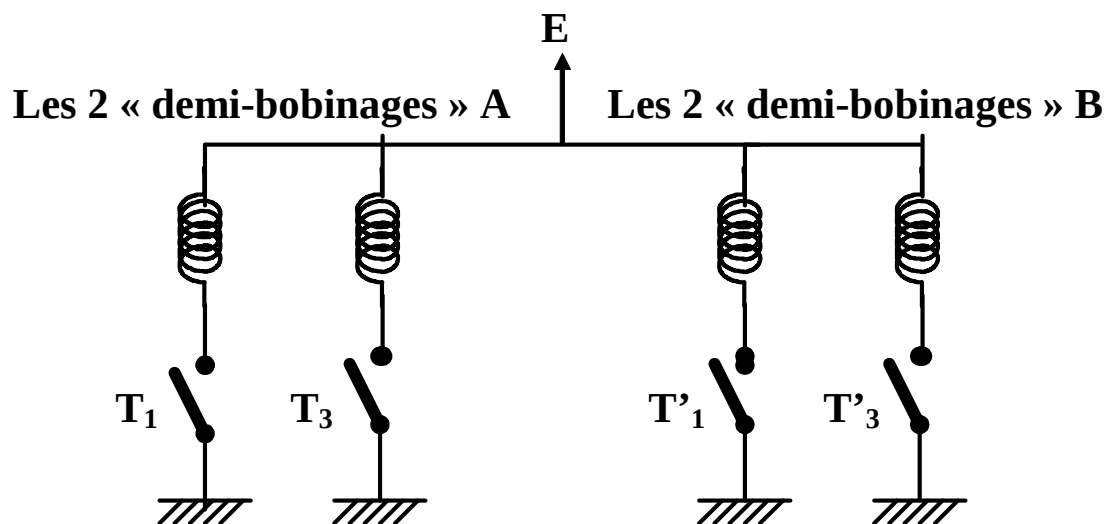
Les 4 « demi-bobinages » deviennent 4 bobinages totalement indépendants (sans aucun point commun).



ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS A AIMANT PERMANENT ELECTRONIQUE DE COMMANDE MOTEUR 6 FILS

En général, on relie les deux points milieu entre eux et à la tension d'alimentation E du moteur.



L'avantage est de n'avoir besoin que de 4 transistors au lieu des 8 pour un moteur 4 fils, ce qui **simplifie et diminue le coût** de l'électronique de commande.

En gardant les mêmes notations que précédemment pour les signaux C_1 (commande de T_1), C_2 (commande de T_3), C_3 (commande de T'_1) et C_4 (commande de T'_3) les chronogrammes de commande restent les même que pour un moteur 4 fils.

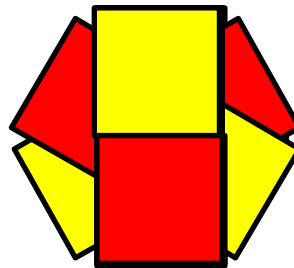
ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS HYBRIDE

Si les principes de fonctionnement sont simples pour les moteurs à aimant permanent, leur principal inconvénient est la taille d'un pas qui reste trop grande, notamment pour des applications plus précises où on souhaite des rotations de faible valeur angulaire.

Pour augmenter le nombre de pas par tour, on remplace l'aimant permanent du rotor (qui n'avait que 2 pôles) par une pièce dentée et aimantée ayant un nombre de pôles plus important.

**Rotor à 6 pôles
d'un moteur
hybride**



Du coup, au lieu de tourner d'un quart de tour (en mode plein pas) ou d'un huitième de tour (en mode demi-pas), l'angle de rotation sera plus faible et va dépendre du nombre de pôles du rotor. Dans l'exemple ci-dessus, ce sera 30° en mode plein pas (soit 12 pas par tour) ou 15° en mode demi-pas).

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

LE MOTEUR PAS A PAS HYBRIDE

Les moteurs hybrides permettent d'obtenir un nombre de pas plus important, pouvant aller jusqu'à plusieurs centaines.

Typiquement, on trouvera des moteurs hybrides à :

- à 24 pas par tour,
- à 100 pas par tour,
- à 200 pas par tour ...

Notons qu'avec 360 pas par tour, chaque pas correspond à une rotation de 1°.

Les modes de fonctionnement (**plein pas, demi-pas, couple max, sens horaire** ou **anti-horaire**) sont toujours possibles et les séquences de commande restent les mêmes.

Idem pour l'électronique de commande, suivant qu'on ait affaire à un moteur hybride 4 fils, 5 fils ou 6 fils.

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

REMARQUES GENERALES POUR TOUTES LES TECHNOLOGIES DE MOTEUR PAS A PAS

En ce qui concerne les courants consommés dans les bobinages, même pour les moteurs pas à pas de très faibles puissances, une broche d'un **PORTX** du PIC du type **RXi** ne peut pas directement commander un bobinage, **au risque de la détruire** car elle ne pourra pas fournir le courant demandé par le bobinage.

Du coup, il faudra toujours prévoir une **interface de puissance** entre les lignes du PORT concernées et les bobinages du moteur.

De plus, cette interface permet aussi de gérer les différences entre la tension d'alimentation du PIC (typiquement 5 V) et la tension d'alimentation du moteur (peut-être aussi 5 V mais qui peut aller jusqu'à 24 V, voire plus, en fonction des caractéristiques électriques du moteur utilisé).

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite)

REMARQUES GENERALES POUR TOUTES LES TECHNOLOGIES DE MOTEUR PAS A PAS

Des circuits intégrés spécifiques existent et qui comportent souvent tous les transistors de puissance nécessaires, ainsi que les diodes de roue libre permettant d'éviter les surtensions à la rupture du courant dans les bobinages.

Citons par exemple :

- le **TEA 3717** (à priori obsolète),
- le **MC 3479**,
- les **L297** et **L298**,
- le **SLA 7024M**,
- les **ULN 2001**, **ULN 2002**, **ULN 2003** et **ULN 2004...**

Les trois pages suivantes peuvent aussi vous être utiles !!!

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite et fin)

DERNIERE AIDE

Comme d'habitude (méthode déjà vue et qui doit maintenant être un réflexe) structurez bien votre projet en écrivant tout d'abord deux fichiers :

↪ **moteur pas a pas.h** : pour le hardware,
(définition des broches...)

↪ **moteur pas a pas.c** : pour les fonctions de » bas
niveau », comme par exemple :

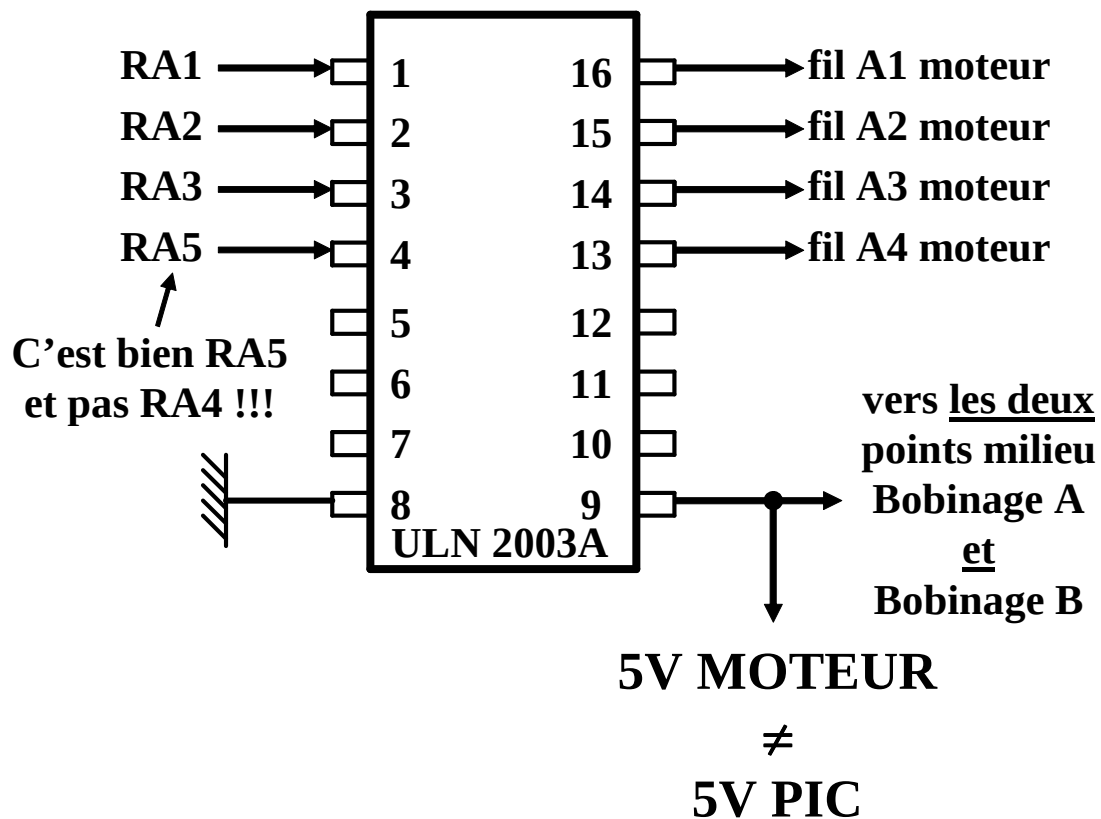
```
// *****  
// initialisation des broches du PIC concernées  
void init_moteur_pas_a_pas(void)  
// *****
```

...

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite et fin)

DERNIERE AIDE

Si de plus, vous utilisez un moteur pas à pas 6 fils en le câblant comme ci-dessous :



alors vous pouvez le tester sous **MPLAB** en faisant :

File Import... et aller chercher **dans votre répertoire**
de travail dans le sous répertoire :

fichiers HEX OK/source en C le fichier :

moteur pas a pas RA0.hex (c'est un exécutable)

ELEMENTS SUR L'IMPLANTATION D'UN MOTEUR PAS A PAS (suite et fin)

DERNIERE AIDE

Puis choisir l'outil **MPLAB ICD3** par :

Debugger Select Tool, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.

Lancer ce programme et changer la vitesse par le potentiomètre **RA0** de la carte **PICDEM 2 PLUS**.

Si de plus vous avez aussi un LCD graphique câblé comme indiqué dans **l'annexe 3**, la vitesse du moteur s'affichera en temps réel sur ce LCD.

Si vous possédez en plus un clavier câblé comme indiqué dans **l'annexe 2**, alors vous pouvez refaire la même manipulation avec cette fois-ci le fichier :

moteur pas a pas clavier.hex.

Lancer ce programme et changer la vitesse en utilisant les touches de clavier. La vitesse s'affiche aussi en temps réel sur le LCD graphique.

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE

INTRODUCTION GENERALE

Il est intéressant de pouvoir commander directement un système par la voix en lui transmettant des ordres oralement.

Le module **VOICE DIRECT 364** de la société **SENSORY** permet de répondre efficacement mais malgré tout assez simplement à cette problématique.



Il intègre un **processeur de reconnaissance vocale** et toutes les interfaces analogique/numérique pour être utilisé avec un simple système microphone/écouteurs.

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) MODES DE FONCTIONNEMENT

Ce module a deux grands modes de fonctionnement :

- le mode complet ou **SLAVE MODE** où on utilise toutes ses possibilités en le reliant à un autre processeur qui peut le contrôler entièrement,
- le mode simplifié ou **STAND ALONE** où il peut être utilisé « seul ».

Nous allons ici donner des informations sur le mode **STAND ALONE**, qui est le plus facile à mettre en œuvre. Même si dans ce mode, un processeur n'est pas vraiment nécessaire, nous utiliserons le PIC pour faciliter et améliorer la commande du module.

Dans le mode **STAND ALONE**, il existe 3 sous modes de fonctionnement :

- le sous mode **SD** ou **SPEAKER DEPENDANT**,
- le sous mode **SWCL** ou **SINGLE WORD CONTINUOUS LISTENING**,
- le sous mode **MWCL** ou **MULTI WORD CONTINUOUS LISTENING**,

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) PRINCIPES D'UTILISATION

Dans ces 3 sous modes, les principes d'utilisation sont assez similaires.

D'abord une **phase d'apprentissage**, où le module peut apprendre jusqu'à **15 ordres oraux différents**. Ces ordres, appelés aussi mots ou phrases de commande sont assez brefs (3 secondes max).

Puis, on place le module en **phase de reconnaissance** justement pour reconnaître les ordres oraux appris.

Chacun d'entre eux pouvant entraîner une action qui lui est propre, comme :

- “ouvre la porte”,
- “allume la lumière”,
- “démarre la voiture”,
- “freine” (mais là, il vaut mieux qu'il soit fiable !!!)...

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) DIFFERENCES ENTRE LES 3 SOUS MODES

Il existe cependant quelques différences entre 3 sous modes.

Le sous mode **SD** nécessite une action manuelle de l'utilisateur (appui sur un bouton poussoir par exemple) entre chaque commande orale, ce qui est parfois gênant.

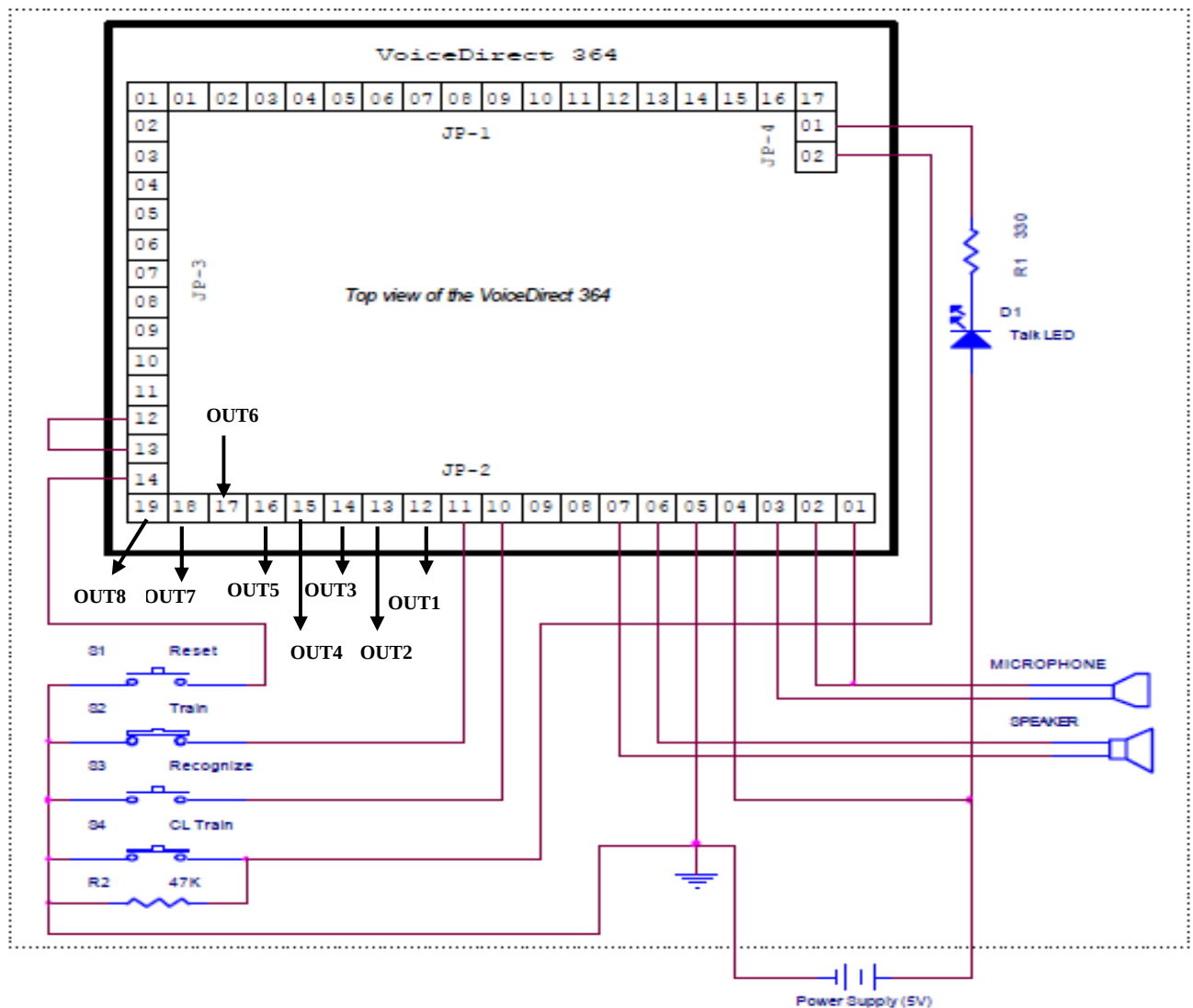
Dans le sous mode **S(ingle)W(ord)CL**, il existe un **seul mot de déverrouillage** qui place le module en « **CONTINUOUS LISTENING** » c'est-à-dire en écoute permanente d'un ordre à venir. Lorsque cet commande est prononcé et, s'il est reconnu, l'action associée sera effectuée. Le module se remet en attente du mot de déverrouillage qu'il faudra donc prononcer entre chaque nouvelle commande. Donc, **plus d'action manuelle de l'utilisateur** entre chaque ordre.

Dans le sous mode **M(ulti)W(ord)CL**, il existe un **trois mots de déverrouillage différents** (même principe qu'en **SWCL**) et cette souplesse offre la possibilité à **3 personnes différentes** de pouvoir donner des ordres (système poly-locuteur).

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) SOUS MODE SWCL

C'est ce sous mode qu'on va implanter ici.

Il faut en regarder le schéma de câblage donné dans la documentation du module **VOICE DIRECT 364**.



ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) RÔLE DES QUATRE BOUTON POUISOIRS

Voici ce qu'entraîne l'appui sur l'un des 4 **BP** :

- appui sur S1 (Reset) :

↳ pour faire un reset du module entraînant son auto calibration (choix du mode de fonctionnement, sensibilité de l'apprentissage et fiabilité (fidélité, exactitude) de la reconnaissance...),

- appui sur S4 (CL Train) :

↳ pour l'apprentissage du mot de déverrouillage,

- appui sur S2 (Train) :

↳ pour l'apprentissage des 15 (au maximum) ordres différents,

- appui sur S3 (Recognize) :

↳ pour rentrer en phase de reconnaissance.

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite)

QUELQUES REMARQUES SUR CE SCHEMA

Notons également que l'appui simultané sur **S2** et **S3** **pendant plus d'une seconde** efface tous les mots appris (MEMORY ERASED).

En pratique, ces 4 BP seront avantageusement remplacés par des lignes de PORT du PIC **programmées en sortie**.

On utilisera un casque avec écouteurs pour le **MICROPHONE** et le **SPEAKER**.

Notons enfin que la liaison entre les pattes **13 (MODE)** et **12 (GND)** du connecteur **JP-3** permet d'indiquer au module (à la mise sous tension ou après un reset « à chaud ») qu'il est dans le mode **STAND ALONE**.

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) PHASE D'APPRENTISSAGE

↳ apprentissage du mot de déverrouillage :

après appui sur **S4**, le module vous invite à prononcer **une première fois** le mot de déverrouillage par le message oral :

'Say word one'

Puis une seconde fois par: **'Repeat'**

S'il y a cohérence et « une ressemblance suffisante » entre les deux prononciations du mot de déverrouillage, le module répond : **'Accepted'**.

Sinon, il faut recommencer : **'Say word one'...**

↳ apprentissage du mot de commande N°1 :

ensuite, après appui cette fois-ci sur **S2**, le module vous invite à prononcer une première fois le **premier ordre** par le message oral :

'Say word one'

Puis une seconde fois par: **'Repeat'**

S'il y a cohérence et « une ressemblance suffisante » entre les deux prononciations du premier ordre, le module répond : **'Accepted'**.

Sinon, il faut recommencer : **'Say word one'...**

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) PHASE D'APPRENTISSAGE (suite)

➤ **apprentissage du mot de commande N°2 :**

ensuite, après un **nouvel** appui sur S2, le module vous invite à prononcer une première fois le deuxième ordre par le message oral :

'Say word two'

Puis une seconde fois par: **'Repeat'**

S'il y a cohérence et « une ressemblance suffisante » entre les deux prononciations du deuxième ordre, le module répond : **'Accepted'**.

Sinon, il faut recommencer : **'Say word two'...**

...

Et on recommence ces manipulations jusqu'au quinzième ordre (si on veut les faire apprendre tous) ou jusqu'au passage dans la phase de reconnaissance.

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) PHASE DE RECONNAISSANCE

On rentre dans la phase de reconnaissance par appui sur **S3**.

Normalement, la LED **Talk LED** est alors **allumée**, indiquant que le module est dans le sous mode **SINGLE WORD CONTINUOUS LISTENING**.

Si vous prononcez alors le mot de déverrouillage, cette LED **s'éteint brièvement** (pour indiquer que ce mot a bien été reconnu) puis vous avez 3 secondes pour donner un ordre quelconque parmi ceux qui ont été appris.

Si cet ordre est reconnu, le module va indiquer oralement le **N° de l'ordre** et va « **pulser** » au niveau haut pendant 1 seconde une (ou deux) de ses sorties **OUTi** situées sur le connecteur **JP-2**.

Le tableau de la page suivante indique la correspondance entre le N° de l'ordre reconnu et la (ou les) sortie(s) qui sera(ont) pulsée(s).

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite) PHASE DE RECONNAISSANCE (suite)

Recognition Word	OUT 1	OUT 2	OUT 3	OUT 4	OUT 5	OUT 6	OUT 7	OUT 8
CL + SD Word 01	A							
CL + SD Word 02		A						
CL + SD Word 03			A					
CL + SD Word 04				A				
CL + SD Word 05					A			
CL + SD Word 06						A		
CL + SD Word 07							A	
CL + SD Word 08								A
CL + SD Word 09	A							A
CL + SD Word 10		A						A
CL + SD Word 11			A					A
CL + SD Word 12				A				A
CL + SD Word 13					A			A
CL + SD Word 14						A		A
CL + SD Word 15							A	A

Note: "A" indicates that the outputs are "Active-high".

Il ne reste plus qu'à détecter la (ou les) sorties
« pulsée(s) » pour lancer la bonne action... Et le tour est
joué !!!

**Les deux pages suivantes peuvent aussi vous être
utiles !!!**

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite et fin) DERNIERE AIDE

Comme d'habitude (méthode déjà vue et qui doit maintenant être un réflexe) structurez bien votre projet en écrivant tout d'abord deux fichiers :

👉 **voice direct.h** : pour le hardware,
(définition des broches...)

👉 **voice direct.c** : pour les fonctions de » bas niveau »,
comme par exemple (liste non exhaustive) :

```
// *****  
// initialisation des broches du PIC concernées  
void init_voice_direct(void)  
// *****  
  
// *****  
// fait un reset « à chaud » du module voice direct  
// *****  
void reset_voice_direct(void)  
  
// *****  
// efface les mots du module voice direct  
// *****  
void efface_voice_direct(void)
```

ELEMENTS SUR L'IMPLANTATION D'UN MODULE DE RECONNAISSANCE VOCALE (suite et fin) DERNIERE AIDE

```
// *****  
// passe en mode apprentissage du mot de déverrouillage  
// *****  
void apprentissage_deverrouillage (void)  
  
// *****  
// passe en mode apprentissage des ordres  
// *****  
void apprentissage_ordre (void)  
  
// *****  
// passe en mode de reconnaissance  
// *****  
void reconnaissance (void)  
  
...
```

Regardez, si vous le souhaitez, les vidéos de démonstration d'**apprentissage** et de **reconnaissance** situées dans le sous répertoire : **documentations/docs voice direct** de votre répertoire de travail.

Ces exemples correspondent au fichier **domotique.hex**.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE

Il est **obligatoire** de lire avant toute chose
l'annexe 3 de ce document qui présente le LCD
graphique DMX 973.

Dans cette partie, nous allons parler du **DMX 973B**,
qui est une version plus récente du DMX 973.

Et nous insisterons ici essentiellement sur les
différences entre ces 2 composants.

Le contrôleur d'écran utilisé pour piloter le DMX 973B
est le **SED 1530**.

(voir sa documentations complète dans le répertoire
documentations de votre répertoire de travail)

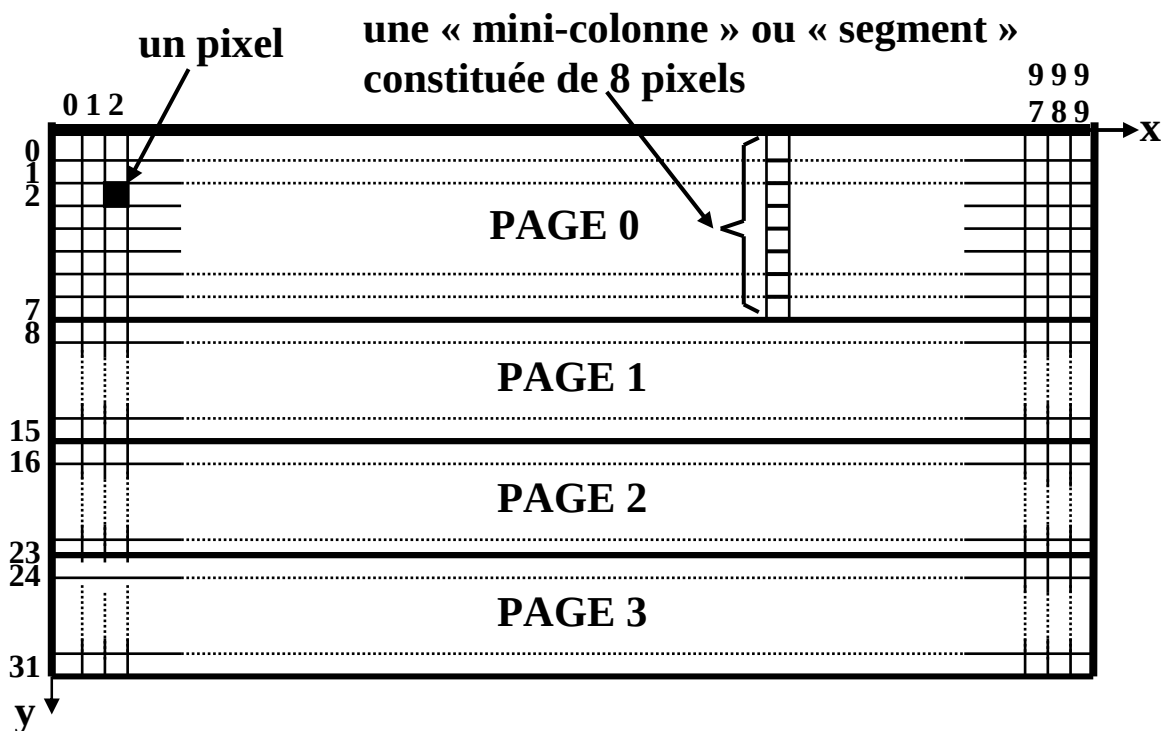
COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

RESOLUTION $100 \times 32 = 3200$ pixels

L'écran comprend **32 lignes** (ligne_0 à ligne_31) de **100 pixels** chacune (colonne_0 à colonne_99)

PAGINATION DE L'ECRAN (même principe que pour le DMX 973)

L'écran est divisé en **4 pages** (page_0 à page_3)



Chaque page a une « hauteur » de 8 pixels et correspond à **100 « mini-colonnes »** ou « **segment** » de taille 8 pixels (soit 8 bits) chacune.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

BROCHAGE

Le brochage du DMX 973B est presque identique avec celui du DMX 973.

Ce qui présente l'avantage de pouvoir remplacer l'un par l'autre sans trop de modifications hardware.

1. 0V.	}Power supply.
2. +5V.	
3. RESET	Signal de RESET (actif à l'état bas)
4. RS.	Register Select input.
5. R/ $\overline{W}$	Read/Write input.
6. E	Enable signal.
7-14. D0-D7.	8Bit bi-directional μ P data bus.
15. LMP-.	}Backlight supply, maximum 90mA.
16. LMP+.	

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

BROCHAGE (suite)

Broches 1.et 2. : masse et 5V

Pour l'alimentation du LCD.

Broche 3. : RESET

Mis à l'état bas pendant au moins 1 μ s, il permet d'initialiser le LCD.

Broche 4. : RS

De façon analogue au DMX 973, cette broche permet de sélectionner :

↪ soit le registre de commande : RS = 0,

↪ soit le registre de données : RS = 1.

Broche 5. : R/ $\overline{W}$

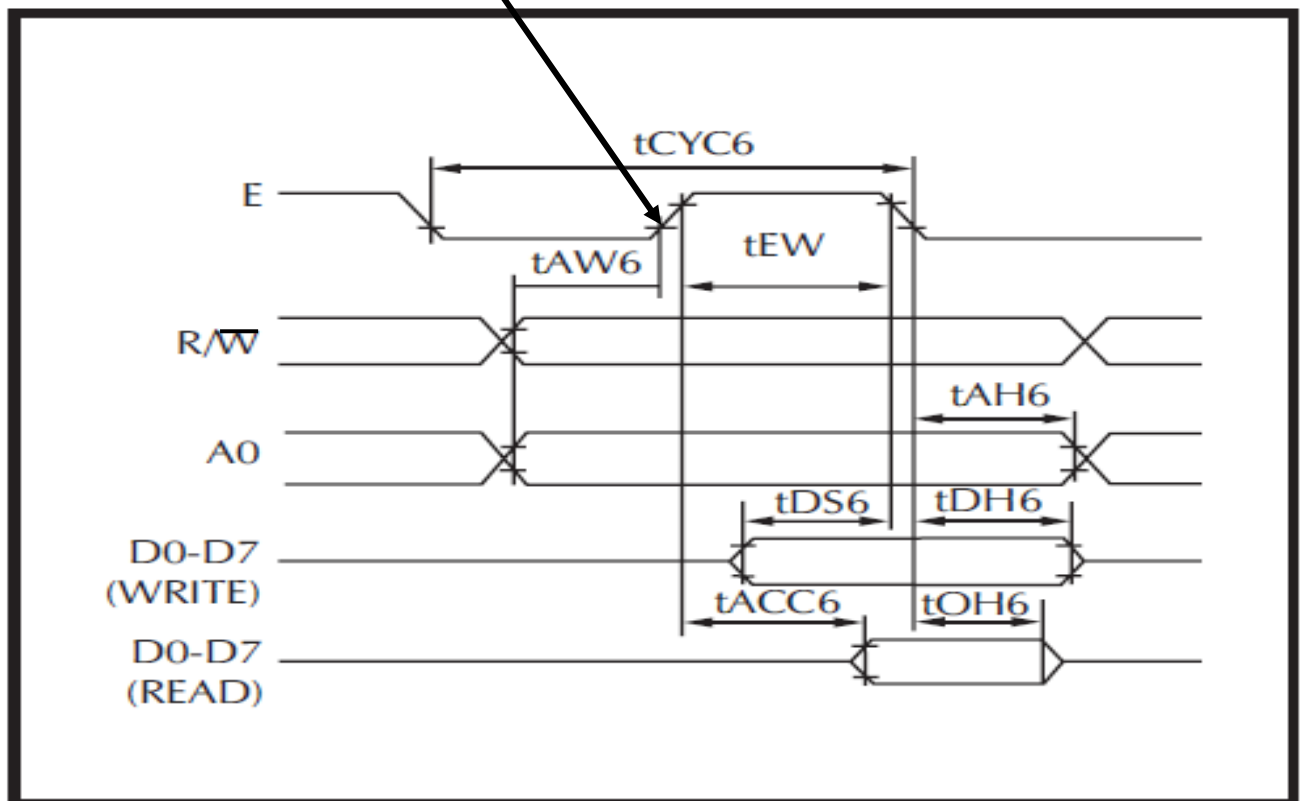
Pour différencier une lecture et une écriture.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

BROCHAGE (suite)

Broche 6. : E

De façon analogue au DMX 973, cette broche sert, en la « **pulsant** », à valider un cycle de lecture ou d'écriture



Broches 7. à 14. : D0 à D7

Pour les échanges en **lecture** ou en **écriture**.

Broches 15.et 16. : LMP^- et LMP^+

Pour le **rétro éclairage** : utile si obscurité ambiante.

Résumé du cours PIC : ANNEXE 7
(informations supplémentaires pour des projets)

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

LISTE DES INSTRUCTIONS

Command	Code											Function
	AD	RD	WR	D7	D6	D5	D4	D3	D2	D1	D0	
(1) Display ON/OFF	0	1	0	1	0	1	0	1	1	1	0 1	Turns on LCD panel when goes high, and turns off when goes low.
(2) Initial Display Line	0	1	0	0	1	Start display address						Specifies RAM display line for COM0.
(3) Set Page Address	0	1	0	1	0	1	1	Page address				Sets the display RAM page in Page Address register.
(4) Set Column Address 4 higher bits	0	1	0	0	0	0	1	Higher column address				Sets 4 higher bits of column address of display RAM in register
(4) Set Column Address 4 lower bits	0	1	0	0	0	0	0	Lower column address				Sets 4 lower bits of column address of display RAM in register
(5) Read Status	0	0	1	Status				0	0	0	0	Reads the status information.
(6) Write Display Data	1	1	0	Write data							Writes data in display RAM.	
(7) Read Display Data	1	0	1	Read data							Reads data from display RAM.	
(8) ADC Select	0	1	0	1	0	1	0	0	0	0	0 1	Sets normal relationship between RAM column address and segment driver when low, but reverses the relationship when high.
(9) Normal/Reverse Display	0	1	0	1	0	1	0	0	1	1	0 1	Normal indication when low, but full indication when high.
(10) Entire Display ON/OFF	0	1	0	1	0	1	0	0	1	0	0 1	Selects normal display (0) or Entire Display ON (1).
(11) Set LCD Bias	0	1	0	1	0	1	0	0	0	1	0 1	Sets LCD drive voltage bias ratio.
(12) Read-Modify-Write	0	1	0	1	1	1	0	0	0	0	0	Increments Column Address counter during each write when high and during each read when low.
(13) End	0	1	0	1	1	1	0	1	1	1	0	Releases the Read-Modify-Write.
(14) Reset	0	1	0	1	1	1	0	0	0	1	0	Resets internal functions.
(15) Set Output Status Register	0	1	0	1	1	0	0	0	1	"	"	Selects COM output scan direction. " Invalid data
(16) Set Power Control	0	1	0	0	0	1	0	1	Operation status			Selects the power circuit operation mode.
(17) Set Electronic Control Register	0	1	0	1	0	0	Electronic control value					Sets V5 output voltage to Electronic Control register.
(18) Set Standby	0	1	0	1	0	1	0	1	1	0	0 1	Selects standby status. 0: OFF 1: ON
(19) Power Save	-	-	-	-	-	-	-	-	-	-	-	Compound command of display OFF and entire display ON
(20) Test Command	0	1	0	1	1	1	1	"	"	"	"	IC Test command. Do not use!
(21) Test Mode Reset	0	1	0	1	1	1	1	0	0	0	0	Command of test mode reset

Note: Do not use any other command, or the system malfunction may result.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

REGISTRES CONCERNES PAR LES INSTRUCTIONS

RS = A0 R/W	0	1
0	Ecriture dans le registre de commande : instructions 1 à 4 et 8 à 21	Ecriture dans le registre de données : instruction 6
1	Lecture du registre de commande : instruction 5	Lecture du registre de données : instruction 7

C'est au niveau de ces instructions de contrôle et de commande que les différences avec le DMX 973 sont les plus importantes.

Les 8 premières instructions sont les mêmes que pour le DMX 973, avec cependant quelques différences que nous allons détailler plus loin.

Nous donnerons aussi quelques éléments complémentaires sur les autres instructions.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

Exemples d'utilisation :

➤ **instruction 1** : (idem que le DMX 973)

pour **allumer ou éteindre** l'écran (pas le contrôleur !!!)
ce qui permettra par exemple les définition suivantes :

```
#define ecran_lcd_on      0xAF      // allume l'écran du LCD
#define ecran_lcd_off    0xAE      // éteint l'écran du LCD
```

➤ **instruction 2** : (code différent avec le DMX 973)

choix de la ligne qui sera affichée en haut de l'écran
(permet par exemple de faire du « scrolling » d'écran)

```
#define ligne_haut_ecran  0x40      // du haut de l'écran COM0
// rajouter un numéro de ligne (de 0 à 31) pour faire du "scrolling"
```

➤ **instruction 3** : (code différent avec le DMX 973)

sélection page xxxx : **00** pour la page **0**,
 01 pour la page **1**,
 10 pour la page **2**,
 11 pour la page **3**.

écrire **0b 1011 00xx** dans le registre de commande

Grâce à cette instruction, le LCD tient à jour un **pointeur de page**. On pourra par exemple définir :

```
#define lcd_p_0      0xB0      // page 0 du LCD
#define lcd_p_1      0xB1      // page 1 du LCD
#define lcd_p_2      0xB2      // page 2 du LCD
#define lcd_p_3      0xB3      // page 3 du LCD
```

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

↳ **instruction 4** : (code différent avec le DMX 973)

sélection « mini-colonne » hhhh llll (de 0 à 99)

Grâce à cette instruction, le LCD tient à jour un **pointeur de « mini-colonne »**.

Cette instruction s'exécute en 2 temps :

- il faut d'abord envoyer les **4 bits de poids forts hhhh** du pointeur de « **mini-colonne** » :

soit d'après la documentation : **0b 0001 hhhh**

Mais attention, des tests ont montré qu'en fait il faut en plus rajouter 0x01 à cette valeur, un peu comme si les 16 premières colonnes n'étaient pas affichées sur l'écran. Ce qui conduit à la définition :

```
#define   colonne_base_high      0x11      // et non 0x10 !!!  
// colonne 17 qui est en fait la première affichée sur l'écran !!!
```

- puis les **4 bits de poids faibles llll** de ce pointeur.
Ce qui conduit à la définition :

```
#define   colonne_base_low      0x00      // colonne 0
```

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

↪ **instruction 5** : (idem que le DMX 973)

pour **lire le registre d'état**.

Utile par exemple pour scruter l'état du **bit 7 BUSY** (ou **BUSY_FLAG**).

Ce bit est à l'état haut quand une commande est en cours d'exécution. Il passe à 0 à la fin d'exécution de cette commande en cours. En le scrutant, on évite ainsi d'envoyer une nouvelle commande alors que la précédente n'a pas terminé son exécution.

↪ **instruction 6** : (idem que le DMX 973)

pour **donner une valeur** sur 8 bits **D7-D0** à la « **mini-colonne** » désignée par le pointeur de page (fixé par l'instruction 3 précédente) et par le pointeur de « **mini-colonne** » (fixé par l'instruction 4 précédente).

- si bit à **1** : pixel **allumé**,
- si bit à **0** : pixel **éteint**.

D0 correspond au pixel de la ligne de plus petit numéro dans la page concernée et **D7** au pixel de la ligne de plus grand numéro dans cette même page.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

↪ **instruction 7** : (idem que le DMX 973)
pour **lire** la valeur sur 8 bits de la « mini-colonne »
désignée par le pointeur de page (fixé par l'instruction 3
précédente) et par le pointeur de « mini-colonne » (fixé
par l'instruction 4 précédente) :

- si pixel **allumé** : bit à **1**,
- si pixel **éteint** : bit à **0**.

Notons que pour les accès au registre de données (en
écriture ou en lecture par les instructions **6** et **7**
précédentes), le pointeur de « mini-colonne » est
automatiquement incrémenté, ce qui permet d'enchaîner
plusieurs accès successifs sans mettre à jour ce pointeur.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

Les autres instructions servent à configurer le LCD.

↳ **instruction 8** : (idem que le DMX 973)
permutation des colonnes (la dernière devient la première, l'avant-dernière la deuxième...) pour faire un effet « miroir ». D'où les définitions suivantes :

```
#define    lcd_normal    0xA0    // mode direct normal
#define    lcd_miroir    0xA1    // sens inverse : effet miroir
```

↳ **instruction 9** : (n'existe pas sur le DMX 973)
pour passer en **vidéo inverse** sur la **totalité** de l'écran.
(sans effacer le contenu de la mémoire).

- si bit à **0** : pixel **allumé**,
- si bit à **1** : pixel **éteint**.

D'où les définitions suivantes :

```
#define    ecran_video_normale 0xA6    // écran en vidéo normale
#define    ecran_video_inverse 0xA7    // écran en vidéo inverse
```

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

↪ **instruction 11** : (n'existe pas sur le DMX 973)
pour **jouer sur le réglage** des tensions internes de
pilotage de l'écran. Dans la documentation, il est conseillé
de faire l'initialisation avec :

```
#define    lcd_bias        0xA3
```

↪ **instruction 14** : (idem que le DMX 973)
pour **faire un RESET logiciel**. A faire en général au tout
début de l'initialisation du LCD

↪ **instruction 16** : (n'existe pas sur le DMX 973)
pour **choisir les modes internes d'alimentation**. Dans la
documentation, il est conseillé de faire l'initialisation
avec :

```
#define    lcd_power       0x2F
```

↪ **instruction 17** : (n'existe pas sur le DMX 973)
pour **choisir et régler le contraste** entre :

```
#define    contraste_min   0x80        // contraste minimum
```

```
#define    contraste_max   0x9F        // contraste maximum
```

Le contraste pourra être fixé une fois pour toute (valeur
conseillée dans la documentation **0x88**) ou réglable par
logiciel et par l'utilisateur.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

Voici enfin une liste (non exhaustive) de fonctions de « bas niveau » qu'il est intéressant d'écrire pour piloter le LCD graphique.

Comme d'habitude, il faut penser d'abord à la **structuration** du projet, pour le rendre réutilisable dans un autre contexte.
(s'inspirer de la méthode déjà utilisée dans les projets précédents...).

Ces fonctions seront donc mises dans un fichier nommé par exemple **lcdgraphique973Bbt.c** qu'il faudra attacher à tous les projets utilisant ce LCD.

Sans oublier d'écrire un fichier **lcdgraphique973Bbt.h** pour les aspects hardware, comme par exemple les #define précédemment définis et les prototypes de fonctions.

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

```
// *****  
// envoi d'une commande de valeur commande dans le  
// registre de commande  
// contrainte : respect des chronogrammes précédents  
void write_commande (unsigned char commande)  
// *****
```

```
// *****  
// envoi d'une donnée de valeur data dans le  
// registre de données  
// data représente la valeur d'une « mini-colonne »  
// (pixels à allumer ou à éteindre)  
// contrainte : respect des chronogrammes précédents  
void write_data (unsigned char data)  
// *****
```

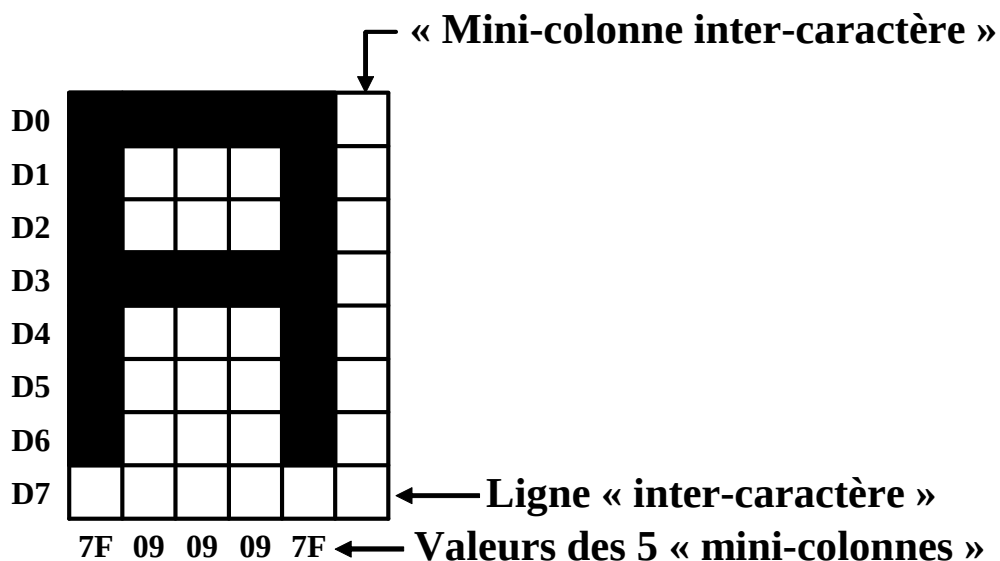
```
// *****  
// lecture d'une donnée dans le registre de données  
// la valeur retournée est la valeur de la « mini-colonne »  
// (pixels allumés ou éteints)  
// contrainte : respect des chronogrammes précédents  
unsigned char read_data (void)  
// *****
```

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

```
// *****
// écriture d'un caractère de code ASCII car sur le LCD
// principe : voir ci-dessous
// penser à utiliser la fonction write_data précédente ...
void write_char (unsigned char car)
// *****
```

Le caractère 'A' en font 5 × 7

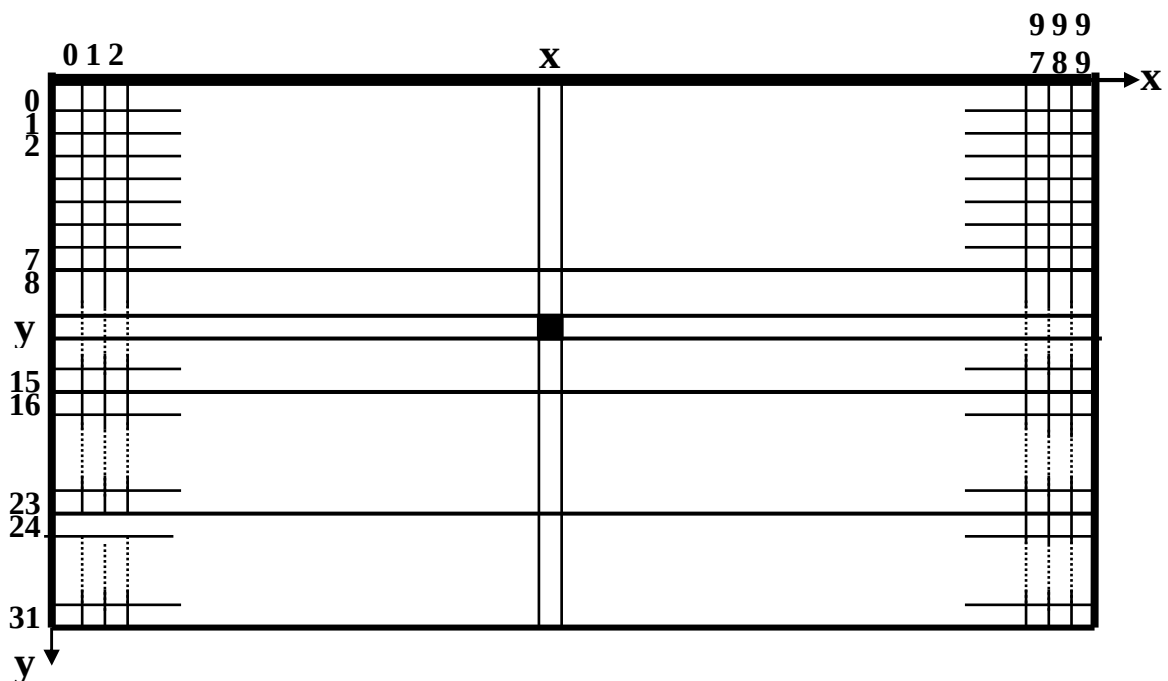


Remarque : on pourrait aussi passer en paramètre la position où on veut afficher ce caractère : sa page et sa « mini-colonne » de début dans cette page...

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

Exemples de fonctions de « bas-niveau » :

```
// *****
// allumage (si etat = 1) ou extinction (si etat = 0) d'un
// pixel de coordonnées x et y sur le LCD
// principe : voir ci-dessous
void pixel (unsigned char x, unsigned char y, char etat)
// *****
```



Il faut donc, à partir des paramètres x et y :

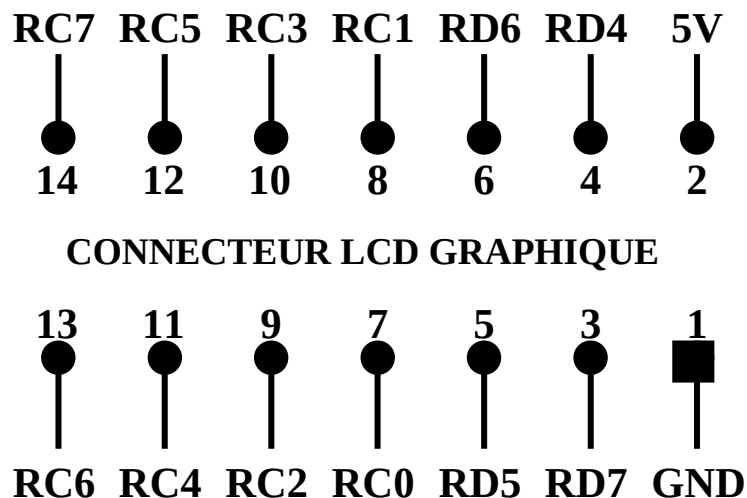
- trouver la page où se trouve le pixel concerné,
- trouver dans cette page la ligne où se trouve ce pixel,
- allumer ce pixel si $etat = 1$ ou l'éteindre si $etat = 0$.

Les deux pages suivantes peuvent aussi vous être utiles !!!

COMPLEMENTS D'ELEMENTS POUR UN LCD GRAPHIQUE (suite)

DERNIERE AIDE

Si vous utilisez les **PORTC** et **PORTD** pour relier votre **LCD 973B**, conformément au schéma ci-dessous :



alors vous pouvez le tester sous **MPLAB** en faisant :

File Import... et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

fichiers HEX OK/source en C le fichier :

lcd graphique 973B.hex (c'est un exécutable)

Puis choisir l'outil **MPLAB ICD3** par :

Debugger Select Tool, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.

Et enfin lancer ce programme.

Note : après réglage du contraste, il faut appuyer sur **RB0** et **RA4** en même temps !! pour continuer...

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR

INTRODUCTION GENERALE

Les **servomoteurs** sont très utilisés dans l'industrie (pour la commande d'électrovannes par exemple) mais aussi en **robotique** (saisie et/ou déplacement d'objets, positionnement et/ou orientation de divers capteurs, gestion d'articulations, bref mouvements en tous genres...).

Il en existe une grande variété de modèles, couvrant une large gamme de puissance et donc d'applications.

**(voir quelques documentations dans le répertoire
documentations de votre répertoire de travail)**

QUEL EST LEUR RÔLE ?

Ils permettent **d'atteindre** une **position angulaire** donnée (régime **transitoire**, le plus **bref** possible) et de la **maintenir**, même en cas de couple contraire important (régime **permanent**, le plus **stable** et le plus **résistant** possible).

Ce sont des systèmes **asservis** (par une boucle de rétroaction).

On parle d'ailleurs d'**asservissement de position**.

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

DE QUOI SONT-ILS CONSTITUES ?

- d'un **moteur** bien sûr (souvent à courant continu),
- d'un **système de mesure de la position angulaire réelle** : souvent un potentiomètre rotatif (appelé aussi **potentiomètre de recopie**) solidaire de l'axe du moteur (sa résistance varie donc avec l'angle réel de rotation et donne ainsi une image de cet angle),
- et d'une **électronique de commande** (en fait la boucle d'asservissement) qui compare en permanence la **position angulaire souhaitée (consigne)** avec la **position réelle**.

L'idée est de maintenir ces deux valeurs égales (régime permanent).

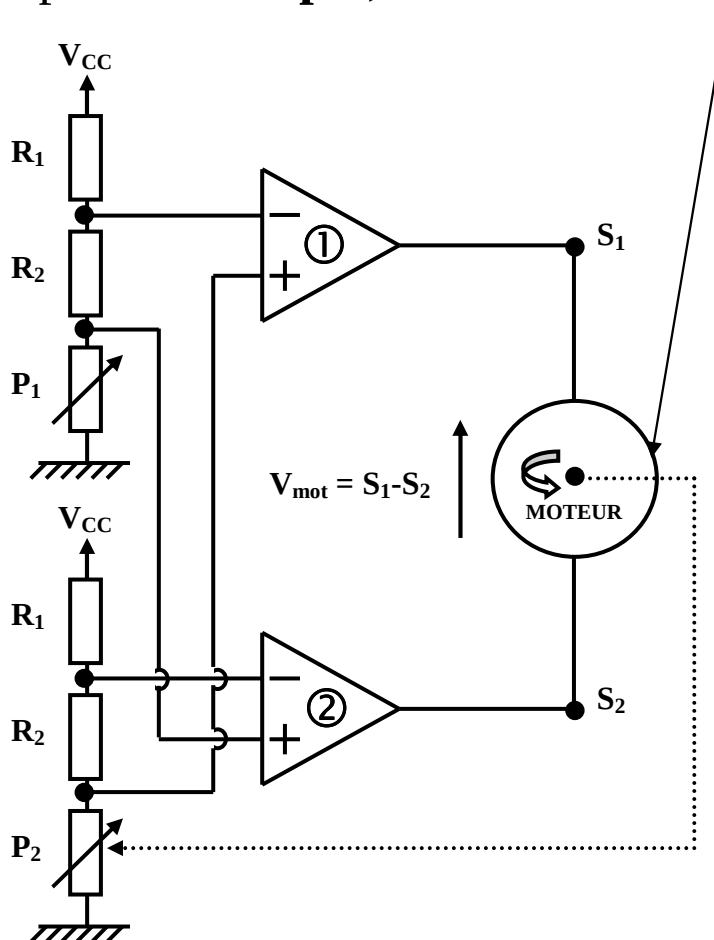
Lorsque ces deux valeurs sont différentes (suite à un changement de consigne par exemple), le moteur reçoit l'ordre de tourner dans un sens ou dans l'autre pour annuler cette différence (régime transitoire).

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

POUR LES PLUS CURIEUX !!! PRINCIPE DE L'ASSERVISSEMENT

Voici un schéma possible pour un asservissement de position utilisant 2 AOP (① et ②) fonctionnant en comparateur (donc leurs sorties S_i valent $\pm V_{CC}$) et deux potentiomètres rotatifs :

- P_1 = pour régler la **consigne**,
- P_2 = pour la **recopie**, fixé à l'axe du servomoteur.



ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

POUR LES PLUS CURIEUX !!! ANALYSE QUALITATIVE DE L'ASSERVISSEMENT

Valeurs possibles pour la tension $V_{\text{mot}} = S_1 - S_2$:

$S_1 \backslash S_2$	$+V_{CC}$	$-V_{CC}$
$+V_{CC}$	0 : arrêt	$+2.V_{CC}$: tourne sens 1
$-V_{CC}$	$-2.V_{CC}$: tourne sens 2	0 : arrêt

$$e_1^- = \frac{R_2 + P_1}{R_1 + R_2 + P_1}, e_1^+ = \frac{P_2}{R_1 + R_2 + P_2}, e_2^- = \frac{R_2 + P_2}{R_1 + R_2 + P_2}, e_2^+ = \frac{P_1}{R_1 + R_2 + P_1}$$

Supposons que $P_1 = P_2$. Alors :

$$e_1^- > e_1^+ \Rightarrow S_1 = -V_{CC} \text{ et } e_2^- > e_2^+ \Rightarrow S_2 = -V_{CC}$$

$V_{\text{mot}} = 0$: le moteur ne tourne pas :

c'est un état **d'équilibre**.

A partir de cet état initial, supposons qu'on change la consigne en diminuant par exemple P_1 .

Alors e_1^- et e_2^+ diminuent, et à partir d'une certaine valeur de P_1 , e_1^- deviendra inférieur à e_1^+ et S_1 passera alors à $+V_{CC}$, S_2 restant quant à lui à $-V_{CC}$. Donc V_{mot} passe à $+2.V_{CC}$ et le moteur va tourner dans un sens, celui qui aura tendance à diminuer P_2 pour retrouver la nouvelle valeur de P_1 : c'est **l'asservissement qui agit**.

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

POUR LES PLUS CURIEUX !!! ANALYSE QUALITATIVE DE L'ASSERVISSEMENT

Notons au passage que le sens de branchement du moteur a une importance, car si on le branche dans l'autre sens (celui qui a tendance à augmenter P_2), l'écart entre P_1 et P_2 se creuse de plus en plus et on n'a plus d'asservissement :

↳ le montage est alors **instable** et la boucle de rétroaction « **négative** » devient une boucle de rétroaction « **positive** » et tend à amplifier cette instabilité, un peu comme quand on fait un rebouclage sur l'entrée non inverseuse + d'un A.O.P.

On pourrait faire un raisonnement similaire, toujours en partant de l'état initial d'équilibre, mais cette fois-ci en augmentant cette P_1 .

A partir d'une certaine valeur de P_1 , V_{mot} passera à $-2.V_{CC}$ et le moteur tournera alors dans l'autre sens, celui qui a tendance à augmenter P_2 .

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

POUR LES PLUS CURIEUX !!! ANALYSE QUALITATIVE DE L'ASSERVISSEMENT

RÔLE DE LA RESISTANCE R_2

La résistance R_2 sert à fixer 2 seuils de basculement (un **seuil bas** et un **seuil haut**) pour les 2 comparateurs (qui deviennent alors des comparateurs à hystérésis) et permet d'éviter l'effet de « sonnette » :

↳ si R_2 était nulle, au moindre écart entre les valeurs de P_1 et de P_2 , les 2 sorties S_1 et S_2 se mettent à « osciller » intempestivement, (à cause entre autre de l'inertie du moteur) ce qui entraînerait une instabilité de l'asservissement et des oscillations du moteur autour de la position d'équilibre).

Il ne faut donc pas prendre R_2 trop petite.

Par contre si R_2 est choisie **trop grande**, la fenêtre de l'asservissement (entre les 2 seuils) serait trop grande, ce qui entraînerait une **moins bonne précision**.

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

POUR LES PLUS CURIEUX !!! ANALYSE QUALITATIVE DE L'ASSERVISSEMENT

Moralité et conclusion :

Dans un asservissement, et cette règle est assez générale

**précision et stabilité sont souvent
contradictoires,
tout est question de dosage !!!**

Il faut également bien avoir en tête que le schéma que l'on vient de présenter **doit et doit seulement** être interprété comme un schéma de principe.

Dans les servomoteurs que l'on présentera plus loin et qui sont utilisés en pratique, la consigne n'est pas réglée manuellement par un potentiomètre P_1 comme ici, mais par la largeur d'impulsion d'un signal PWM.

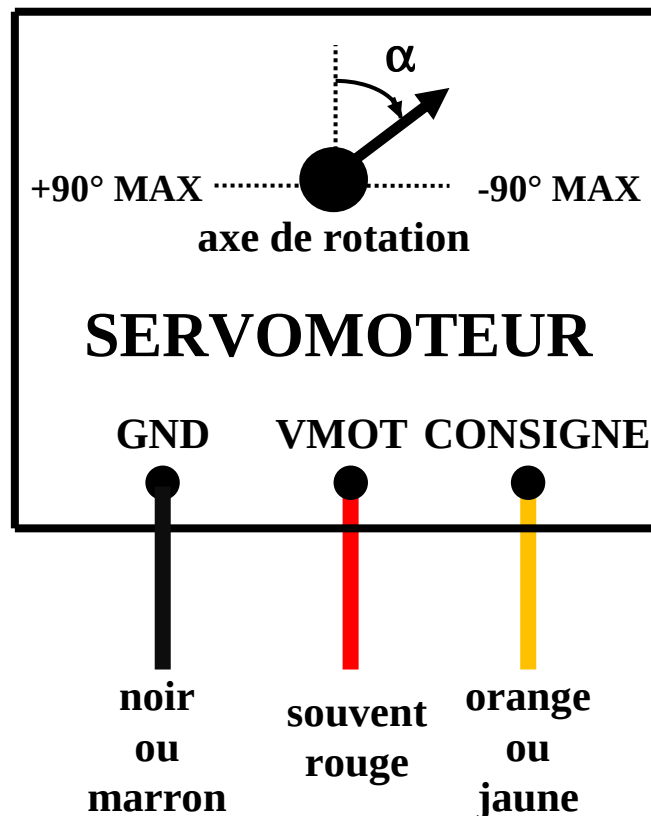
ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

DEUX REMARQUES

- 1.** On pourrait aussi utiliser un moteur pas à pas pour ce genre d'application (cf. annexe 5) mais :
 - risque de sauter des pas et donc d'avoir une dégradation de la précision,
 - en général moins de couple de maintien disponible avec un moteur pas à pas.

- 2.** En pratique, et pour augmenter le couple de maintien du servomoteur (ce qui permet de maintenir la position angulaire même en cas de couple contraire important), le potentiomètre de recopie n'est pas directement monté sur l'axe du moteur mais sur un axe secondaire, mécaniquement relié à l'axe du moteur par un système type « roue dentée à engrenages » permettant, aux pertes de frottement près, de diminuer la vitesse de rotation de l'axe secondaire et ainsi d'augmenter le couple de maintien sur cet axe secondaire.

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite) CÂBLAGE FREQUENT AVEC 3 FILS



💣 💣 **Attention** : l'alimentation **GND / VMOT** du servomoteur sera **obligatoirement** une alimentation 0-5V **extérieure** (car l'alimentation 5V de la carte PICDEM 2 PLUS ne peut pas fournir assez de courant pour le servomoteur et risquerait **sérieusement** (voire même à coup sûr) d'être **définitivement détériorée !!!**)

Avant d'aborder comment gérer l'entrée **CONSIGNE**, il faut faire le lien avec l'**angle de rotation α** .

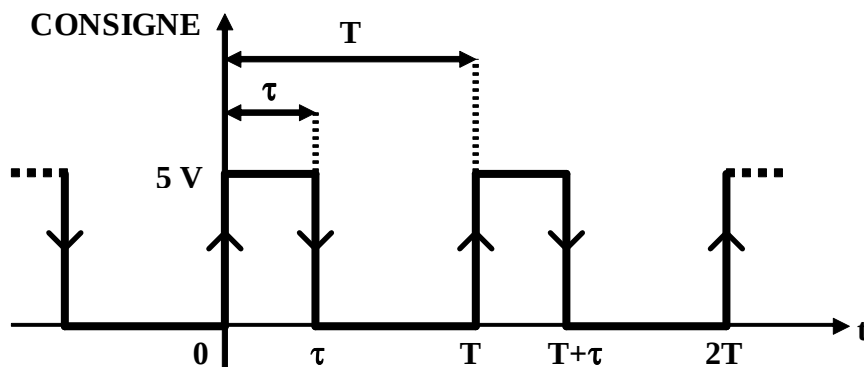
ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

ANGLE DE ROTATION α ET CONSIGNE

Cet angle varie dans l'intervalle $[-90^\circ; +90^\circ]$ par rapport à une position centrale, des butées mécaniques empêchant une rotation au-delà de cet intervalle.

Lorsqu'on n'envoie aucun signal sur l'entrée **CONSIGNE**, l'axe de rotation tourne librement « à la main », sans exercer de couple de maintien dans une position donnée (asservissement déverrouillé).

Pour valider l'asservissement, et donc utiliser « normalement » le servomoteur, il faut envoyer un signal de type **PWM** sur l'entrée de consigne, avec deux paramètres **T** et τ .



ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

ANGLE DE ROTATION α ET CONSIGNE (suite)

Comment choisir les paramètres T et τ du signal PWM ?

Si :

- **T est trop grand** : on n'a plus de couple de maintien au-delà de 100 ms,
- **T est trop petit** : on assiste à une instabilité de l'asservissement en dessous de 5 ms,

le servomoteur **ne fonctionne pas** correctement.

PREMIERE CONCLUSION :

Il est souvent préconisé, lorsque c'est possible, de prendre :

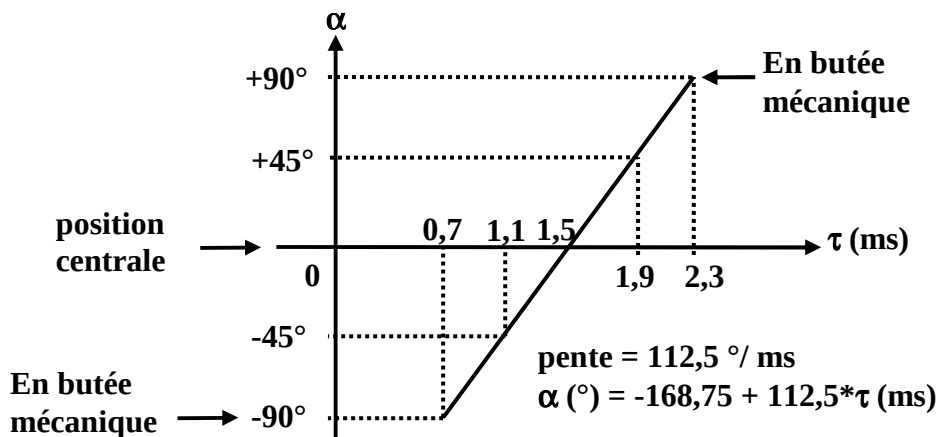
$$\boxed{T = 20 \text{ ms.}}$$

(valeur que l'on retrouve souvent dans la « littérature »)

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite)

ANGLE DE ROTATION α ET CONSIGNE (suite)

Pour la valeur de τ (bien sûr $< T$) un étalonnage avec un servomoteur **NARO+ F** de chez **GWS** a montré qu'il y a une relation **affine** entre α et τ ($\forall T \in [5ms; 100ms]$) comme indiqué ci-après, mais il semble qu'on obtienne des résultats tout à fait comparables pour la plupart des servomoteurs de base (valeurs à peu près normalisées).



SECONDE CONCLUSION :

On retiendra les valeurs suivantes :

Valeurs de τ	Angle α de rotation
0,7 ms	-90 : limite de la butée mécanique
1,5 ms	0 : position centrale
2,3 ms	+90 : limite de la butée mécanique

Par sécurité, on pourra prendre : **$0,8 ms < \tau < 2,2 ms$**

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite et fin)

Dernières pages qui peuvent aussi vous être utiles !!!

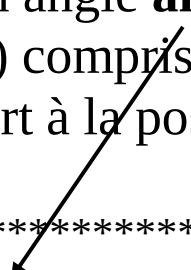
DERNIERE AIDE

Comme d'habitude (méthode déjà vue et qui doit maintenant être un réflexe) structurez bien votre projet en écrivant tout d'abord deux fichiers :

↳ **servomoteur.h** : pour le hardware,
(définition des broches, prototypes des fonctions...)

↳ **servomoteur.c** : pour les fonctions de « bas niveau »,
comme par exemple :

```
// *****  
// initialisation des broches du PIC concernées  
// *****  
void init_servomoteur (void)  
// *****  
// *****  
// génération d'un signal PWM sur l'entrée CONSIGNE  
// permettant de tourner d'un angle alpha (passé en  
// paramètre et exprimé en °) compris entre -90° et +90°  
// (valeur comptée par rapport à la position centrale) et  
// de maintenir cet angle  
// *****  
void tourne (signed char alpha)  
// *****  
...
```



ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite et fin)

DERNIERE AIDE (suite)

Notez qu'avec ces fonctions, il est facile :

- de saisir un objet :

```
void prend (void)
{
    tourne (90);      // voire tourne(80) pour éviter
}                    // la butée mécanique
```

- de lâcher un objet :

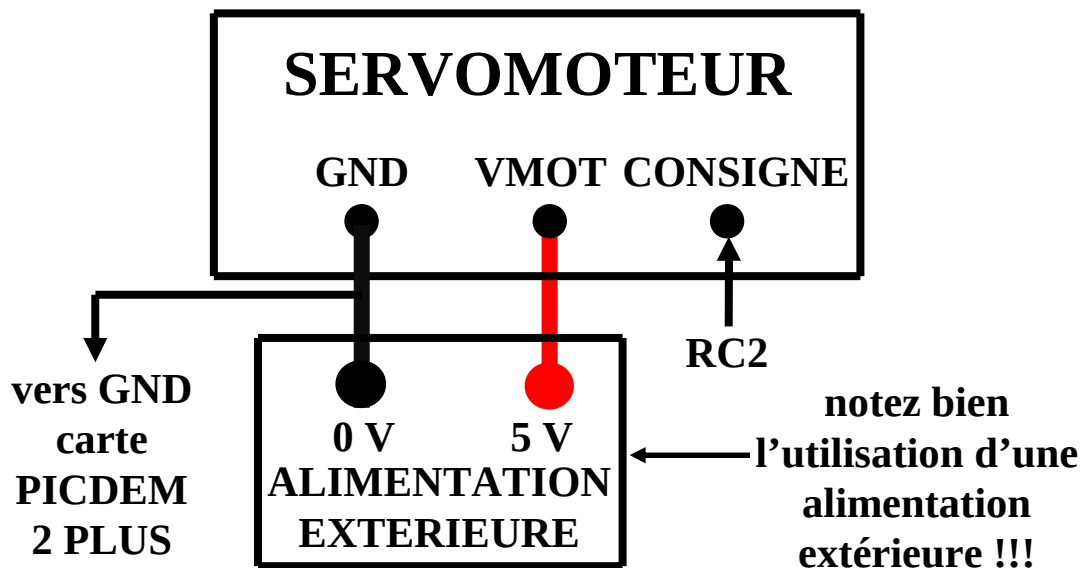
```
void lache (void)
{
    tourne (-90);     // voire tourne(-80) pour éviter
}                    // la butée mécanique
```

...

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite et fin)

DERNIERE AIDE (suite)

Si de plus, vous utilisez le câblage suivant pour relier le servomoteur à la carte **PICDEM 2 PLUS** :



alors vous pouvez le tester sous **MPLAB** en faisant :

File Import... et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

fichiers HEX OK/source en C le fichier :

servomoteur lcd ascii.hex (c'est un exécutable)

ELEMENTS SUR L'IMPLANTATION D'UN SERVOMOTEUR (suite et fin)

DERNIERE AIDE (suite et fin)

Puis choisir l'outil **MPLAB ICD3** par :

Debugger Select Tool, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.

Et enfin lancer ce programme et laissez vous guider...

***Note** : pour avancer dans cette démonstration il faut appuyer sur **RB0** et **RA4** en même temps !!!*

*Vous pouvez également visualiser sur un oscilloscope en même temps le signal PWM sur **RC2**.*

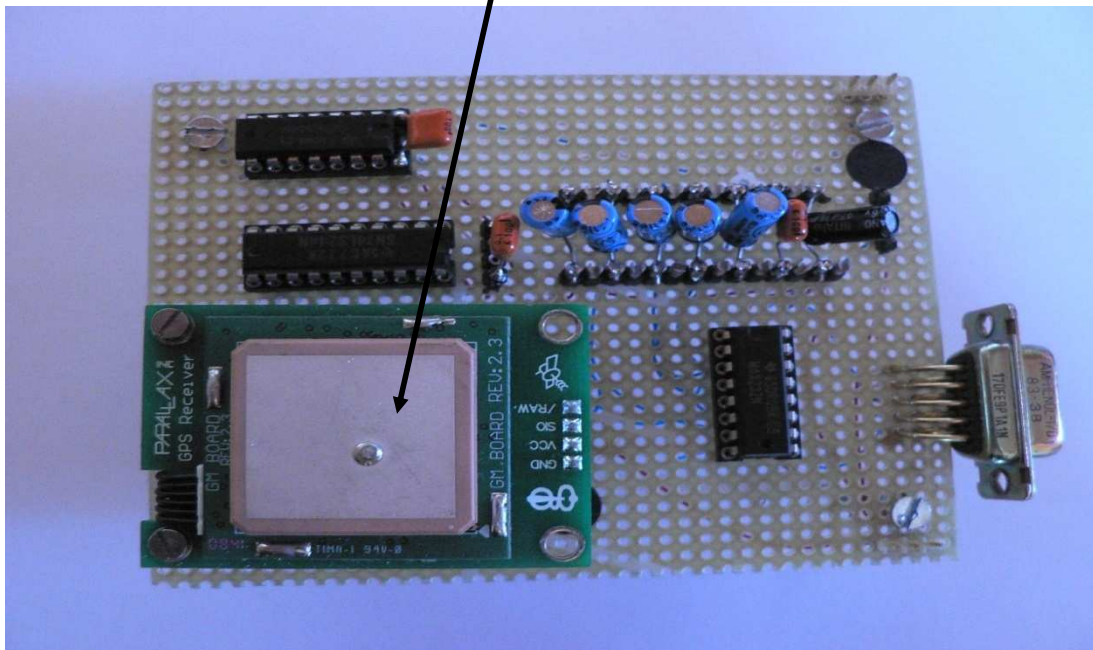
ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS

INTRODUCTION GENERALE

Le système **GPS** (Global Positioning System) développé par les U.S.A. permet, grâce à un réseau de 24 satellites, de se positionner n'importe où dans le monde grâce au procédé de la triangularisation.

Si l'horloge interne du **récepteur GPS** était parfaitement synchrone avec celle, commune à tous ces satellites, seuls trois satellites en vue seraient suffisants pour assurer cette fonction de positionnement.

En pratique, un 4^{ème} satellite est nécessaire pour résoudre ce décalage de l'horloge du récepteur GPS.



ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite)

INTRODUCTION GENERALE (suite)

Le **récepteur GPS** utilisé est développé par la société **PARALLAX**.

Il permet, en plus de la position (latitude et longitude) de fournir également :

- l'heure UTC/GMT (**Greenwich Mean Time**),
- la date,
- l'altitude,
- la vitesse et la direction du déplacement...

FORMAT DES DONNEES FOURNIES

Les informations fournies par ce récepteur GPS sont au format **TTL NRZ série asynchrone** :

- 4800 bauds,
- 8 bits de données,
- pas de parité,
- 1 bit de stop.

Du coup, pour utiliser cette liaison série avec le **PIC**, il est vivement conseillé de se reporter au **chapitre VIII.7.** du polycopié de cours et plus particulièrement le sous chapitre **VIII.7.2.** sur les **liaisons séries asynchrones** et sur la **façon de les mettre en œuvre** avec le **PIC**.
En voici l'essentiel.

GENERALITES SUR LES LIAISONS

Deux grands types de procédés de liaison :

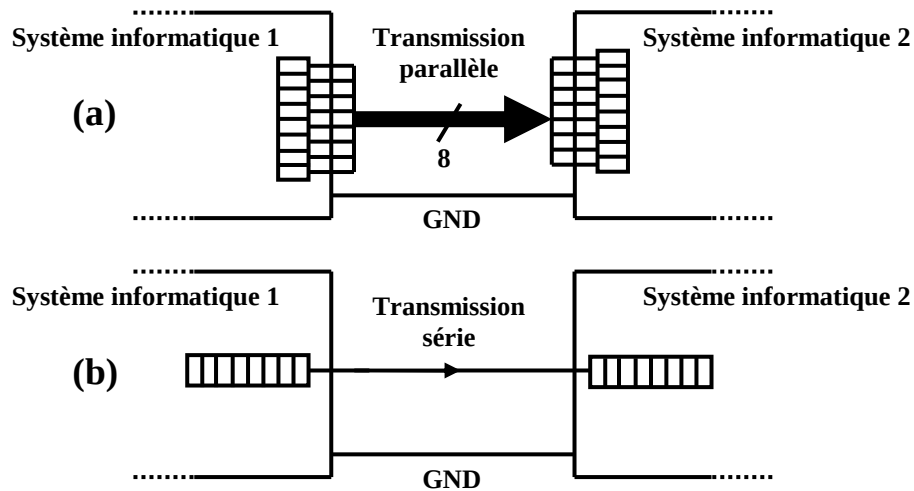


Figure VIII.115. (page 348)

- la transmission parallèle (a) :

les octets sont échangés les uns après les autres dans le temps mais les 8 bits constitutifs de chaque octet sont envoyés (et reçus) en même temps (dans le même coup d'horloge).

⇒ c'est **rapide** mais **beaucoup de fils**.

- la transmission série (b) :

les octets sont toujours échangés les uns après les autres dans le temps mais les 8 bits constitutifs de chaque octet sont envoyés (et reçus) les uns derrière les autres, suivant un rythme bien établi.

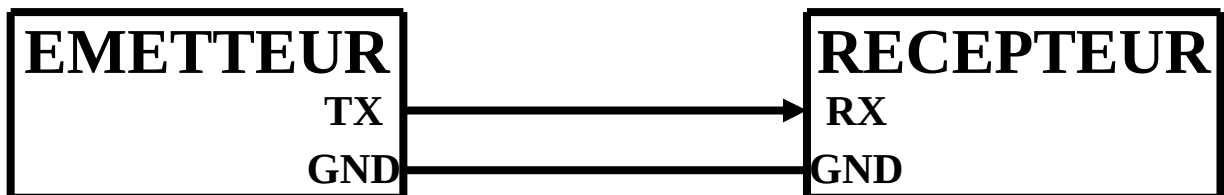
⇒ c'est **plus lent** mais **moins de fils**.

LIAISONS SERIES ASYNCHRONES

Sur une liaison série **asynchrone**, **pas d'horloge** entre :

- **l'émetteur** de bits : sortie **TX**,
- et **le récepteur** de bits : entrée **RX**.

Conclusion : deux fils suffisent au minimum pour une communication dans un sens.



Il faudra un fil supplémentaire soit trois fils au minimum pour une liaison « vrai full duplex » : dans les deux sens et éventuellement au même moment.

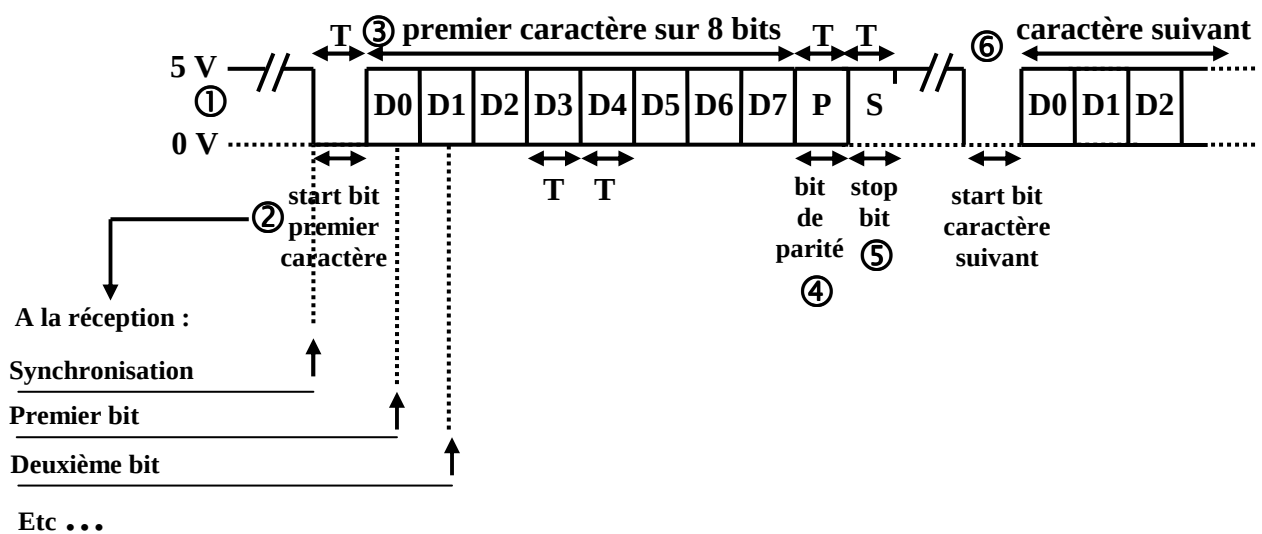
Les caractères sont envoyés sur **TX** (et reçus par **RX**) **les uns après les autres et bit après bit**.

Mais pour qu'ils se comprennent et que l'information transmise soit **correctement** reçue, il faut que le format de la liaison soit **le même** côté émetteur et côté récepteur, la synchronisation à la réception s'effectuant au début de chaque caractère et perdurant pendant la réception des bits du caractère, car chacun de ces bits a la même durée **T**, connue et de l'émetteur et du récepteur.

Le **débit** (en **bits/seconde** ou **baud**) vaudra donc **1/T**.

LIAISONS SERIES ASYNCHRONES (suite)

Voici le format complet d'une trame série asynchrone qu'on pourrait observer sur le fil TX → RX avec un oscilloscope.



① au **repos** (pas de caractère sur la liaison) la ligne est au **niveau haut** TTL soit **5V** :

appelé **état d'attente ou mark**.

Le niveau de **travail** est le **niveau bas** TTL soit **0V** :

appelé **space**.

② chaque caractère commence par un **start bit** au niveau **bas** faisant donc passer la ligne au **0V** et permettant la synchronisation (sur ce front descendant) du récepteur sur l'arrivée de ce caractère.

Comme les bits ont tous la même durée **T**, l'échantillonnage de chaque bit reçu (ou la stabilisation de chaque bit émis) se fera au milieu de chaque bit reçu (ou émis).

LIAISONS SERIES ASYNCHRONES (suite)

③ après le start bit, les 8 bits du caractère sont émis (ou reçus) les uns après les autres, en commençant par le **LSB D0**, au format très simple appelé **NRZ (Non Return to Zero)**:

- si le bit **$D_i = 1$** $\Rightarrow$ **5 V** sur la ligne,
- si le bit **$D_i = 0$** $\Rightarrow$ **0 V** sur la ligne.

④ après le **MSB D7**, on peut rajouter (ou pas) un bit **P** dit **bit de parité** permettant de détecter des erreurs de transmission.

↪ soit en parité dite **paire** :
si le **nombre de bits à 1** du caractère et **P** est **pair**.

↪ soit en parité dite **impaire** :
si le **nombre de bits à 1** du caractère et **P** est **impair**

⑤ après **P** (ou **D7**), se trouve le **bit de stop S**, au niveau **5V**, permettant systématiquement le retour de la ligne à l'état de repos, pour se préparer à l'émission (ou la réception) du caractère suivant.

⑥ le caractère suivant précédé de son start bit et au même format...

Conclusion

↪ pour transmettre 8 bits effectifs, il en faut 10 ou 11...

Ordre de grandeur

↪ à **9600 Bauds**, la transmission d'un caractère dure :
1 ms environ.

LIAISONS SERIES ASYNCHRONES (suite)

Exemple en TTL

Le format complet est donné ci-dessous pour le caractère ASCII 'A' = **0x41** = 0b 0100 0001

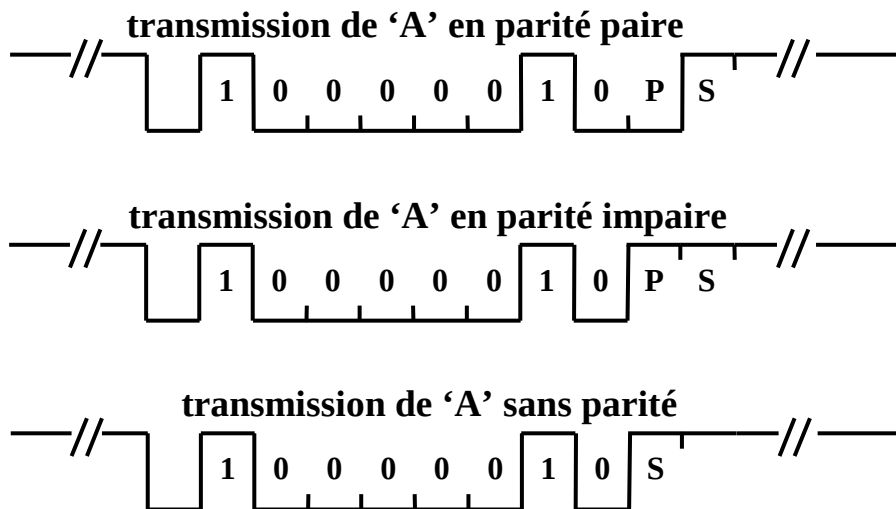


Figure VIII.120. (page 354)

En pratique, les deux niveaux TTL (0 V et 5 V) sont trop proches l'un de l'autre pour envisager des liaisons séries **fiables** sur des distances importantes (typiquement au-delà de 50 cm à 1 mètre).

↳ d'où la norme **V24** (Europe) ou **RS232** (U.S.A.) qui a « éloigné » ces niveaux en $\pm 12V$ avec la convention :

- le **+12 V RS232** correspond au **0 V TTL**,
- le **-12 V RS232** correspond au **5 V TTL**.

Note : il y a donc une **inversion logique** entre TTL et RS232.

LIAISONS SERIES ASYNCHRONES (suite)

Exemple en RS232

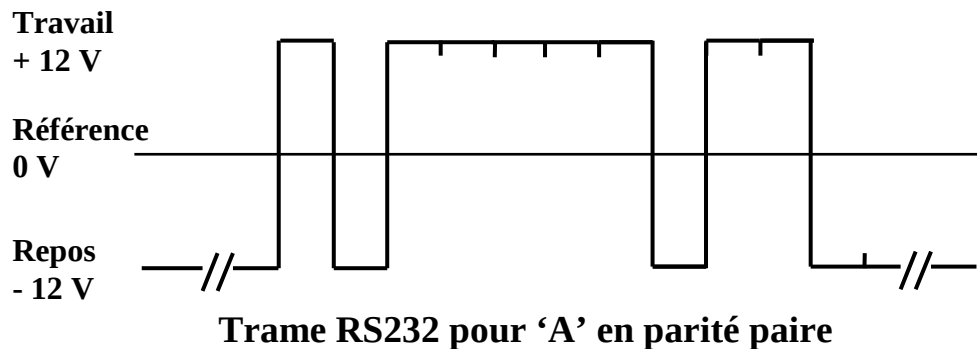


Figure VIII.125. (page 359)

Le signal sur une liaison RS232 :

- ↳ devient **bipolaire**,
- ↳ l'état de **repos** sur la ligne correspond au **-12 V**,
- ↳ et l'état de **travail** est le **+12V**.

Conclusion

- ↳ moins grande sensibilité aux bruits divers et aux parasites,
- ↳ et distance maximum possible de plusieurs mètres.

MODULE UART du PIC

UART : Universal Asynchronous Receiver Transmitter.

Emetteur : synoptique

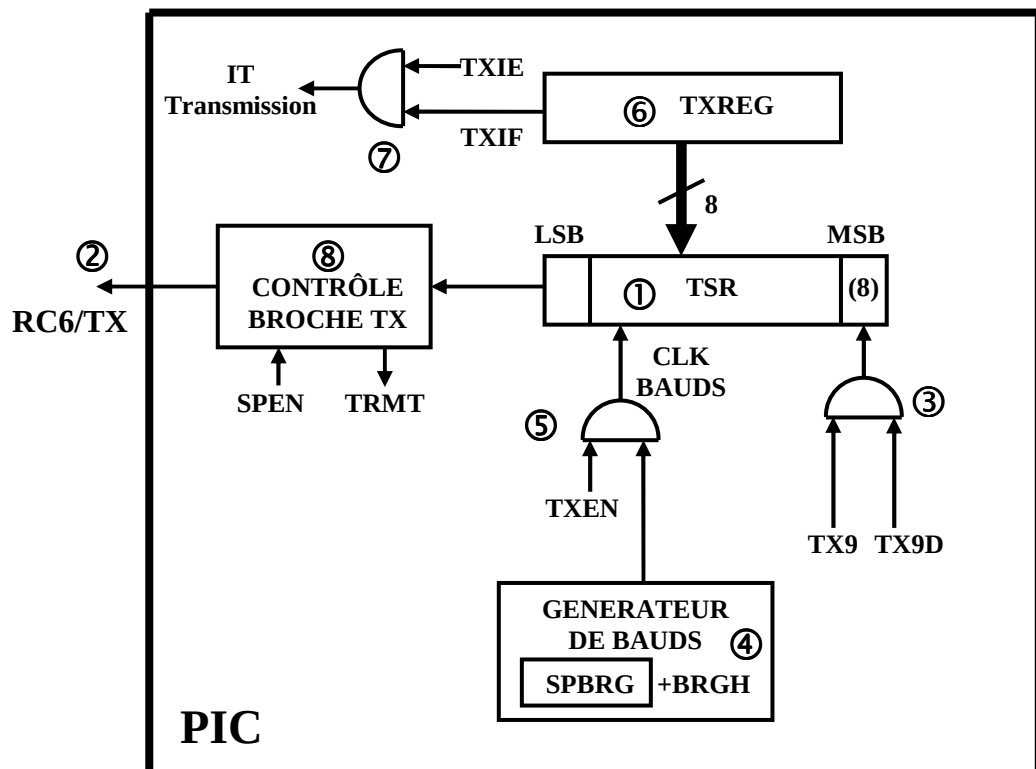


Figure VIII.127. (page 363)

① **le cœur** de l'émetteur :

registre à décalage **TSR** pour la conversion parallèle $\Rightarrow$ série vers la broche de sortie **RC6/TX** (②)

Grâce au bloc ③, on peut rajouter un 9^{ème} bit (comme la parité **P** par exemple, mais calculée par soft).

MODULE UART du PIC (suite)

Emetteur : synoptique (suite)

Le bloc ④ **GENERATEUR DE BAUDS** permet, grâce au registre **SPBRG** et au bit **BRGH**, de fabriquer l'horloge du registre à décalage **TSR** et de fixer ainsi la vitesse **V** de la liaison (exprimée en bauds) :

BRGH = 0 (mode low speed)	BRGH = 1 (mode high speed)
$V = \frac{F_{osc}}{64 \cdot (SPBRG + 1)}$	$V = \frac{F_{osc}}{16 \cdot (SPBRG + 1)}$

Figure VIII.128. (page 364)

Il faut bien sûr (cf. ⑤) mettre le bit **TXEN** à **1** (validation de l'émetteur) pour que le décalage des bits soit effectif.

L'accès à **TSR** (non mappé) se fait via **TXREG** (cf. ⑥) : une écriture dans **TXREG** (du caractère à transmettre) entraîne automatiquement le chargement de ce caractère dans **TSR** et le **début de l'émission** (si **TXEN=1**).

Le flag **TXIF** passe alors à **1** (**TXREG** vide) et sera géré :

- soit par **interruption** (cf. ⑦ si le masque **TXIE = 1**),
- soit par **polling**,

pour éviter que le chargement du caractère suivant (qui fera passer automatiquement **TXIF** à **0** : **TXREG** plein) n'ait lieu pendant l'émission du précédent et donc un écrasement.

Notons enfin le bit **SPEN** (cf. ⑧) pour valider le module **USART** en entier (y compris en mode Synchrone).

Résumé du cours PIC : chapitre VIII.7.2.
(informations supplémentaires pour des projets)

MODULE UART du PIC (suite)

Emetteur : registre de contrôle associé

TXSTA (situé à l'adresse 0x098)
(page 111 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R-1	R/W-0
TXSTA	CSRC	TX9	TXEN	SYNC	_	BRGH	TRMT	TX9D
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
CSRC		Utilisé dans le mode synchrone de l'U.A.R.T. Et donc indifférent en mode asynchrone !!!
TX9		Pour choisir le mode 8 bits ou 9 bits par caractère transmis
	0	Mode 8 bits en transmission
	1	Mode 9 bits en transmission
TXEN		Validation de l'émetteur
	0	Emetteur non validé
	1	Emetteur validé : on peut l'utiliser !!!
SYNC		Choix du mode de fonctionnement l'U.A.R.T.
	0	Mode asynchrone
	1	Mode synchrone
BRGH		Pour la vitesse V de la liaison
	0	Mode low speed
	1	Mode high speed
TRMT		Flag indiquant l'état du registre TSR
	0	Registre TSR plein (en cours d'émission)
	1	Registre TSR vide : transmission du caractère courant finie
TX9D		9 ^{ème} bit du caractère transmis si mode 9 bits choisi (par exemple le bit de parité)
	0	Si 9 ^{ème} bit vaut 0
	1	Si 9 ^{ème} bit vaut 1

Figures VIII.129. et VIII.130. (page 366)

Le bit **SPEN** est dans le registre **RCSTA** vu plus loin.

MODULE UART du PIC (suite)

Récepteur : synoptique

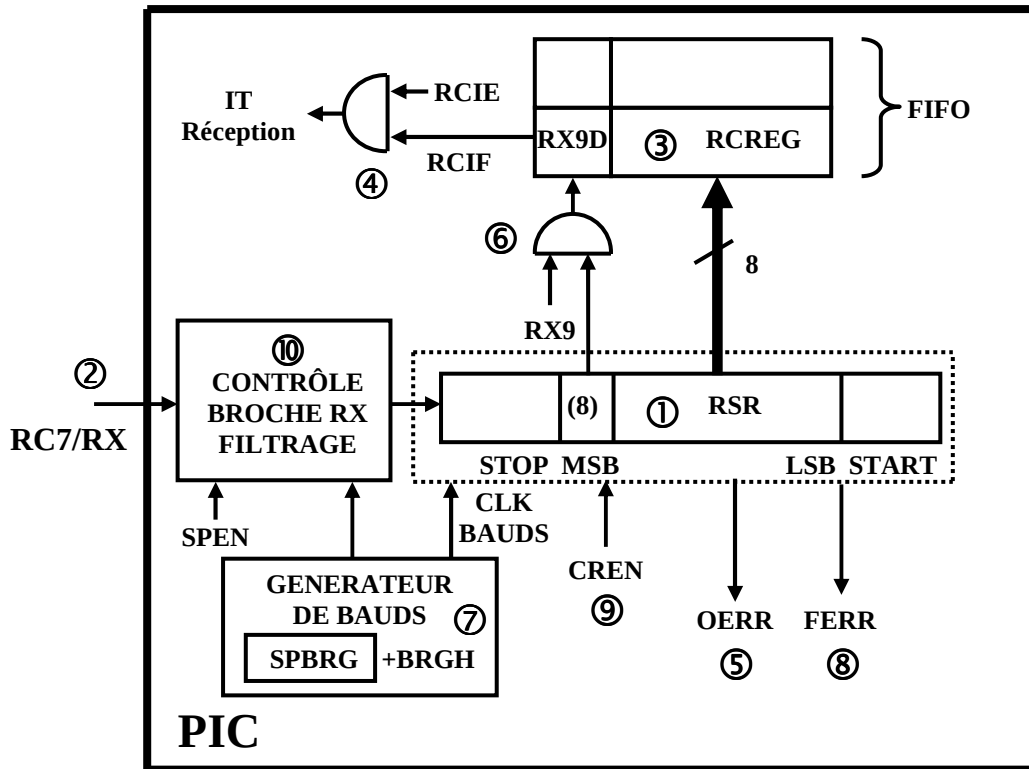


Figure VIII.131. (page 368)

① **le cœur** du récepteur :

registre à décalage **RSR** récupérant les bits arrivant de la broche d'entrée **RC7/RX** (②) et utilisé pour la conversion série $\Rightarrow$ parallèle.

Grâce au bloc ⑥, on peut récupérer aussi un 9^{ème} bit (comme la parité **P** par exemple mais vérifiée ensuite par soft).

MODULE UART du PIC (suite)

Récepteur : synoptique (suite)

Après réception du stop bit, le contenu de **RSR** est **automatiquement** placé dans **RCREG** (cf. bloc ③) structuré en **FIFO** à 2 positions, et s'il reste une place dans cette **FIFO**, le flag **RCIF** (**RCREG** plein) passe alors **automatiquement** à **1** indiquant qu'un caractère complet est arrivé. Ce flag **RCIF** est géré :

- soit par **pooling**, ce qui revient, en régime établi, à attendre que le caractère complet ait été entièrement reçu par le récepteur,
- soit sous **interruption** si le bit **RCIE** a aussi été mis à **1** (cf ④).

Par contre, on ne peut remettre ce flag **TCIF** à **0** par soft : c'est un « **flag hardware** » qui sera remis **automatiquement** à **0** par l'U.A.R.T. (**RCREG** vide) quand **RCREG** aura été lu par le programme.

Si un troisième caractère arrive, alors que la **FIFO** est pleine, le bit d'erreur **OERR** (cf ⑤ : **overrun error** ou **erreur d'écrasement**) passera à **1**: ce troisième caractère sera perdu.

La remise à **0** du bit **OERR** sera faite en remettant **CREN** à **0** puis à **1**. **Mais attention, notez bien :**
si on ne remet pas le bit OERR à 0 lorsqu'il est passé à 1, plus aucune réception n'est possible, car le chargement de RCREG par RSR ne se fait plus !!!

MODULE UART du PIC (suite)

Récepteur : synoptique (suite)

Notons également l'existence du flag **FERR** (cf. ⑧: **framing error** ou **erreur de cadrage**) qui se produit si le **bit de stop** est lu à **0** au lieu de **1**. Cela signifie que la communication est « désynchronisée » et donc qu'il y a un problème :
vitesse ou format de liaison non compatibles entre émetteur et récepteur par exemple.

La vitesse de la liaison est paramétrable par le bloc ⑦ **GENERATEUR DE BAUDS**, qui est le même que celui utilisé par l'émetteur.

Emission et réception se font donc à la même vitesse !!!

Le bloc ⑩ **CONTRÔLE BROCHE RX FILTRAGE**, piloté par le bit **SPEN** déjà vu (qui doit être mis à **1** pour valider globalement la liaison série) sert à effectuer un filtrage suivant la technique du « **vote majoritaire** » : pendant la durée d'un bit, plusieurs valeurs sont prises et au final ce bloc décide si ce bit est :

- à **1** : plus de valeurs à **1** qu'à **0**,

ou

- à **0** : plus de valeurs à **0** qu'à **1**.

Et enfin le bit **CREN** (cf. ⑨) permet, quand on le met à **1**, de valider la liaison en réception.

Résumé du cours PIC : chapitre VIII.7.2. (informations supplémentaires pour des projets)

MODULE UART du PIC (suite)

Récepteur : registre de contrôle associé

RCSTA (situé à l'adresse 0x018)
(page 112 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R-0	R-0	R-x
RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
SPEN		Pour valider la liaison série complète (mode asynchrone ou synchrone)
	0	Dévalidée
	1	Validée : on peut l'utiliser
RX9		Pour choisir le mode 8 bits ou 9 bits par caractère reçu
	0	Mode 8 bits en réception
	1	Mode 9 bits en réception
SREN		Utilisé dans le mode synchrone de l' U.A.R.T. Et donc indifférent en mode asynchrone !!!
CREN		Validation du récepteur
	0	Récepteur non validé
	1	Récepteur validé : on peut l'utiliser !!!
ADDEN		En mode 9 bits, bit de détection d'adresse
	0	Mode détection d'adresses non validé
	1	Validation du mode détection d'adresses
FERR		Flag signalant une erreur de cadrage (Framing ERRor)
	0	Pas d'erreur de cadrage
	1	Erreur de cadrage
OERR		Flag signalant une erreur d'écrasement de caractère (Overrun ERRor)
	0	Pas d'erreur d'écrasement
	1	Erreur d'écrasement
RX9D		9 ^{ème} bit du caractère reçu si mode 9 bits choisi (par exemple bit de parité)
	0	Si 9 ^{ème} bit vaut 0
	1	Si 9 ^{ème} bit vaut 1

Figures VIII.132 et VIII.133. (page 371)

MODULE UART du PIC (suite)

METHODOLOGIE DE PROGRAMMATION

Les différentes étapes (initialisation, émission, réception), permettant de mettre en œuvre l'**UART** sont détaillées dans le polycopié de cours :

- **page 367** pour l'**émetteur**,
- **page 372** pour le **récepteur**.

Jetez y un coup d'œil pour vérifier si vous avez bien compris jusque là ce résumé.

Deux exemples complets d'application :

- en mode **pooling**,
- ou sous **interruption**,

sont également fournis dans les pages **373 à 376 incluses** de ce même polycopié de cours.

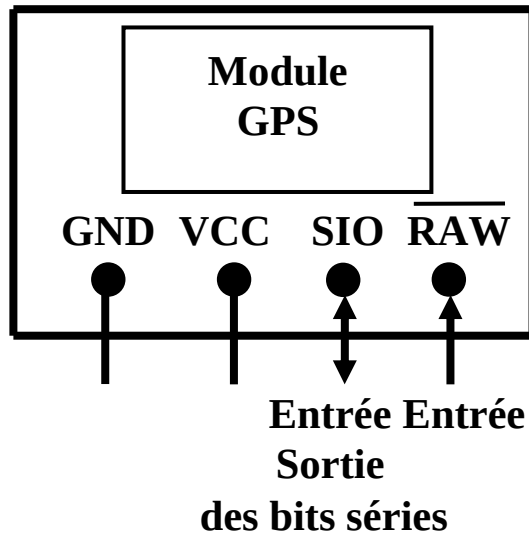
Vous y trouverez notamment des **fonctions génériques** pour l'initialisation, l'émission et/ou la réception de caractères.

N'hésitez pas à vous en inspirer !!!

Remarque : le mode **synchrone** de l'**UART** qui devient alors **USART** (Synchronous) n'est pas présenté dans ce résumé (voir **chapitre VIII.7.3.** du polycopié de cours).

ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite)

BROCHAGE DU RECEPTEUR GPS



RECEPTEUR GPS
(vue de dessus)

DEUX MODES DE FONCTIONNEMENT

↳ si l'entrée **RAW** = 0 : mode dit **RAW**.

Le récepteur envoie en permanence et régulièrement des trames d'octets (au format **NMEA0183 v2.2**) sur sa broche **SIO** qui fonctionne alors uniquement en sortie : ces trames contiennent toutes les informations comme la position, l'heure, la date, le nombre de satellites acquis...

↳ si l'entrée **RAW** = 1 (ou en l'air) : mode dit **SMART**.

Le récepteur répond (sur sa broche **SIO**) aux requêtes reçues sur cette même broche **SIO** (protocole question/réponse).

La broche **SIO** devient alors **bidirectionnelle** : tantôt en **entrée**, tantôt en **sortie**.

ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite) PROTOCOLE DE COMMUNICATION MODE RAW

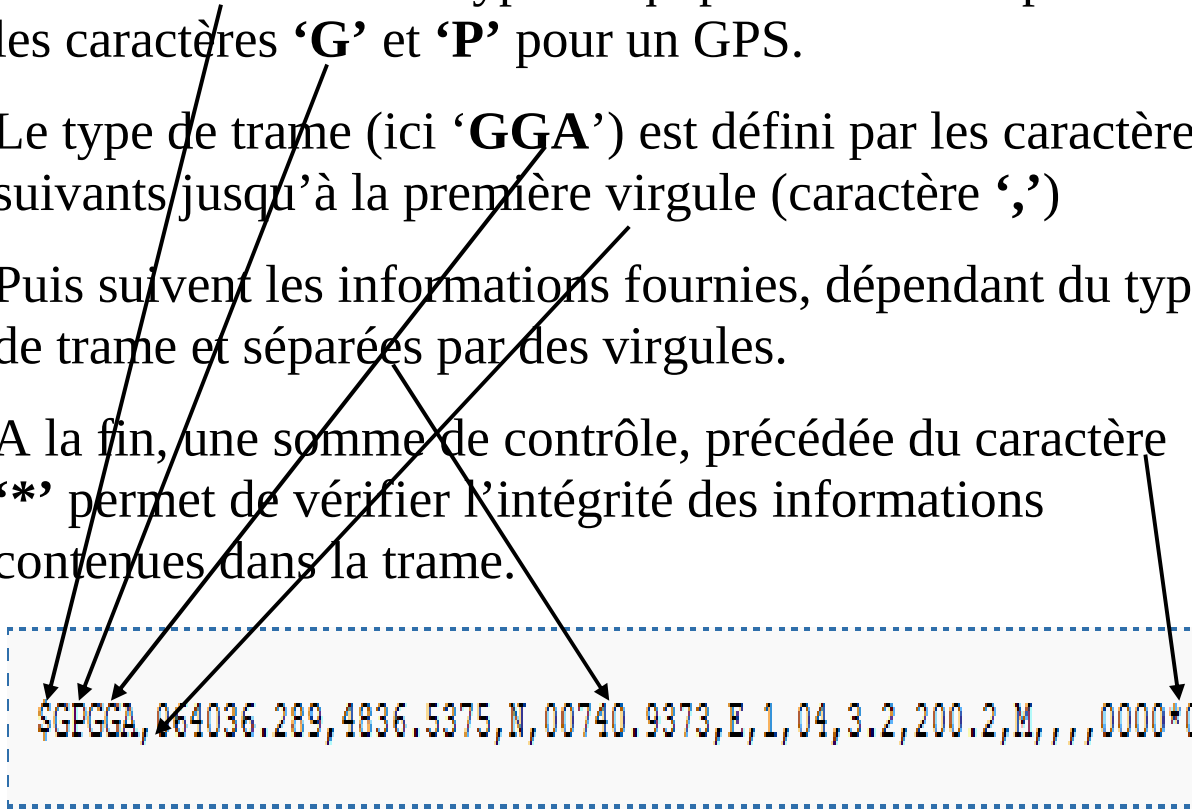
Il existe beaucoup de trames **NMEA** différentes.
(voir fichier **trames NMEA0183.pdf** de votre répertoire documentations/docs GPS).

Elles sont toutes au format **ASCII** et commencent par le caractère '\$' suivi du type d'équipement de réception. Ici les caractères '**G**' et '**P**' pour un GPS.

Le type de trame (ici '**GGA**') est défini par les caractères suivants/jusqu'à la première virgule (caractère ',')

Puis suivent les informations fournies, dépendant du type de trame et séparées par des virgules.

A la fin, une somme de contrôle, précédée du caractère '*' permet de vérifier l'intégrité des informations contenues dans la trame.



\$GPGGA,06.40,36.289,48,36.5375,N,007,40.9373,E,1,04,3.2,200.2,M,,,0000*0E

Ce sera **à la charge** du programme de l'utilisateur d'extraire les informations contenues dans ces trames.

Résumé du cours PIC : ANNEXE 9 (informations supplémentaires pour des projets)
--

ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite)

MODE RAW (suite et fin)

Exemples de trames envoyés par le récepteur :

TRAME GGA :

\$GPGGA	: Type de trame
064036.289	: Trame envoyée à 06h40m36,289s (heure UTC)
4836.5375,N	: Latitude 48,608958° Nord = 48°36'32.25" Nord
00740.9373,E	: Longitude 7,682288° Est = 7°40'56.238" Est
1	: Type de positionnement (le 1 est un positionnement GPS)
04	: Nombre de satellites utilisés pour calculer les coordonnées
3.2	: Précision horizontale ou HDOP (Horizontal dilution of precision)
200.2,M	: Altitude 200,2, en mètres
,,,,,0000	: D'autres informations peuvent être inscrites dans ces champs
*0E	: Somme de contrôle de parité, un simple XOR sur les caractères précédents

TRAME RMC :

\$GPRMC	: type de trame
053740.000	: heure UTC exprimée en hhmmss.sss : 5h37m40s
A	: état A=données valides, V=données invalides
2503.6319	: Latitude exprimée en ddmm.mmmmm : 25°03.6319' = 25°03'37,914"
N	: indicateur de latitude N=nord, S=sud
12136.0099	: Longitude exprimée en dddmm.mmmmm : 121°36.0099' = 121°36'00,594"
E	: indicateur de longitude E=est, W=ouest
2.69	: vitesse sur le fond en nœuds
79.65	: route sur le fond en degrés
100106	: date exprimée en ddmmyy : 10 janvier 2006
,	: deviation magnétique en degrés (souvent vide pour un GPS)
,	: sens de la déviation E=est, W=ouest (souvent vide pour un GPS)
A	: mode de positionnement A=autonome, D=DGPS, E=DR
*53	: somme de contrôle de parité

ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite)

MODE SMART

↪ Phase de requête ou de commande :

On envoie sur la broche **SIO** (qui fonctionne en **entrée**) les quatre caractères ASCII **‘!’ ‘G’ ‘P’** et **‘S’** suivis d'un **numéro** identifiant le **type de commande**. (cf. colonne ①).

①

Cmd	Constant	Description	Returned Bytes	Variables (1 Byte Each Unless Noted)
0x00	GetInfo	GPS Receiver Module version	2	Hardware, Firmware
0x01	GetValid	Check validity of data string	1	0 = Not Valid, 1 = Valid
0x02	GetSats	Number of acquired satellites (12 maximum)	1	
0x03	GetTime	Time (UTC/Greenwich Mean Time)	3	Hours, Minutes, Seconds
0x04	GetDate	Date (UTC/Greenwich Mean Time)	3	Month, Day, Year
0x05	GetLat	Latitude	5	Degrees, Minutes, Fractional Minutes (Word), Direction (0 = N, 1 = S)
0x06	GetLong	Longitude	5	Degrees, Minutes, Fractional Minutes (Word), Direction (0 = E, 1 = W)
0x07	GetAlt	Altitude above mean-sea-level (in tenths of meters, 65535 maximum)	2	Altitude (Word)
0x08	GetSpeed	Speed (in tenths of knots)	2	Speed (Word)
0x09	GetHead	Heading/direction of travel (in tenths of degrees)	2	Heading (Word)

④

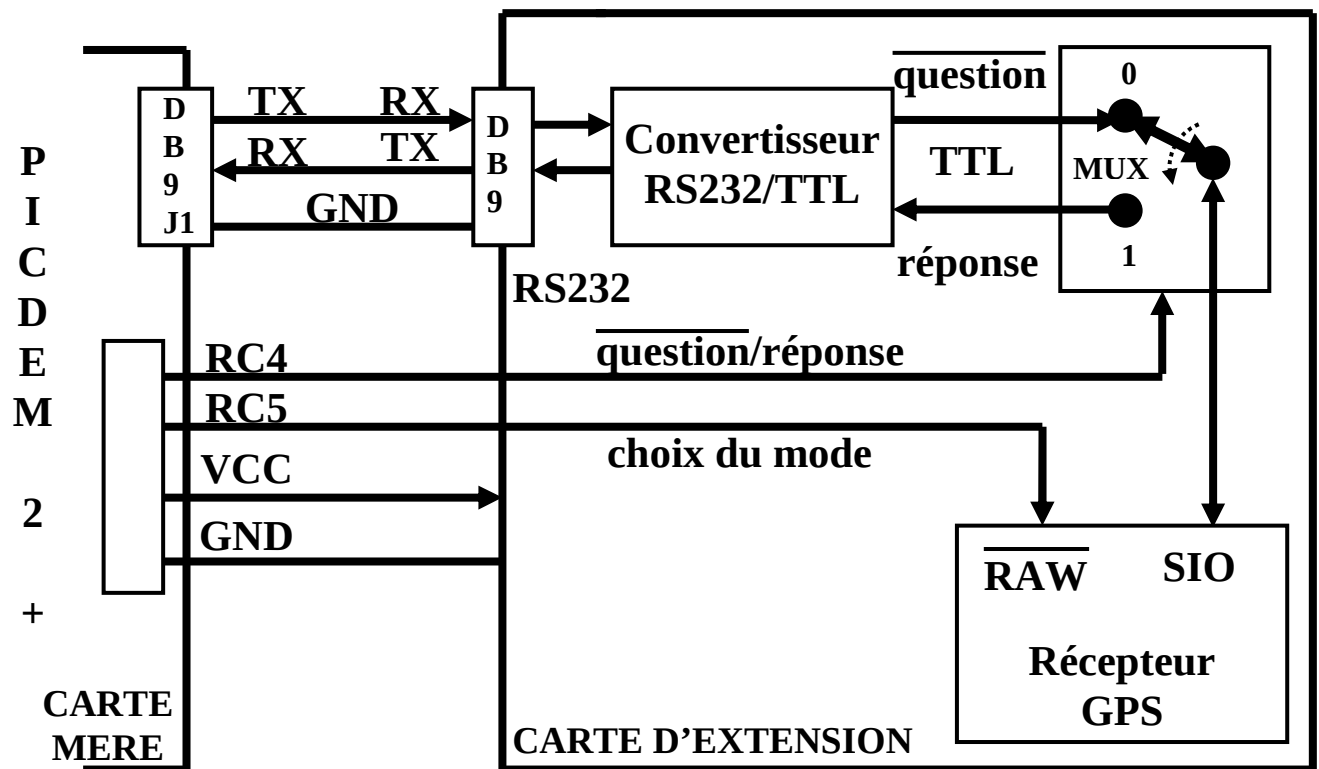
↪ Phase de réponse :

En retour, le récepteur renvoie sur sa broche **SIO** (qui fonctionne alors en **sortie**) les informations demandées et au format précisé dans la colonne ④.

C'est à la charge du programme de l'utilisateur d'initier les questions et de décoder les réponses du module GPS.

ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite)

SYNOPTIQUE POSSIBLE POUR L'IMPLANTATION DU RECEPTEUR GPS



ELEMENTS SUR L'IMPLANTATION D'UN RECEPTEUR GPS (suite)

DERNIERE AIDE

Si de plus, vous avez respecté le synoptique précédent (notamment en utilisant également les broches **RC4** et **RC5** du **PORTC** et leur logique de commande), vous pouvez vous relier à la carte **PICDEM 2 PLUS**.

Vous pouvez alors le tester sous **MPLAB** en faisant :

File Import... et aller chercher **dans votre répertoire** de travail dans le sous répertoire :
fichiers HEX OK/source en C le fichier :

gps mode raw.hex ou **gps mode smart.hex**
(ce sont des exécutables)

Puis choisir l'outil **MPLAB ICD3** par :

Debugger Select Tool, et envoyer cet exécutable sur l'ICD3 par **Debugger Program**.

Et enfin lancer ce programme.

Les informations du **GPS** défileront en appuyant sur **RB0**.
(penser aussi à enlever aussi le cavalier **J6**...)

MODULE MSSP du PIC

Le module **MSSP** (**M**aster **S**ynchronous **S**erial **P**ort) est une autre interface série (fonctionnant en **mode synchrone**) très utilisée pour interfacer des composants et des périphériques aussi divers que :

- des mémoires série,
- des afficheurs,
- des CAN ou des CNA,
- différents types de capteurs,
- des circuits audio et ou vidéo,
- des récepteurs divers,
- des circuits « expanseurs » de bus...
- mais aussi d'autres systèmes à base de micro/processeurs/contrôleurs,

disposant tous de ce type d'interface.

Ce module **MSSP** peut fonctionner selon 2 modes différents :

↪ le mode **SPI** que nous présenterons dans un premier temps,

↪ le mode **I²C** que nous détaillerons ensuite.

MODE SPI du PIC

INTRODUCTION GENERALE

La **SPI** (Serial Peripheral Interface) est une interface de type **série synchrone** permettant la connexion en **Full Duplex** de plusieurs circuits en mode **Maître/Esclave** (**Master/Slave**).

Les données et l'horloge sont sur des fils différents : horloge **non** « **incluse** » (puis « extraite ») dans le flot des bits série, mais bien véhiculée sur un fil **supplémentaire**.

Il y aura donc au total **4 fils** sur une liaison **SPI** :

↳ **2 fils** pour **les bits échangés** : un fil pour chaque sens de la communication notés dans cette intro générale :

- **M2S** ou **Master To Slave** : sens **maître vers esclave**.
- **S2M** ou **Slave To Master** : sens **esclave vers maître**.

↳ **1 fil** pour **l'horloge** noté **SCK** (comme **Serial Clock**).

↳ **1 fil** pour **la masse GND**.

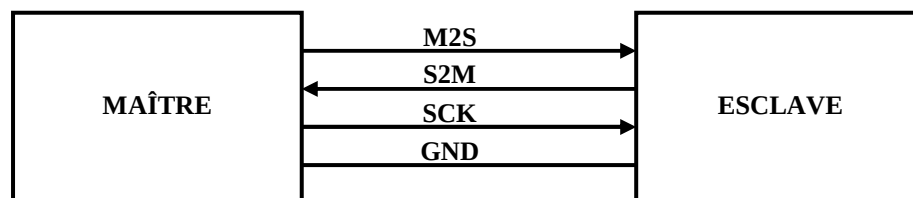


Figure VIII.144. (page 391)

C'est le maître qui s'occupe totalement de la communication, l'(ou les) esclave(s) répondant aux requêtes du maître.

MODE SPI du PIC (suite)

COMMENT ÇA MARCHE ?

Au cœur de la **SPI**, se trouvent **deux registres à décalage**

- à **entrée série SDI** (comme **Serial Data In**),
- à **sortie série SDO** (comme **Serial Data Out**),
- et de taille **8 bits** chacun.

↳ le **MSR** ou **Master Séri**al **R**egister dans le **maître**,

↳ le **SSR** ou **Slave Séri**al **R**egister dans l'**esclave**.

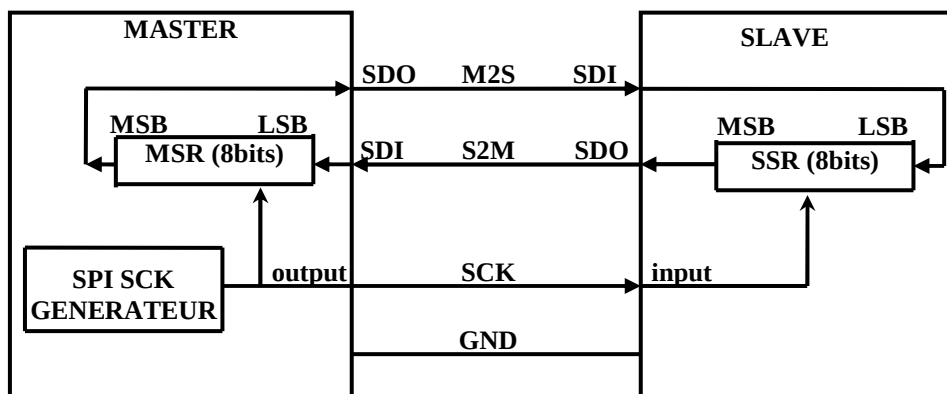


Figure VIII.145. (page 392)

L'horloge **SCK** est une sortie pour le maître et une entrée pour l'esclave car c'est **le maître qui génère l'horloge** par son bloc **SPI SCK GENERATEUR**.

Elle permet :

- la synchronisation (à l'émission),
- et l'échantillonnage (à la réception) des bits série.

MODE SPI du PIC (suite)

COMMENT ÇA MARCHE ? (suite)

La ligne **M2S** relie la broche de sortie **SDO** du maître et la broche d'entrée **SDI** de l'esclave.

Et enfin, la ligne **S2M** relie la broche de sortie **SDO** de l'esclave et la broche d'entrée **SDI** du maître.

Ainsi, à chaque coup d'horloge, le maître et l'esclave s'échangent un bit (le **MSB** en premier).

Après huit coups d'horloge le maître a transmis un octet complet à l'esclave et vice-versa.

Ainsi, l'ensemble des 2 registres à décalage 8 bits (**MSR** côté maître et **SSR** côté esclave) forment **l'équivalent d'un registre à décalage 16 bits unique et commun**, partagé entre le maître et l'esclave :

l'émission par le maître d'un octet contenu dans son **MSR** vers l'esclave provoque en même temps la réception par le maître de l'octet contenu le **SSR** de l'esclave (le registre à décalage du maître « se vide » dans celui de l'esclave pendant que celui de l'esclave « remplit » celui du maître).

Mais c'est toujours le maître qui décide des transferts :
il doit émettre un octet,
pour en recevoir un autre.

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC

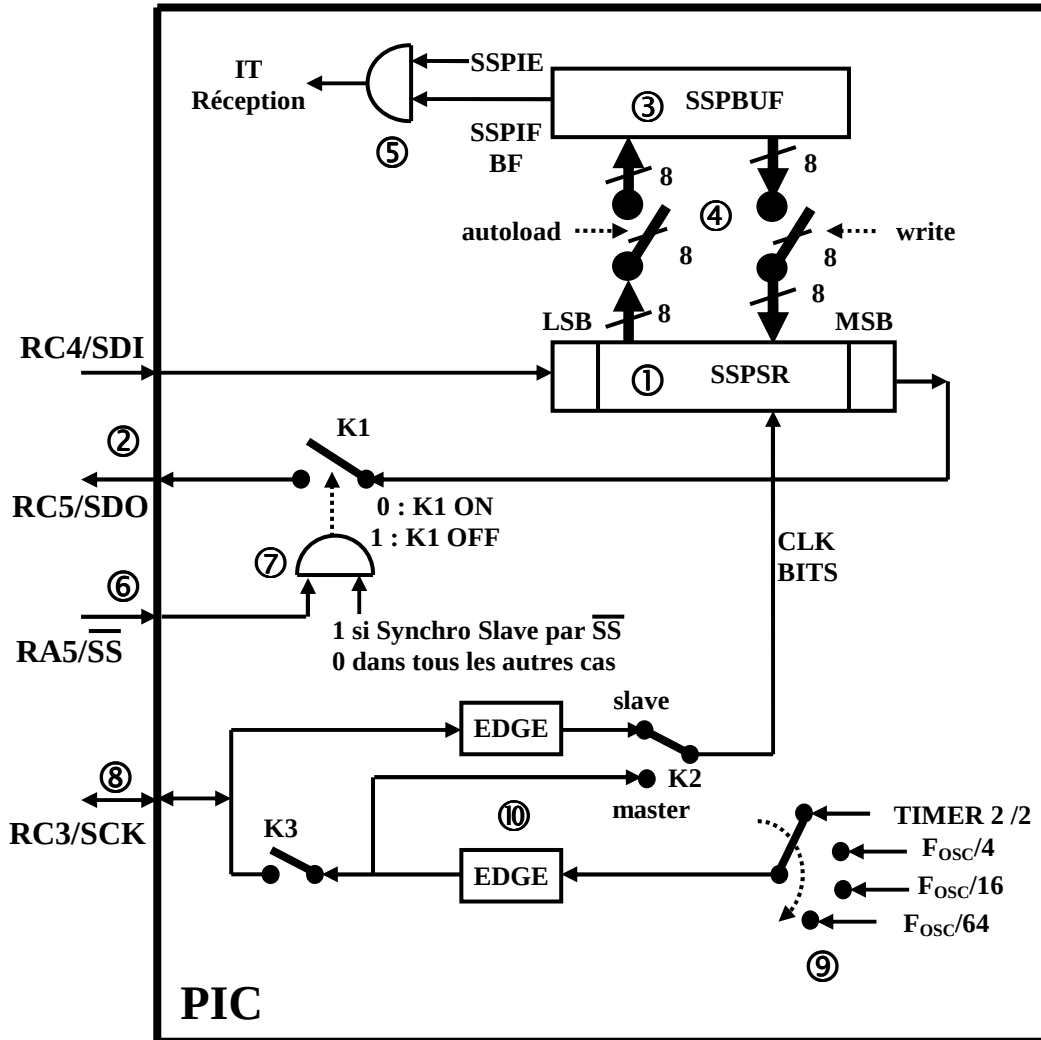


Figure VIII.146. (page 393)

Le cœur de la **SPI** est le registre à décalage **SSPSR** (cf. ①) :

il contient l'octet à envoyer au début de l'échange et contiendra l'octet reçu à la fin de l'échange.
(après 8 coups d'horloge).

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC (suite)

Les bits sortent un à un (**MSB** en premier) à chaque coup d'horloge par la sortie **SDO** (broche **RC5/SDO**) et rentrent également un par un, sur ce même coup d'horloge, par l'entrée **SDI** (broche **RC4/SDI**) comme indiqué en ②.

Comme le registre **SSPSR** n'est pas mappé (pas accessible directement), il faut utiliser le registre tampon **SSPBUF** ③ pour échanger les données avec la **SPI**.

Quand le maître écrit un octet dans **SSPBUF** (cf. **write** en ④), cet octet est **automatiquement** chargé dans **SSPSR**.

Et le transfert peut commencer...

↪ une salve de 8 coups d'horloge **SCK** est générée par le maître et les bits émis sortent par **SDO** et ceux reçus en même temps arrivent en provenance de l'esclave sur **SDI** à chaque coup d'horloge.

Lorsque les 8 bits de l'octet reçu sont arrivés dans **SSPR**, cet octet est **automatiquement** transféré (cf. **autoload** en ④) dans **SSPBF** et le bits **BF** et **SSPIF** passent à 1.

Fin de l'échange d'un octet.

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC (suite)

Comme d'habitude, on peut travailler :

↳ soit sous **interruption** : si le bit **SSPIE** est à **1** (cf. ⑤).
L'acquittement de cette interruption de réception se fera en remettant **SSPIF** à **0** par soft : **SSPIF = 0**.

↳ soit par **pooling** sur le bit **BF** : il faut attendre que ce bit passe à **1** (attendre que l'octet complet soit arrivé), puis lire **SSPBUF** (qui contient l'octet reçu, ce qui remettra **automatiquement** **BF** à **0**) puis écrire dans ce même registre **SSPBUF** le prochain caractère à envoyer.

Erreur de collision.

Remarquons également que si on écrit dans **SSPBF** alors que le transfert est en cours (octet précédent pas encore reçu), il y aura une erreur signalée par la mise à **1** du bit **WCOL** (signalant une « **Write COLlision** »).

Erreur d'écrasement.

De même, en mode esclave, si un nouvel octet arrive alors que le précédent n'a pas encore été lu (toujours présent dans **SSPBUF**), le bit **SSPOV** passera à **1** (signalant un **OVerflow**).

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC (suite)

La broche **RA5/ $\overline{SS}$** (cf. ⑥) permet, en mode multi-esclaves, de sélectionner quel **unique** esclave sera concerné par l'échange qui va suivre :

↪ seul l'esclave ayant sa broche d'entrée **RA5/ $\overline{SS}$** au **niveau bas répondra au maître, les autres seront muets.**

Il faudra bien sûr dans ce cas utiliser par exemple un port parallèle du maître pour sélectionner l'esclave (et lui seul) qui doit répondre à cette requête du maître.
(voir plus loin les registres de contrôle du mode **SPI** et le mode sélection par **$\overline{SS}$**)

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC (suite)

L'horloge **SCK** est, rappelons le :

- **générée** par le maître (interrupteur **K2** en position **master**):

l'interrupteur **K3** est alors fermé pendant les phases de transfert, **SCK** sera présente sur la broche **RC3/SCK** (cf. ⑧).

Grâce au multiplexeur ⑨ à 4 entrées et 1 sortie, le signal de **SCK** peut provenir :

- ↳ soit de la sortie du **TIMER 2**
après avoir divisé sa fréquence par 2,
- ↳ soit directement du quartz (F_{osc})
dont la fréquence a été divisée par 4, 16 ou 64.

Ainsi, avec un quartz à **4 MHz**, on peut faire fonctionner la liaison SPI à une vitesse de **1 Méga bits par seconde**.

- **reçue** par l'esclave (interrupteur **K2** en position **slave**) :

l' interrupteur **K3** est ouvert en permanence, **SCK** arrive de l'extérieur sur la broche **RC3/SCK** (cf. ⑧).

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC (suite)

Les deux blocs **EDGE** référencés ⑩ sont utiles pour la **synchronisation** des échanges de bits entre le maître et l'esclave.

Ils servent à fixer les instants d'horloge **SCK** où les bits seront :

- soit **sortis** et **stables** sur la broche **SDO** pour l'émission,
- soit **stables** et **échantillonnés** sur la broche **SDI** pour la réception.

En pratique, il existe trois bits (voir plus loin les registres de contrôle du mode **SPI**) qui permettent de choisir différentes combinaisons pour fixer ces instants :

↪ le bit **CKP** : choix du niveau de **repos** de l'horloge **SCK**,

↪ le bit **CKE** : choix du front **actif** de l'horloge **SCK**,

↪ et le bit **SMP** : choix des instants d'échantillonnage sur l'entrée **SDI** du maître.

Nous n'allons pas ici décrire toutes les possibilités offertes par les différentes combinaisons de ces trois bits (c'est fait dans le polycopié complet de cours).

MODE SPI du PIC (suite)

SYNOPTIQUE DANS LE PIC (suite et fin)

Mais pour comprendre la « mécanique » d'un échange complet entre maître et esclave, voici un chronogramme correspondant à un cas fréquemment utilisé où :

- **CKP = 1** : horloge **SCK** au repos au niveau **haut**,
- **CKE = 0** : sortie des bits par le maître et l'esclave sur le premier front **descendant** de **SCK**,
- **SMP = 0** : lecture par le maître et l'esclave **au milieu** du bit.

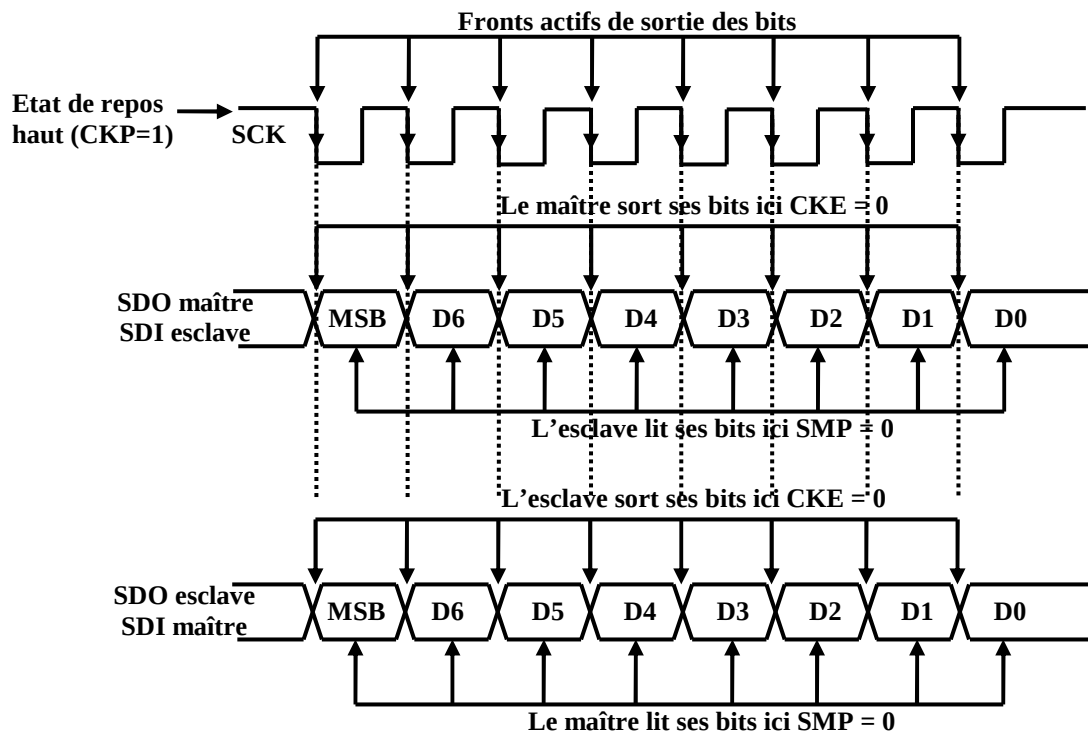


Figure VIII.148. (page 398)

MODE SPI du PIC (suite)

CONFIGURATION DES BROCHES SPI

Voici enfin comment configurer les broches concernées par la liaison **SPI** :

- **SDI/RC4** est **automatiquement** géré par le module **SPI** (rien à faire),
- **SDO/RC5** est une **sortie** : **TRISC5 = 0**,
- **SCK/RC3** du **maître** est une **sortie** : **TRISC3 = 0**,
- **SCK/RC3** de l'**esclave** est une **entrée** : **TRISC3 = 1**,
- **$\overline{SS}$ /RA5** est une **entrée** : **TRISA5 = 1**,
(penser aussi à initialiser les bits **PCFG3 :PCFG0** du registre **ADCON1** pour déclarer la broche **RA5** comme E/S logique)

L'inconvénient de la liaison **SPI**, c'est qu'il n'y a pas d'acquittement de la part de l'esclave dans le cas d'un échange d'octet : le maître peut parler « dans le vide » sans le savoir. D'où le mode **I²C** qu'on présentera plus loin.

L'avantage de la liaison **SPI** par rapport à l'**U.S.R.T.** est d'avoir des vitesses de liaison plus importantes.

MODE SPI du PIC (suite)

REGISTRES DE CONTRÔLE ASSOCIES

En premier bien sûr, le registre **SSPBUF** qui contient l'octet à émettre (au début de l'échange) et celui reçu (à la fin de l'échange) :

- **en mode maître :**

une écriture dans **SSPBUF** lance le transfert.

Lorsque le transfert est terminé, ce registre contient la donnée renvoyée par l'esclave, que le maître pourra lire.

- **en mode esclave :**

la donnée **préalablement** placée dans **SSPBUF** sera renvoyée vers le maître quand ce dernier exécutera un transfert.

Lorsque le transfert sera terminé, ce registre contiendra la donnée envoyée par le maître, que l'esclave pourra alors lire.

Résumé du cours PIC : chapitre VIII.8.1. (informations supplémentaires pour des projets)

MODE SPI du PIC (suite)

REGISTRES DE CONTRÔLE ASSOCIES (suite)

① le registre SFR SSPSTAT :

SSPSTAT (situé à l'adresse 0x094)
(page 72 du datasheet)

	R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SSPSTAT	SMP	CKE	« I ² C »	« I ² C »	« I ² C »	« I ² C »	« I ² C »	BF
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.149. (page 399)

Les bits repérés « I²C » ne servent pas dans le mode **SPI** du **MSSP** (mais en mode **I²C**).

Bit	Valeur	Signification et commentaires
SMP		SaMPlE Bit : pour échantillonnage bits sur entrée SDI
		Mode maître
	0	Echantillonnage au milieu du bit sorti sur SDO
	1	Echantillonnage à la fin du bit sorti sur SDO
		Mode esclave : toujours 0
CKE		ClocK Edge : front actif de sortie des bits sur SDO
	0	Front repos vers actif de SCK
	1	Front actif vers repos de SCK
Remarque : si on choisit CKE = 1 , il faut obligatoirement choisir le mode synchronisation par SS au niveau de l'esclave pour assurer un premier front permettant de positionner le MSB de l'esclave sur sa sortie SDO .		
BF		Buffer Full
	0	Octet reçu non complètement arrivé (en cours de réception) : SSPBUFF est vide
	1	Octet reçu complètement arrivé : SSPBUFF est plein (remis à 0 par lecture de SSPBUF)

Figure VIII.150. (page 399)

Résumé du cours PIC : chapitre VIII.8.1. (informations supplémentaires pour des projets)

MODE SPI du PIC (suite)

② le registre SFR SSPCON :

SSPCON (situé à l'adresse 0x014)
(page 73 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
WCOL		Write COLLision detect bit
	0	Pas de collision
	1	Ecriture dans SSPBUF pendant un transfert en cours (remis à 0 par soft par WCOL = 0)
SSPOV		OVerflow detect bit : écrasement d'octet à la réception par l'esclave
	0	Pas d'overflow
	1	Un octet est reçu par l'esclave alors que le précédent n'a pas été lu dans SSPBUF (remis à 0 par soft par SSPOV = 0)
Remarque : en principe SSPOV ne passe jamais à 1 en mode maître car la transmission commence dès qu'on écrit (après l'avoir lu) dans SSPBUF		
SSPEN		Validation de la liaison SPI
	0	SPI non utilisée : les lignes concernées sont des I/O classiques
	1	SPI validée : lignes concernées (SDI , SDO , SCK et SS) affectées à la SPI
Remarque : la direction des lignes (in ou out) concernées doit être faite avant de mettre SSPEN à 1		
CKP		ClocK Polarité : fixe l'état de repos de SCK
	0	Repos à l'état bas
	1	Repos à l'état haut
SSPM3:SSPM0		Différents modes de fonctionnement de la liaison SPI
	0000	Mode maître : SCK générée sur RC3 à la fréquence de F_{OSC}/4
	0001	Mode maître : SCK générée sur RC3 à la fréquence de F_{OSC}/16
	0010	Mode maître : SCK générée sur RC3 à la fréquence de F_{OSC}/64
	0011	Mode maître : SCK générée sur RC3 à la fréquence de TIMER 2 /2
	0100	Mode esclave , SCK extérieure sur RC3 , synchro par SS validé
	0101	Mode esclave , SCK extérieure sur RC3 , synchro par SS non validé

Figures VIII.151. et VIII.152. (page 400)

MODE SPI du PIC (suite)

METHODOLOGIE DE PROGRAMMATION

Les différentes étapes (initialisation, émission, réception), permettant de mettre en œuvre la liaison **SPI** sont détaillées dans le polycopié complet de cours :

- **page 401** pour le **maître**,
- **page 402** pour l'**esclave**.

Jetez y un coup d'œil pour vérifier si vous avez bien compris jusque là ce résumé.

Un exemple complet d'application est également fourni entre les pages **404** et **407 incluses** de ce même polycopié de cours.

Vous y trouverez notamment des **fonctions génériques** pour l'initialisation et l'émission de caractères.

N'hésitez pas à vous en inspirer !!!

MODE I²C du PIC

INTRODUCTION GENERALE

Développé dans les années 1980 par la société **Philips**, le bus **I²C** (**I**nter **I**ntegrated **C**ircuit bus) a été conçu à l'origine pour minimiser la connectique et le nombre de fils de liaison entre les divers composants situés à l'intérieur des appareils vidéo :

- téléviseurs,
- magnétoscopes,
- lecteurs CD....

C'est une interface de type **série synchrone** permettant la connexion en **Full Duplex à l'alternat** de plusieurs circuits en mode **Maître/Esclave**.

Ce qui veut dire :

tantôt dans un sens,
tantôt dans l'autre,
mais **jamais** dans les deux sens en même temps.

Comme pour la liaison SPI, **les données et l'horloge sont sur des fils différents :**

horloge **non** « **incluse** » (puis « extraite ») dans le flot des bits série, mais bien véhiculée sur un fil **supplémentaire**.

MODE I²C du PIC

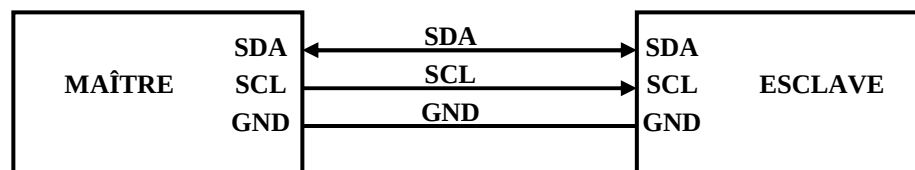
INTRODUCTION GENERALE (suite)

Il y aura donc au total **3 fils** sur un bus I²C :

↳ **1 fil** pour **les bits échangés** appelé **SDA** (comme **Signal DAta**) entre les broches **SDA**,

↳ **1 fil** pour **l'horloge** noté **SCL** (comme **Serial CLock**) entre les broches **SCL**,

↳ **1 fil** pour **la masse GND**.



Cas simple d'un maître et d'un seul esclave.

Figure VIII.157. (page 408)

Se pose alors le problème purement électronique suivant :

Peut-on « impunément » et donc sans risque, relier entre elles les broches SDA (broches d'entrée/sortie) et qui peuvent donc être potentiellement des sorties au même moment ?

A cause des étages de sorties de type « totem pôle » la réponse est : **NON !!!**

Alors, comment faire ?

MODE I²C du PIC (suite)

HARDWARE

La solution retenue pour relier les broches **SDA** ensemble consiste à utiliser un étage de sortie type « collecteur ouvert » (cf. figure VIII.160.) qui montre à quoi est reliée cette broche **SDA** à l'intérieur du circuit **I²C**.

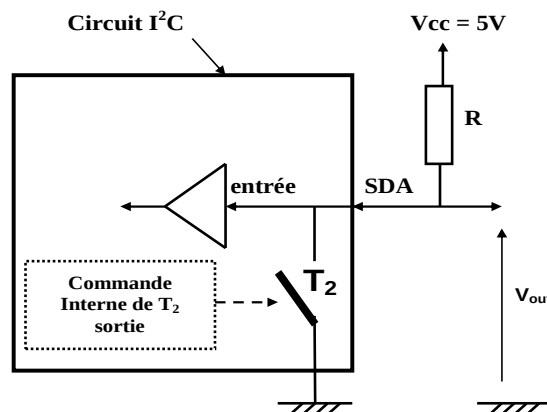


Figure VIII.160. (page 410)

R est une résistance de « pull up » à rajouter extérieurement pour fixer le potentiel sur **SDA** quand T₂ est ouvert.

Fonctionnement en sortie :

↳ si T₂ est **fermé** : V_{out} = 0 V donc **SDA = 0**,

↳ si T₂ est **ouvert** : V_{out} = 5 V (grâce à R) donc **SDA = 1**.

Fonctionnement en entrée : T₂ est alors **ouvert**.

La valeur sur **SDA** dépend de la tension qu'on lui applique :

soit **0 V**, soit **5 V**.

Le risque de court-circuit de V_{cc} a disparu !!!

MODE I²C du PIC (suite)

Ainsi, en reliant plusieurs circuits, il n'y a plus de soucis.

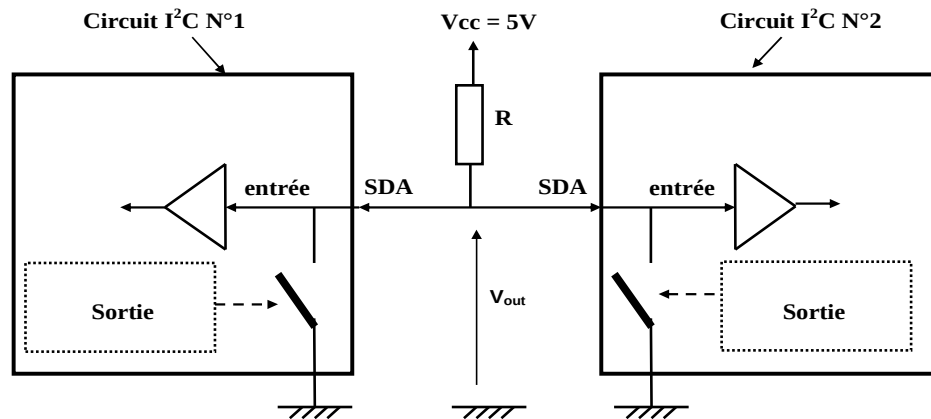


Figure VIII.161. (page 411)

Ce problème est aussi le même pour la ligne **SCL** qui est également (potentiellement) une broche d'entrée/sortie.

Au final, un bus **I²C** peut comporter un nombre plus important de circuits :

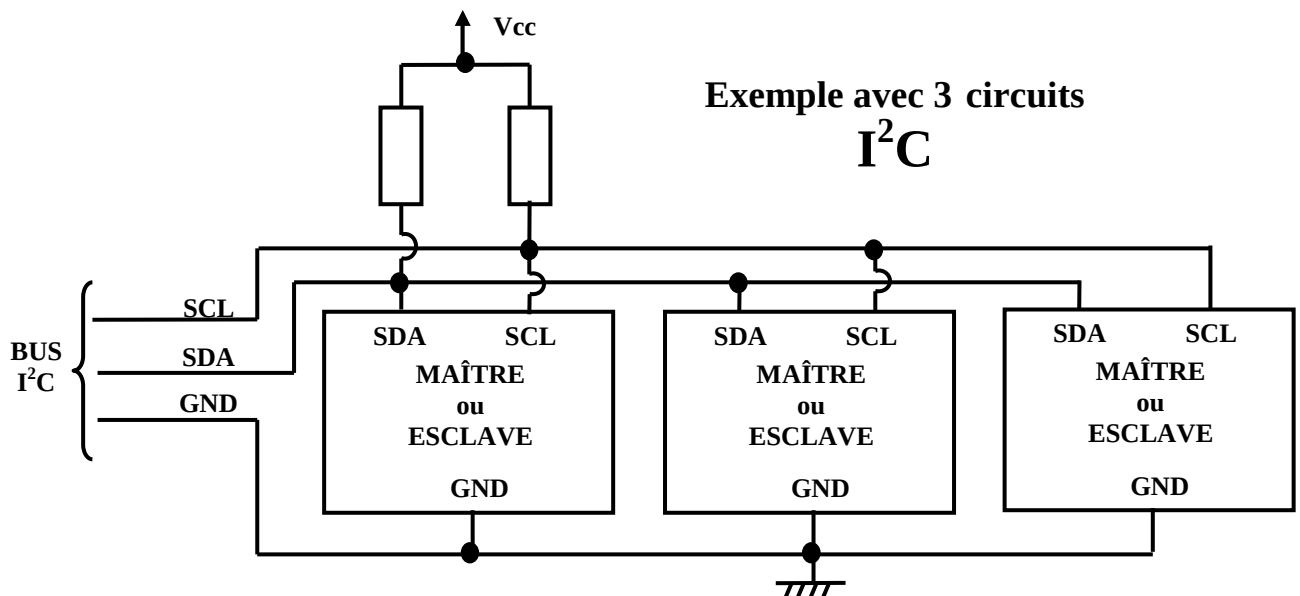


Figure VIII.162. (page 411)

MODE I²C du PIC (suite)

HARDWARE

Remarquons, sur la figure précédente, que chaque composant peut être soit **maître**, soit **esclave**.

En pratique et dans le cas général, on trouvera :

- ↳ des composants qui ne pourront être qu'**esclave**,
- ↳ d'autres qui ne seront que **maître**,
- ↳ et enfin, comme sur la figure, certains qui pourront être tantôt **maître**, tantôt **esclave**.

Mais à chaque instant, il n'y aura qu'un **seul maître** : ce sera lui qui génèrera alors l'horloge **SCL** et qui initiera les séquences de communication.

Les autres seront **esclaves** et recevront l'horloge du maître.

De plus, chaque esclave aura sa propre adresse (ou sa propre plage d'adresses), et seul l'esclave ayant reconnu son adresse (ou sa plage d'adresses) dans la requête du maître pourra recevoir des données du maître ou envoyer des données vers le maître, les autres (non adressés) resteront sourds ou muets.

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES

PRISE DE CONTRÔLE DU BUS

Au repos (pas d'échange de bits sur **SDA**), les lignes **SDA** et **SCL** sont au niveau **haut**.

⇒ on dit alors que le bus est **disponible**.

Le circuit qui désire prendre le contrôle du bus I²C, **et qui en deviendra de fait le maître**, fait passer sa sortie **SDA** à **0** (alors que **SCL** est au niveau **haut**) puis le confirme en faisant passer **ensuite** sa sortie **SCL** aussi à **0**.

Cette séquence de prise de contrôle du bus par un maître, résumée sur la figure VIII. 163. est appelée une :

START CONDITION

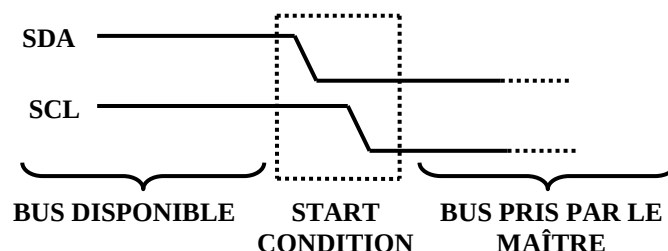


Figure VIII.163. (page 412)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

TRANSMISSION D'UN BIT PUIS D'UN OCTET

Il faut placer la valeur de chacun des bits sur la ligne **SDA** (quand **SCL** est au niveau bas) et d'envoyer une impulsion d'horloge de validation sur la ligne **SCL**, comme indiqué sur la figure VIII.164.a.

Cette impulsion, reçue par le (ou les) esclaves, lui (leur) signalera qu'un bit valide est disponible sur leur entrée **SDA** et qu'il(s) peut(vent) le lire.

Cette séquence sera répétée 8 fois (par 8 impulsions d'horloge) pour transmettre un octet, comme sur la figure VIII.164.b., le **MSB D7** étant le premier bit de l'octet.

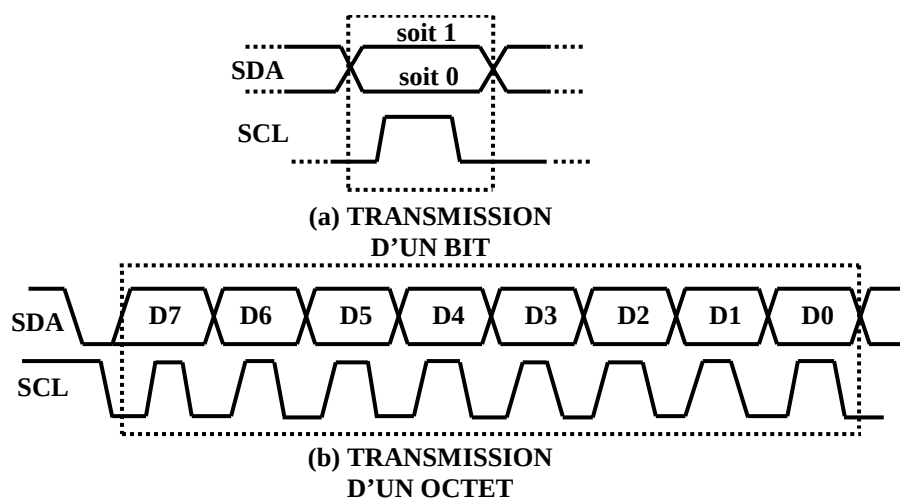


Figure VIII.164. (page 413)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

ACQUITTEMENT/NON ACQUITTEMENT

Une fois les 8 bits transmis par le maître, rien n'assure que ces bits ont bien été reçus par l'esclave.

↳ un **système d'acquittement** ou **accusé de réception** a été rajouté pour le bus I²C (qui n'existait pas sur le bus SPI) après la transmission du 8^{ème} bit de l'octet émis :

- le maître fait passer sa broche **SDA** à **1** :
cette broche devient alors une **entrée** pour le maître,
- si tout c'est bien passé, l'esclave qui a reçu l'octet fait passer sa broche **SDA** à **0** :
cette broche devient une sortie pour l'esclave.
Le bit ainsi renvoyé par l'esclave, au **niveau bas**, s'appelle alors **ACK** comme **ACKnowledge**.
- sinon la ligne **SDA** restera au **niveau haut**.
Le bit (ainsi renvoyé par l'esclave ou par aucun esclave), au **niveau haut** s'appelle alors **NOACK** comme **NOACKnowledge**.

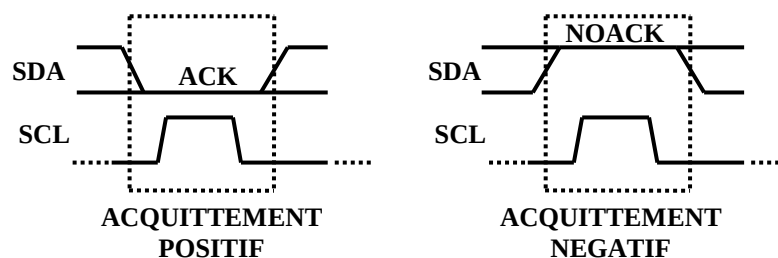


Figure VIII.165. (page 413)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

ACQUITTEMENT (suite)

Grâce à l'impulsion **SCL** suivante (et donc une 9^{ème} générée par le maître), le maître peut lire ce bit d'acquiescement et savoir ainsi si l'octet qu'il vient juste d'envoyer avant est acquiescé ou pas.

Enfin, et dans le cas général, lors de tout transfert d'octet entre l'émetteur (qui n'est pas forcément le maître mais peut être l'esclave) et le récepteur (qui n'est pas forcément l'esclave mais peut être le maître), l'acquiescement n'est rien d'autre qu'un 9^{ème} bit, rajouté à la suite de cet octet, et qui transite toujours dans le sens récepteur de l'octet vers émetteur de cet octet.

Il faudra donc 9 coups d'horloge **SCL** pour transmettre un octet (cf. figure VIII.166.)

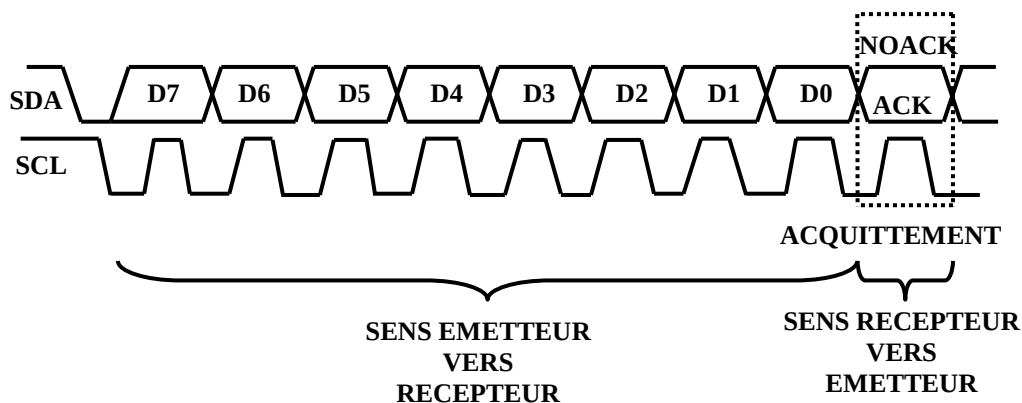


Figure VIII.166. (page 414)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

TRANSMISSION D'UNE ADRESSE

Pour pouvoir différencier les composants sur un bus I²C, chacun doit avoir une adresse **unique**.

Ainsi, lorsqu'un circuit prend le bus (après l'émission d'une **START CONDITION**) et en devient donc de fait le maître, le premier octet qui est envoyé par ce maître est un **octet dit d'adresses** :

le maître envoie sur sa sortie **SDA** l'adresse de l'esclave avec lequel il veut initier une séquence d'échange.

Historiquement, cette adresse a été codée sur **7 bits**, notés **A0** à **A6**, ce qui donne 2⁷ combinaisons différentes.

Pourquoi 7 bits ?

Car le maître utilise le 8^{ème} bit (de l'octet d'adresse) comme bit particulier, appelé bit **R/W**.

- si ce bit **R/W** = **0** (**Write**) cela signifie que l'esclave sélectionné est en **mode écriture** : il doit s'attendre à **recevoir** du maître, juste après, un ou plusieurs octets,
- si ce bit **R/W** = **1** (**Read**) cela signifie que l'esclave sélectionné est en **mode lecture** : il doit **renvoyer** vers le maître, juste après, un ou plusieurs octets.

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

TRANSMISSION D'UNE ADRESSE (suite)

L'octet d'adresse se termine aussi par un acquittement positif (**ACK**) ou négatif (**NOACK**) émanant de l'esclave sélectionné.

⇒ si le maître reçoit **ACK**, l'échange continue soit dans un sens, soit dans l'autre en fonction du bit **R/W**.

⇒ si le maître reçoit **NOACK**, et donc s'il y a eu une erreur, le maître peut générer une condition d'arrêt (voir plus loin **STOP CONDITION**) pour libérer le bus ou renvoyer l'adresse de l'esclave une ou plusieurs fois de suite suivant une stratégie qui sera déterminée par le programme pilotant le maître.

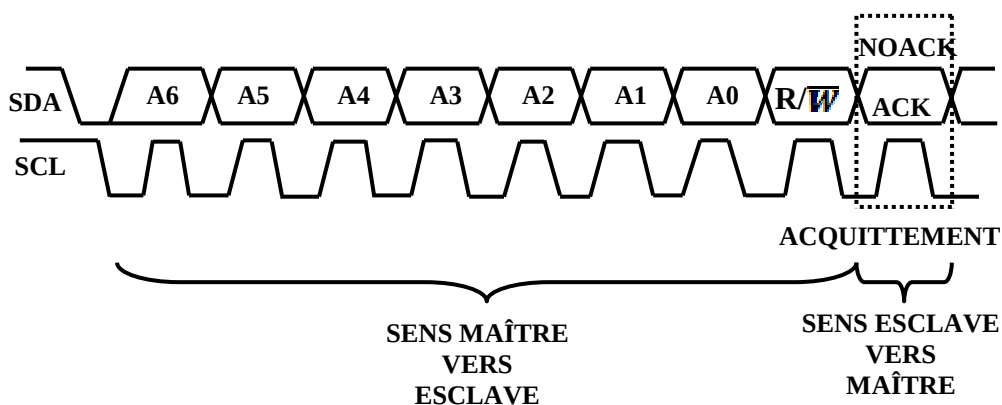


Figure VIII.168. (page 416)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

ECRITURE D'UNE DONNEE

Si le bit **R/W** de l'octet d'adresse vaut **0** et si l'esclave renvoie **ACK**, le maître fait suivre, juste après, le premier octet de donnée à transmettre à l'esclave sélectionné.

S'il y a plus qu'un octet de données à transmettre, ils le seront les uns à la suite des autres.

Chaque octet reçu par l'esclave doit être validé par l'esclave par un **ACK**.

Le maître peut mettre, à tout moment, fin à l'émission :

- ↳ soit à la réception d'un **NOACK (erreur)**,
- ↳ soit à la fin de l'émission de tous les octets à envoyer à l'esclave (**fin normale**),
- ↳ ou même après la réception d'un **ACK** quelconque en générant une **STOP CONDITION** (voir plus loin).

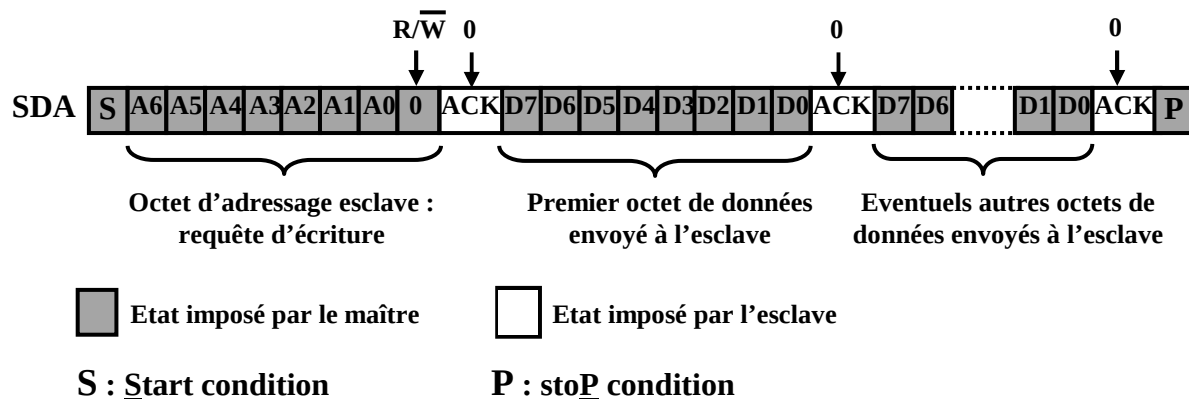


Figure VIII.169. (page 416)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

LECTURE D'UNE DONNEE

Si le bit **R/W** de l'octet d'adresse vaut **1** et si l'esclave renvoie **ACK**, il y a **inversion du sens de l'échange** puisque c'est l'esclave qui va renvoyer un (ou plusieurs octets) en réponse à la demande de lecture du maître.

SDA devient entrée pour le maître et sortie pour l'esclave.
Mais SCL est toujours générée par le maître !!!

Si le premier octet reçu par le maître est validé par le maître par un **ACK**, cela signifie que le maître attend un autre octet (l'esclave sait interpréter ce **ACK** et donc qu'il doit émettre un octet supplémentaire) et ainsi de suite....

Par contre, si à la réception d'un octet par le maître (y compris pour le premier octet) le maître renvoie un **NOACK**, cela signifie que la lecture est terminée. L'esclave saura interpréter ce **NOACK** comme une fin d'émission de données de sa part.

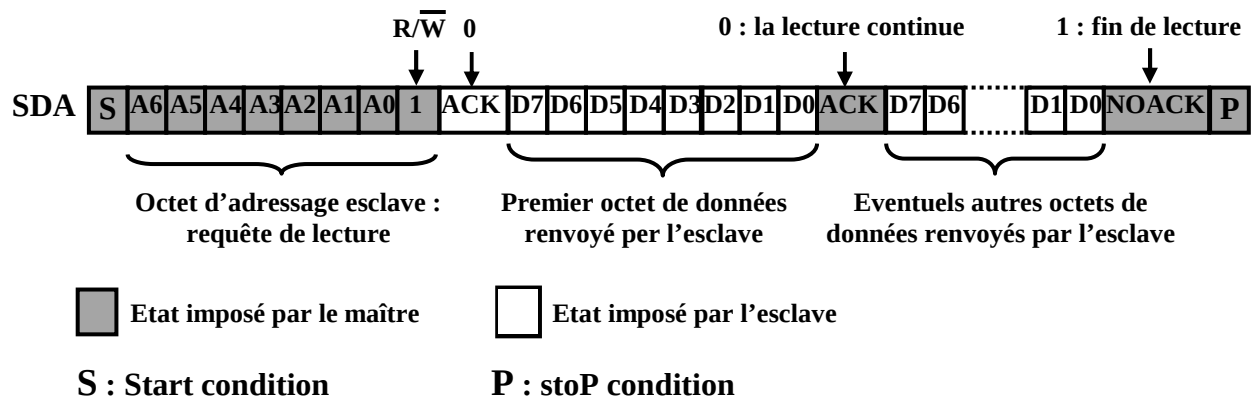


Figure VIII.170. (page 417)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

LIBERATION DU BUS

De façon symétrique à la **START CONDITION**, le maître, à la fin d'une séquence d'échange, génère une condition d'arrêt permettant **de libérer** le bus.

Pour cela, à partir de l'état où les lignes **SDA** et **SCL** sont au niveau bas, le maître commence par faire passer **SCL** au niveau **haut** puis place ensuite **SDA** au niveau **haut**.

Cette séquence de libération du bus par un maître est appelée une :

STOP CONDITION

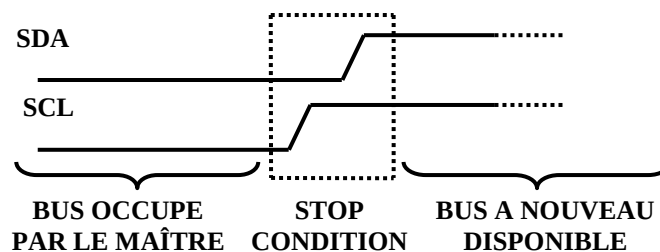


Figure VIII.171. (page 418)

A partir de cet instant, le bus **I²C** est à nouveau disponible pour une nouvelle prise de contrôle par un autre maître (ou par le même !!!)

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

REPEATED START CONDITION

En pratique, et pour certains composants comme les **RAM I²C** par exemple, on peut enchaîner écriture et lecture de l'esclave sans passer par une condition d'arrêt.

Le maître envoie, avec le bit **R/ $\overline{W}$** = 0 (cycle d'écriture), l'adresse de base de la **RAM** esclave concernée qui lui renvoie **ACK**. Puis il transmet l'adresse de la case mémoire visée dans cette **RAM**, qui lui renvoie **ACK**. Et enfin, il devrait envoyer la valeur de l'octet à écrire puis terminer cette écriture par une **STOP CONDITION**.

Mais le maître peut enchaîner une nouvelle **START CONDITION**, sans passer par l'envoi de cet octet et de cette **STOP CONDITION** (on parle de **RESTART** ou de **REPEATED START CONDITION**) et envoyer à nouveau, mais cette fois-ci avec le bit **R/ $\overline{W}$** = 1 (cycle de lecture) l'adresse de base de cette **RAM**, qui lui retourne **ACK**.

Et là, le sens de l'échange s'inverse, et la **RAM** renvoie le contenu de la case mémoire demandée.

Si le maître renvoie **ACK**, la **RAM** est prévue pour envoyer le contenu de la case suivante et ainsi de suite.

Mais quand le maître décide que cet échange est terminé, il renvoie **NOACK** puis génère une **STOP CONDITION**.

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

MOYEN MNEMOTECHNIQUE POUR S'Y RETROUVER !!!

Finalement, les :

↩ START CONDITION,

↩ REPEATED START CONDITION,

↩ STOP CONDITION,

sont **les seuls cas** où la ligne **SDA** peut changer de niveau alors que la ligne **SCL** est au niveau **haut**.

Dans tous les autres cas, **SDA** doit changer de niveau alors que lorsque **SCL** est au niveau **bas** !!!

MODE I²C du PIC (suite)

ORGANISATION DES ECHANGES (suite)

ET LA LIAISON I²C DANS LE PIC ?

LE PIC SOUVENT MAÎTRE,

Souvent, lorsqu'on utilise un **PIC** sur un bus **I²C**, ce sera lui qui en sera le maître « principal » (car il possède, grâce à sa **CPU** et au programme qui le fait fonctionner, une capacité de traitement de l'information) les autres circuits étant alors des esclaves (capteurs et actionneurs divers, circuits périphériques ou mémoires auxiliaires...). L'interface **I²C** du module **MSSP** fonctionnera alors en mode **maître**.

MAIS PAS TOUJOURS !!!

Mais dans le cas de systèmes **multiprocesseurs**, on peut très bien envisager de relier plusieurs **PIC** (ou même des micro/processeurs/contrôleurs différents) entre eux par un bus **I²C**.

Dans ce cas, à certains moments le **PIC** peut être **maître** et à d'autres, il peut être **esclave**.

L'interface **I²C** du module **MSSP** devra donc **aussi** pouvoir fonctionner en mode **esclave** !!!

MODE I²C du PIC (suite)

SYNOPTIQUE SIMPLIFIÉ DU MODULE MSSP DU PIC EN MODE I²C

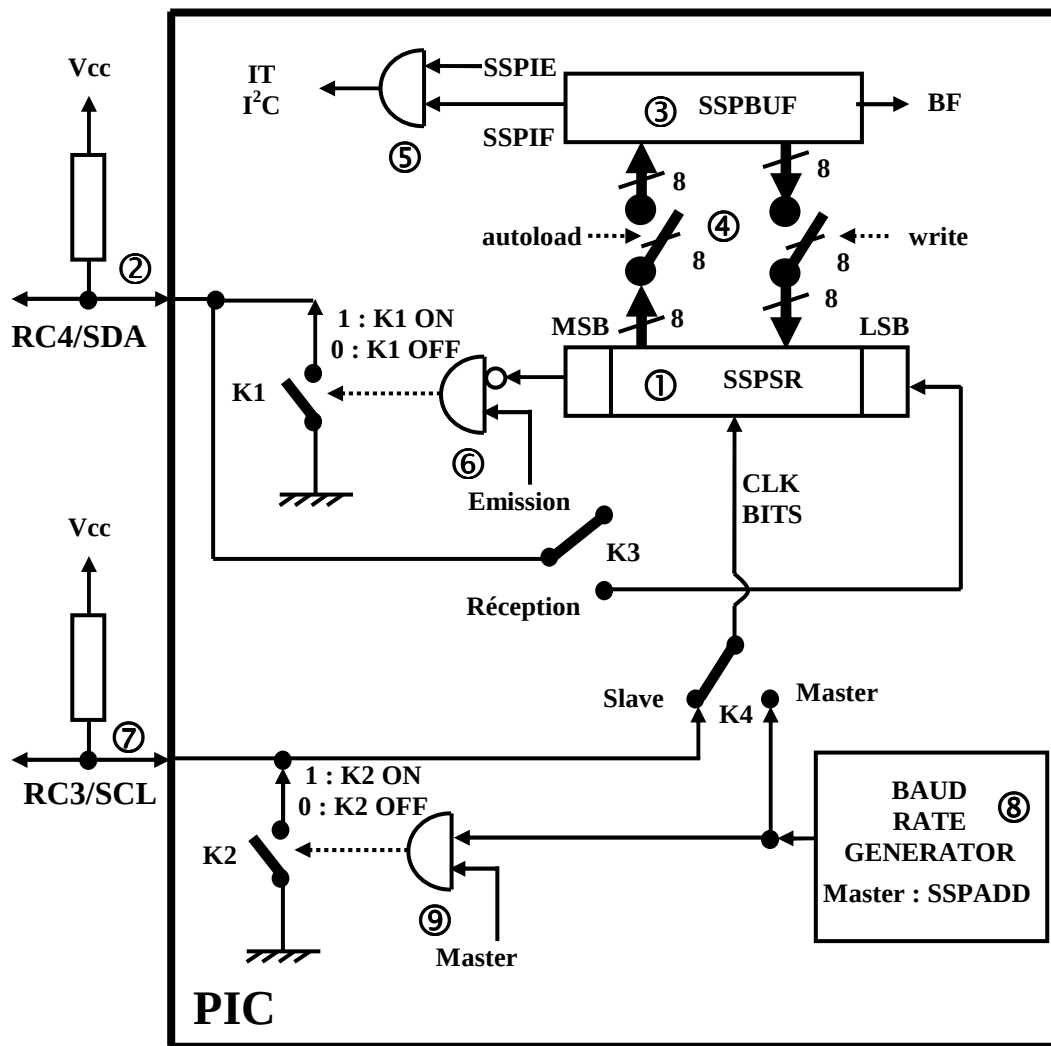


Figure VIII.175. (page 424)

MODE I²C du PIC (suite)

FONCTIONNEMENT SIMPLIFIÉ DU MODULE MSSP DU PIC EN MODE I²C

① registre à **décalage SSPSR** = cœur du module I²C :
contient l'octet à transmettre ou celui qui a été reçu sur la
liaison I²C.

Emission : pilotage de K1

les bits sortent un à un par la broche **RC4/SDA** (cf. ②).
(sortie **MSB** du **SSPSR**).

Réception : K1 ouvert

les bits arrivent aussi par cette même broche **RC4/SDA**.
(entrée **LSB** du **SSPSR**).

Noter également la présence de la résistance de « pull-up »
(à rajouter extérieurement) entre **RC4/RDA** et **Vcc**.

SSPSR, non mappé, n'est pas directement accessible au
programmeur : il faut utiliser le registre tampon **SSPBUF**
③ pour échanger les données avec la liaison I²C.

MODE I²C du PIC (suite)

FONCTIONNEMENT SIMPLIFIÉ DU MODULE MSSP DU PIC EN MODE I²C (suite)

Procédure de transmission d'un octet

Quand l'émetteur (maître ou esclave) écrit un octet dans **SSPBUF** (commande **write** du bloc ④) :

⇒ cet octet est automatiquement écrit dans **SSPSR**,

⇒ le flag **BF** (**B**uffer **F**ull) passe automatiquement à **1** : indiquant que **SSPBUF** est plein et qu'une phase d'émission est en cours.

La transmission peut alors commencer...

A la fin de la transmission de l'octet :

⇒ le flag **BF** repasse automatiquement à **0**,

⇒ le flag **SSPIF** passe à **1**, ce qui peut déclencher une interruption I²C si elle a été localement démasquée par **SSPIE = 1** (cf. ⑤).

Le flag **BF** servira donc, par pooling, à savoir si le registre **SSPBUF** est libre ou pas.

Et si on écrit dans **SSPBUF** alors que **BF** vaut **1**, un autre flag **WCOL** passera à **1**, indiquant une erreur de **COL**lision et l'écriture dans **SSPBUF** ne se fera pas !!!

MODE I²C du PIC (suite)

FONCTIONNEMENT SIMPLIFIÉ DU MODULE MSSP DU PIC EN MODE I²C (suite)

Procédure de réception d'un octet

De même, à la réception d'un octet complet dans **SSPSR** (par le maître ou par l'esclave) :

↪ cet octet est automatiquement transféré dans le registre **SSPBUF** (commande **autoload** du bloc ④)

↪ le flag **BF** passe automatiquement à **1** : indiquant qu'un octet est arrivé et qu'on peut le lire dans **SSPBUF**, ce qui remettra automatiquement **BF** à **0**,

↪ et le flag **SSPIF** passe aussi à **1** ce qui peut également déclencher une interruption I²C si elle a été localement démasquée par **SSPIE** = **1** (cf. ⑤).

Là aussi, le flag **BF** servira, par pooling, à savoir si un octet est complètement arrivé pour pouvoir le lire dans **SSPBUF**.

Et si un nouvel octet arrive alors que **BF** est à **1** (ce qui signifie que le précédent n'a pas été lu) le flag **SSPOV** passera à **1**, indiquant une erreur d'**écrasement** (**OVerflow**) et cet octet sera donc perdu.

MODE I²C du PIC (suite)

FONCTIONNEMENT SIMPLIFIÉ DU MODULE MSSP DU PIC EN MODE I²C (suite)

Horloge SCL

- ↳ **générée** par le maître
 - ↳ **reçue** par l'esclave
- sur la broche **RC3/SCL** (cf. ⑦).

Noter également la présence de la résistance de « pull-up » (à rajouter extérieurement) entre **RC3/SCL** et **Vcc**.

En mode maître

- ↳ Le signal d'horloge de décalage des bits de **SSPSR** provient du bloc **BAUD RATE GENERATEUR** référencé ⑧ et utilisé uniquement en mode maître.
- ↳ Il se retrouve aussi sur la broche **RC3/SCL** pour tous les esclaves reliés au bus.
- ↳ Sa fréquence, et donc la vitesse de transfert sur la liaison **I²C** sera obtenue, en mode **maître** grâce à un registre **SSPADD**.

En mode esclave

L'interrupteur **K2** est ouvert et l'horloge de décalage des bits provient alors de la broche d'entrée **RC3/SCL**.

MODE I²C du PIC (suite)

REGISTRES DE CONFIGURATION/CONTRÔLE

Cinq principaux registres de configuration et de contrôle :

- ① **SSPBUF** (déjà vu pour le mode **SPI**),
- ② **SSPCON** (déjà vu pour le mode **SPI**),
- ③ **SSPSTAT** (déjà vu pour le mode **SPI**),
- ④ **SSPCON2** (spécifique au mode **I²C**),
- ⑤ **SSPADD** (spécifique au mode **I²C**).

REMARQUE PRELIMINAIRE IMPORTANTE :

Pour utiliser le module **MSSP** en mode **I²C**, il faut programmer au départ les lignes **RC3/SCL** et **RC4/SDA en entrée** (via bien sûr le registre **TRISC**).

Le module **MSSP** les mettra **automatiquement** en sortie quand ce sera nécessaire (en fonction du mode maître ou esclave programmé, suivant le sens de l'échange...).

Et c'est là une des principales difficultés quand on travaille en mode **I²C** :

savoir et bien comprendre ce qui est généré **automatiquement** par le module **MSSP** du PIC dans ce mode de fonctionnement et ce qui relève du programmeur via le programme pilotant le bus **I²C**.

MODE I²C du PIC (suite)

REGISTRES EN MODE MAÎTRE

D'abord bien sûr le registre :

① **SSPBUF** qui contient comme on l'a déjà dit :

↪ **l'octet à émettre** : si le maître est en **transmission**,

ou

↪ **l'octet reçu** : si le maître est en **réception**,

Résumé du cours PIC : chapitre VIII.8.2.
(informations supplémentaires pour des projets)

MODE I²C du PIC (suite)

REGISTRES EN MODE MAÎTRE (suite)

② le registre **SSPCON** (déjà vu dans le mode **SPI**)

SSPCON (situé à l'adresse 0x014)
(page 82 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.176. (page 428)

Bit	Valeur	Signification et commentaires
WCOL		W rite C OLLision detect bit
	0	Pas de collision
	1	Ecriture dans SSPBUF pendant un transfert en cours (remis à 0 par soft par WCOL = 0)
SSPOV		O Verflow detect bit : écrasement d'octet à la réception
	0	Pas d'overflow
	1	Un octet est reçu alors que le précédent n'a pas été lu dans SSPBUF (remis à 0 par soft par SSPOV = 0)
Remarque : SSPOV n'est pas utilisé en transmission		
SSPEN		V alidation de la liaison I²C
	0	I²C non utilisée : broches concernées en I/O classiques
	1	I²C validée : broches SDA et SCL affectées à l' I²C
Remarque : direction des lignes (input) concernées à faire avant de faire SSPEN à 1		
CKP		Non utilisé en mode maître
SSPM3:SSPM0		Différents modes de fonctionnement de la liaison I²C
	1000	Mode maître : $F = F_{OSC}/(4*(SSPAD+1))$ Pour fixer la fréquence (la vitesse) de la liaison I²C

Figure VIII.177. (page 428)

Résumé du cours PIC : chapitre VIII.8.2. (informations supplémentaires pour des projets)

MODE I²C du PIC (suite)

REGISTRES EN MODE MAÎTRE (suite)

③ le registre **SSPSTAT** (déjà vu dans le mode SPI)

SSPSTAT (situé à l'adresse 0x094)

(page 81 du datasheet)

	R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SSPSTAT	SMP	CKE	D/ $\bar{A}$	P	S	R/ $\bar{W}$	UA	BF
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
SMP		Contrôle du slew rate de l'horloge
	0	pour F = 400 kHz
	1	pour F = 100 kHz et 1 MHz
CKE		Contrôle des niveaux de tension en entrée
	0	compatibilité I ² C
	1	compatibilité SMBUS
D/$\bar{A}$		Non utilisé en mode maître
P		Indique la détection d'une STOP CONDITION (libération bus)
	0	pas de détection de STOP CONDITION
	1	détection d'une STOP CONDITION
Remis automatiquement à 0 quand une autre condition est détectée		
S		Indique la détection d'une START CONDITION (prise de contrôle bus)
	0	pas de détection de START CONDITION
	1	détection d'une START CONDITION
Remis automatiquement à 0 quand une autre condition est détectée		
R/$\bar{W}$		Indique qu'une transmission du maître est en cours
	0	pas de transmission en cours
	1	transmission en cours
UA		Non utilisé en mode maître
BF		Buffer Full
En transmission :		
	0	pas de transmission en cours ou terminée : SSPBUF est vide
	1	transmission en cours : SSPBUF est plein
En réception :		
	0	réception en cours ou pas de réception : SSPBUF est vide
	1	réception terminée : SSPBUF est plein

Résumé du cours PIC : chapitre VIII.8.2. (informations supplémentaires pour des projets)

MODE I²C du PIC (suite)

REGISTRES EN MODE MAÎTRE (suite)

④ le registre **SSPCON2** (spécifique au mode I²C)

SSPCON2 (situé à l'adresse 0x014)

(page 83 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
GCEN		Non utilisé en mode maître
ACKSTAT		Maître en transmission : image de l'acquittement renvoyé par l'esclave concerné
	0	ACK reçu de l'esclave concerné
	1	NOACK reçu de l'esclave concerné
ACKDT		Maître en réception : valeur du bit d'acquittement renvoyé par le maître vers l'esclave quand la séquence d'acquittement sera initiée par ACKEN=1
	0	le maître renverra ACK
	1	le maître renverra NOACK
ACKEN		Maître en réception : permet de lancer une séquence d'acquittement du maître vers l'esclave et de renvoyer le bit ACKDT
	0	pas de lancement d'une séquence d'acquittement du maître vers l'esclave
	1	lancement d'une séquence d'acquittement du maître vers l'esclave remis automatiquement à 0 par le hardware après l'acquittement
RCEN		pour placer le maître en réception
	0	réception inhibée
	1	passage en mode réception
PEN		pour démarrer une séquence de STOP CONDITION et libérer le bus
	0	pas de lancement d'une séquence de STOP CONDITION
	1	lancement d'une séquence de STOP CONDITION remis automatiquement à 0 par le hardware après STOP CONDITION
RSEN		pour démarrer une séquence de RESTART CONDITION
	0	pas de lancement d'une séquence de RESTART CONDITION
	1	lancement d'une séquence de RESTART CONDITION remis automatiquement à 0 par le hard après RESTART CONDITION
SEN		pour démarrer une séquence de START CONDITION et prendre le bus
	0	pas de lancement d'une séquence de START CONDITION
	1	lancement d'une séquence de START CONDITION remis automatiquement à 0 par le hard après START CONDITION

MODE I²C du PIC (suite)

REGISTRES EN MODE MAÎTRE (suite)

⑤ le registre **SSPADD** (spécifique au mode I²C) n'a pas le même rôle en mode maître qu'en mode esclave.

En mode maître, il permet de définir la fréquence (vitesse) **F** de l'horloge **SCL** générée par le maître du bus I²C suivant la formule :

$$F = \frac{F_{osc}}{4 \cdot (SSPADD + 1)}$$

↪ **F_{osc}** représente la valeur du **quartz** du PIC

↪ **SSPADD** le contenu du registre **SSPADD**.

En pratique, cette formule permet de trouver la valeur à écrire dans le registre **SSPADD** en fonction de la vitesse de communication **F** souhaitée.

Exemple

De façon générale, les vitesses sont normalisées :

100 kHz, 400 kHz et 1 MHz.

Avec un quartz à **F_{osc} = 4 MHz** et pour travailler à :

F = 100 kHz

(durée **T** d'un bit sur le bus I²C sera de $T = \frac{1}{F} = 10 \mu s$ et donc une vitesse de **100 kbits/s**), on aura :

$$SSPADD = \frac{F_{osc}}{4 \cdot F} - 1 = 9$$

MODE I²C du PIC (suite)

FONCTIONNEMENT EN MODE MAÎTRE

En mode maître, **sept actions** possibles à programmer :

- ① générer une **START CONDITION** pour la prise de contrôle du bus
(après avoir vérifié éventuellement que le bus est libre),
- ② générer une **RESTART (ou REPEATED START) CONDITION** pour une inversion du sens de l'échange avec l'esclave concerné sans libérer le bus,
- ③ générer une **STOP CONDITION** pour signaler la fin de l'échange en cours et libérer le bus,
- ④ émettre une adresse ou une donnée sur le bus,
- ⑤ gérer l'acquittement (**ACK** ou **NOACK**) renvoyé par l'esclave concerné,
- ⑥ passer en mode réception et recevoir une donnée venant de l'esclave concerné,
- ⑦ générer un acquittement (**ACK** ou **NOACK**) pour accuser réception de la donnée émanant de l'esclave concerné.

MODE I²C du PIC (suite)

MAÎTRE EN TRANSMISSION

L'examen des chronogrammes de la figure VIII.182. du polycopié de cours complet pour un **maître en transmission** permet de proposer un organigramme possible de traitement :

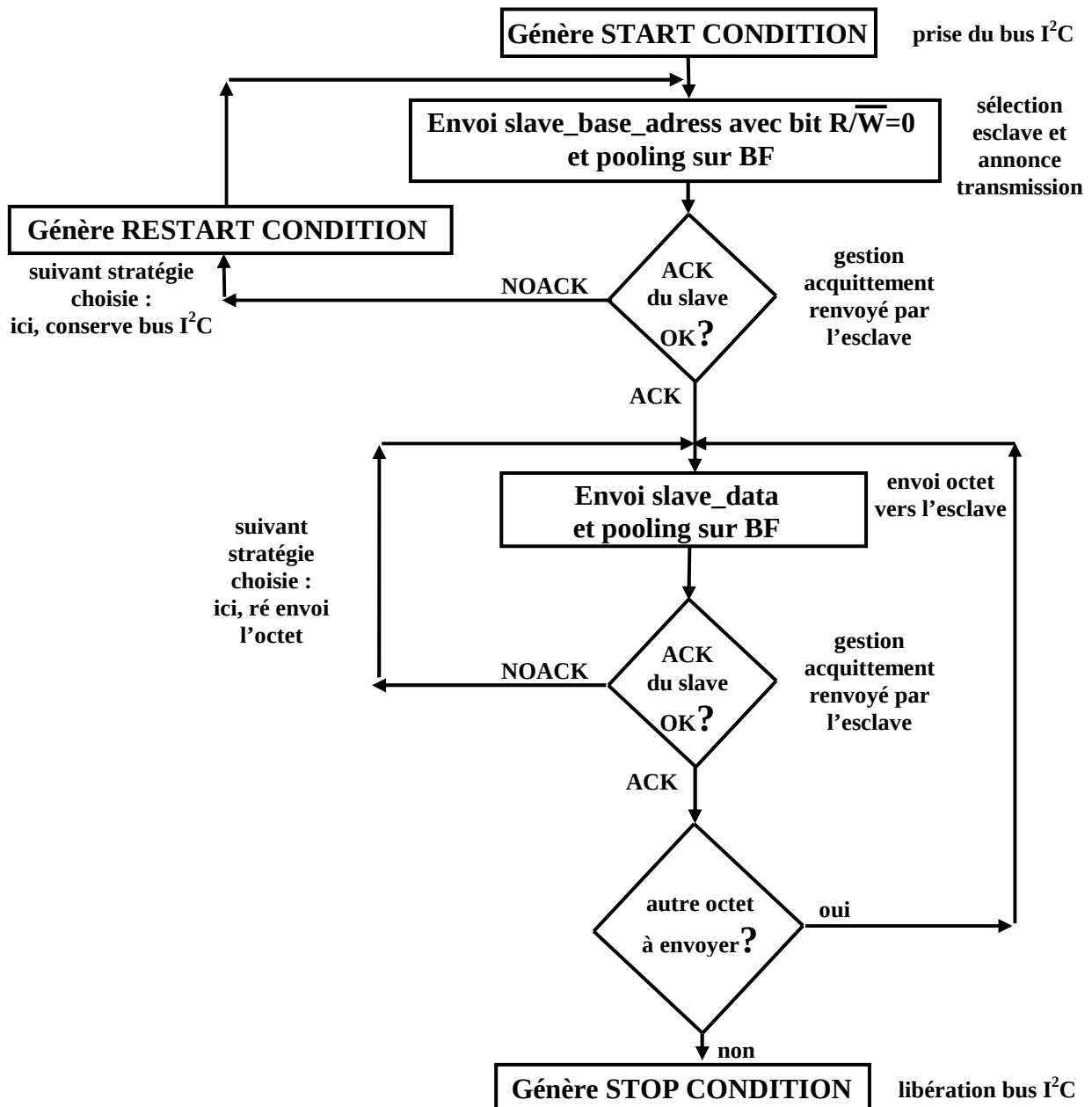


Figure VIII.183. (page 434)

MODE I²C du PIC (suite)

MAÎTRE EN RECEPTION

L'examen des chronogrammes de la figure VIII.184. du polycopié de cours complet pour un **maître en réception** permet de proposer un organigramme possible de traitement :

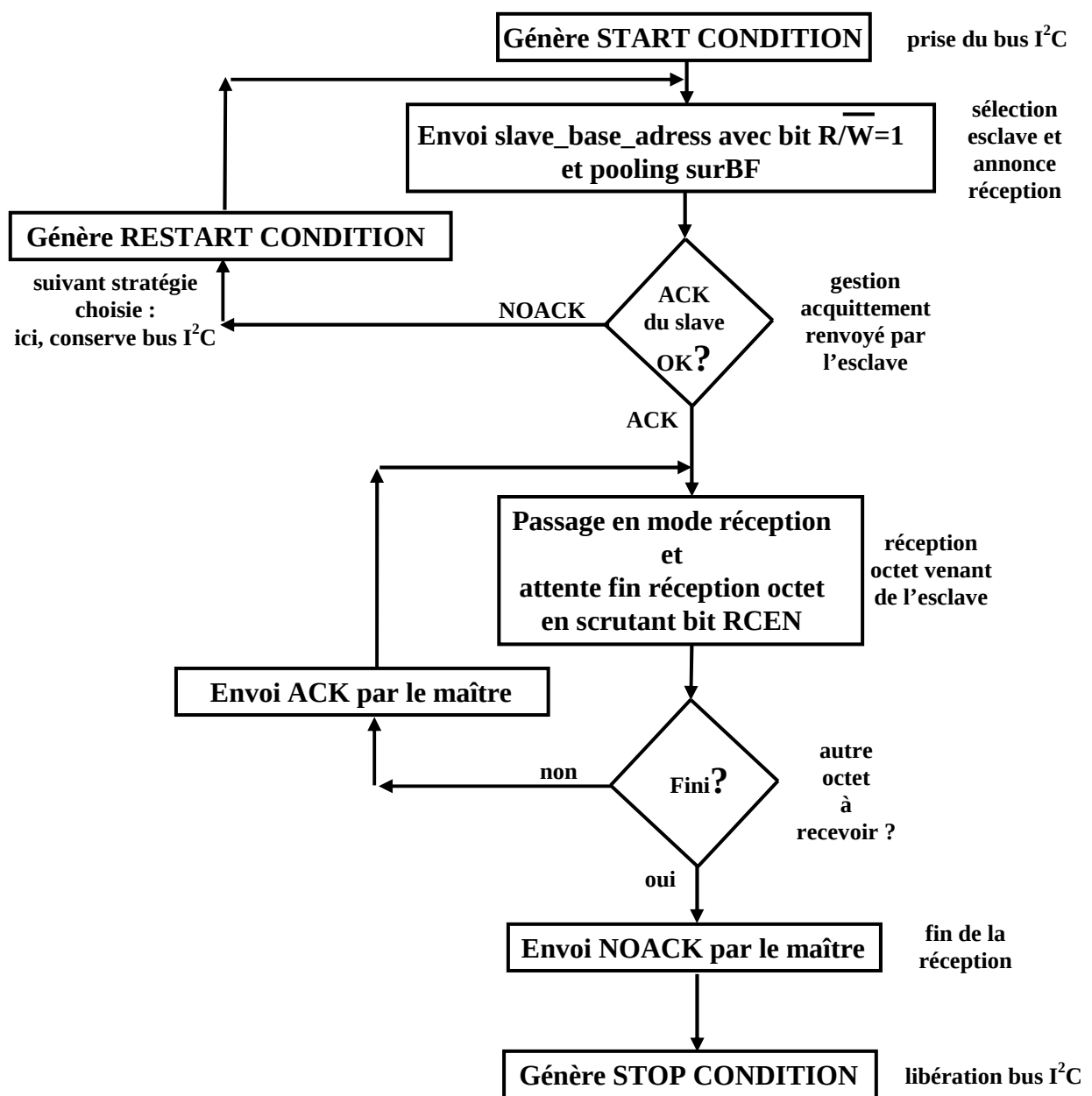


Figure VIII.185. (page 436)

MODE I²C du PIC (suite)

REGISTRES ET FONCTIONNEMENT EN MODE ESCLAVE

Ce sont les mêmes que pour le mode maître, mais la signification de certains bits ou champs de bits est différente dans le mode esclave.

En pratique, comme le PIC est souvent utilisé en maître, nous ne présenterons pas le mode esclave dans ce résumé.

Si vous êtes toutefois intéressé, reportez vous au polycopié complet de cours et au chapitre **VIII.8.2.4.**

ELEMENTS SUR L'IMPLANTATION D'UNE MEMOIRE I²C

Mémoire de référence **24LC256**
(taille **256 Kbits**, soit **32 Koctets**).

Implantation sur carte PICDEM 2 PLUS

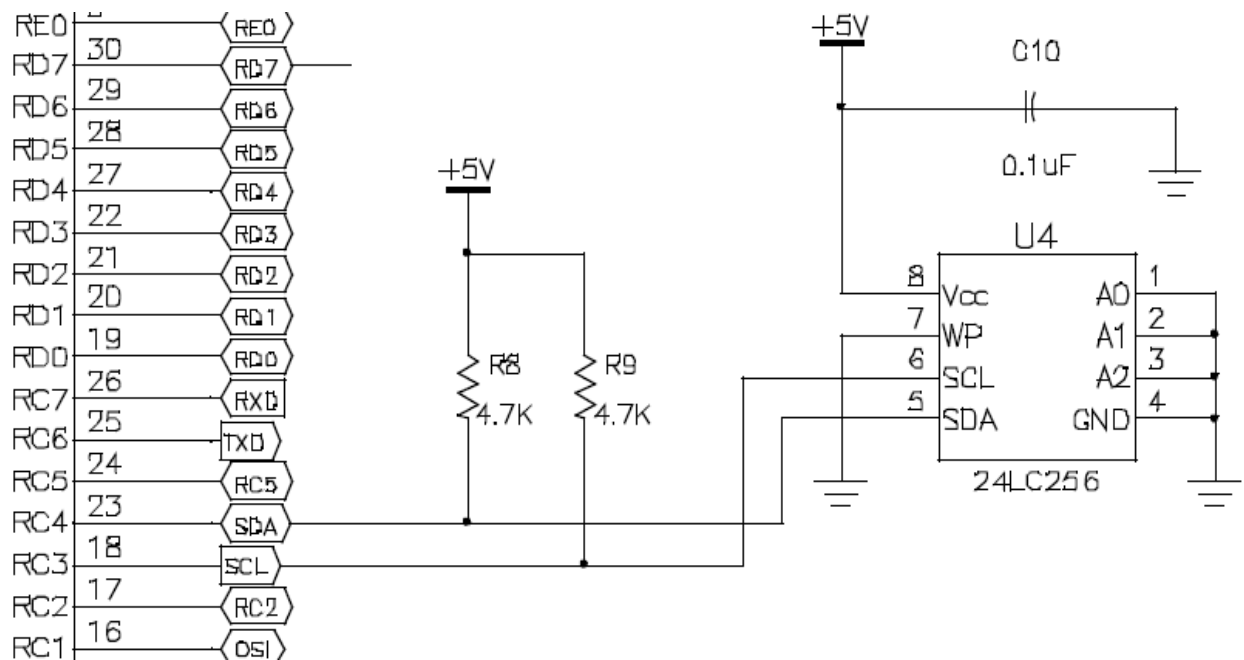


Figure VIII.197. (page 449)

L'entrée **WP** (**W**rite **P**rotect) ici mise à **0** par la masse :

➡ cette mémoire n'a ici **aucune protection** en écriture.

Cette mémoire fonctionne toujours en tant qu'**esclave** :

➡ le module **MSSP** sera donc initialisé en **mode maître**

Choix de la vitesse de la liaison :

➡ on choisit un débit de **100 kbit/s** et donc fréquence
F de SCL = 100 kHz.

ELEMENTS SUR L'IMPLANTATION D'UNE MEMOIRE I²C (suite)

ADRESSAGE DE LA MEMOIRE

D'après la documentation, l'adresse de base pour l'EEPROM (adresse de sélection sur le bus I²C à laquelle l'EEPROM est sensible) est constituée du :

CONTROL BYTE

apparaissant sur la figure VIII.198.

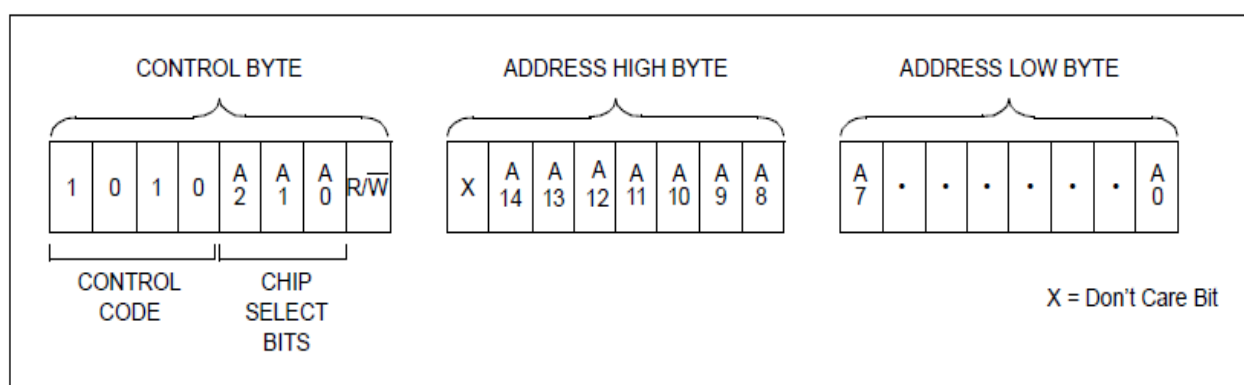


Figure VIII.198. (page 450)

Les valeurs des bits **A2**, **A1** et **A0** y apparaissant correspondent aux valeurs des tensions qui ont été appliquées sur les pattes d'entrées **A2**, **A1** et **A0** de l'EEPROM :

comme sur la carte **PICDEM 2 PLUS**, elles sont reliées à la masse, il faudra prendre : **A2 = 0**, **A1 = 0** et **A0 = 0**.

Les deux octets qui suivent **ADDRESS BYTE HIGH** et **ADDRESS BYTE LOW** permettent de sélectionner une adresse dans l'EEPROM.

ELEMENTS SUR L'IMPLANTATION D'UNE MEMOIRE I²C (suite)

ECRITURE DANS LA MEMOIRE

2 modes d'écriture (ou de programmation) de l'EEPROM :

- le mode « **octet par octet** »,
- le mode par « **page** ».

Nous nous intéresserons au mode « **octet par octet** », dont la séquence d'écriture est donnée sur la figure VIII.199. suivante :

FIGURE 6-1: BYTE WRITE

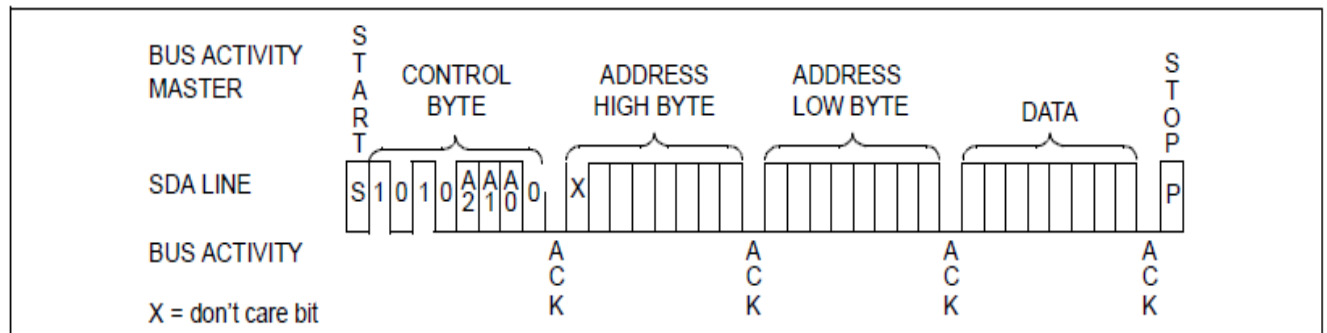


Figure VIII.199. (page 450)

La valeur **DATA** y apparaissant sera écrite à l'adresse sélectionnée par les deux octets **ADDRESS BYTE HIGH** et **ADDRESS BYTE LOW**.

ELEMENTS SUR L'IMPLANTATION D'UNE MEMOIRE I²C (suite)

LECTURE DE LA MEMOIRE

3 modes de lecture de l'EEPROM :

- le mode « **adresse courante** »,
- le mode « **aléatoire** »,
- le mode « **séquentiel** ».

Nous nous intéresserons au mode « **aléatoire** », dont la séquence de lecture est donnée sur la figure VIII.200. suivante :

FIGURE 8-2: RANDOM READ

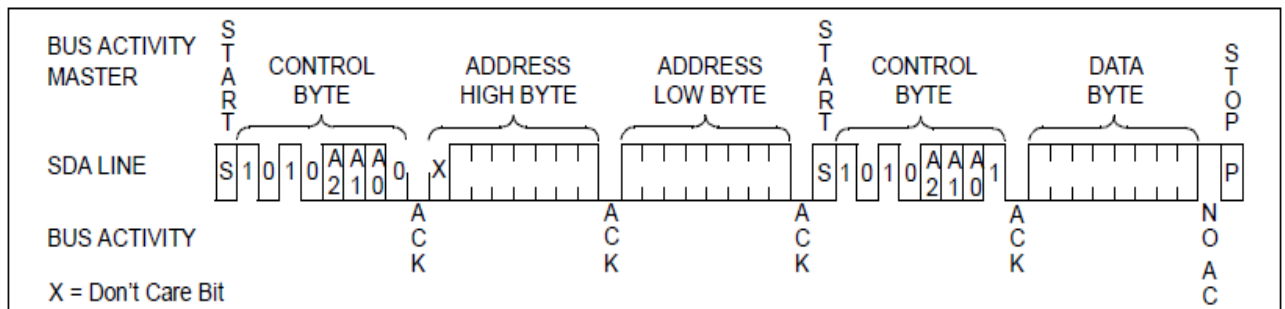


Figure VIII.200. (page 451)

La valeur **DATA BYTE** y apparaissant correspond au contenu (renvoyé par l'EEPROM) de l'adresse sélectionnée par les deux octets **ADDRESS BYTE HIGH** et **ADDRESS BYTE LOW**.

ELEMENTS SUR L'IMPLANTATION D'UNE MEMOIRE I²C (suite)

PROGRAMME DE PILOTAGE DE L'EEPROM

Les différentes fonctions (initialisation, écriture, lecture...)

↳ de bas niveau (c'est-à-dire proche du hardware),

↳ ou de niveau plus élevé (c'est-à-dire proche de l'utilisation)

permettant de mettre en œuvre la liaison I²C sont détaillées dans le polycopié complet de cours, entre les pages 452 à 458.

Jetez y un coup d'œil et lisez bien les commentaires pour vérifier si vous avez bien compris jusque là ce résumé.

Ces fonctions (et vous pouvez le vérifier !!!) sont conformes aux organigrammes donnés sur les figures VIII.183. et VIII.185. précédentes et également sur les séquences constructeur fournies sur les figures VIII.199. et VIII.200.

**N'hésitez pas à vous en inspirer
et à suivre les conseils fournis !!!**

SOMMAIRE

ANNEXE 1 : LCD carte PICDEM 2 PLUS ...	002
ANNEXE 2 : Clavier matriciel	014
ANNEXE 3 : LCD graphique 1	027
ANNEXE 4 : Emetteur/Récepteur Ultrasons .	041
ANNEXE 5 : Moteur pas à pas	047
ANNEXE 6 : Reconnaissance vocale	072
ANNEXE 7 : LCD graphique 2	085
ANNEXE 8 : Servomoteur	103
ANNEXE 9 : Récepteur GPS	119
Résumé bus SPI	141
Résumé bus I²C	157
ANNEXE 10 : Mémoire I²C	189